The Complete Javascript Course

Section 1: Welcome, Welcome, Welcome!

Course Structure and Projects

1. Knowing nothing about JS to expert JS developer
2. The best way to learn JS
3. Overview:
    1. 20 sections - 60 hours
    2. 2 - 3: Fundamentals (solid foundations)
    3. 5 - Developer skills
    4. 7 - DOM Manipulation - 3 simple applications
    5. 8 - How JavaScript works
    6. 9 - Modern operators (ES6+)
    7. 10 - Functions (strings, closures, ...)
    8. 11 - Arrays
    9. 12 - Numbers, Dates, Timers
    10. 13 - Advanced DOM
        1. Lazy image loading
        2. Tabed component
        3. Slider
        4. ...
    11. 14 - Object-Oriented JS (ES6 classes, constructors)
    12. 15 - MAPTY Project
        1. Planning & structuring
    13. 16 - Asynchronous JS
        1. AJAX
    14. 17 - Modern JS Applications
    15. 18 - Forkify Project
    16. 19 - Deployment and Git
        1. NPM
        2. Babel
        3. Parcel
        4. ES6 modules
        5. Netlify

Read Before You Start!

1. Download starter files and final code from GitHub repo:
    1. https://github.com/jonasschmedtmann/complete-javascript-course
2. Download course material:
    1. course-material-v2.zip
3. Friendly community with students: Updates as well
    1. https://discord.gg/uhMkpf4
4. Online tools

Watch Before You Start!

1. Code along
    1. Just watching will not help learn
2. Try all coding challenges
    1. Try to understand the lectures and solve if not able to
3. Understand section by section
    1. Practice
    2. Understand
    3. Move on
4. Try to solve questions
5. Ask questions
6. Have fun

Setting Up Our Code Editor

1. Code editor:
    1. VS Code - best code editor that exists
        1. Alternatives:
            1. Atom
            2. Brackets
        2. Open VS Code
            1. Cog
                1. Color Theme
                    1. Monokai Pro
                        1. Extensions
                            1. Search: Monokai Pro
                                1. Download - paid theme
                    2. Monokai Classic - free (good theme)
                2. Settings
                    1. Auto Save: OnFocusChange (when we move away, it will save)
                    2. Font Size
                    3. Multi Cursor Modifier: ctrl/Cmd
                    4. Work Wrap: On
                    5. Format On Save: On
                        1. We need to install formatter

3. File Icon Theme
    1. Seti
2. Install latest version of Google chrome

Section 2: JavaScript Fundaments - Part 1

Section Intro

1. Fundamentals:
    1. Variables
    2. Data types
    3. Operators
    4. If/Else statements
    5. ...

Hello World!

1. Open browser:
    1. Ctrl+Alt+J
        1. Console
    2. Right click > Inspect > Console
    3. View > Developer > Javascript Console
2. Console:

```
alert("Hello World!") // opens popup window
let js = 'amazing'
if (js === 'amazing') alert('JavaScript is FUN!')
js = 'boring'
if (js === 'amazing') alert('JavaScript is FUN!')
// hit up arrow to go to previous commands

// Math
40 + 8 + 23 - 10
```

A Brief Introduction to JavaScript

1. Section: Javascript Fundamentals - Part 1
2. What is Javascript?
    1. Javascript is a high-level, object-oriented, multi-paradigm, programming language
    2. Programming language: It is a tool that allows us to write code that will instruct a computer to do something
    3. High-level language: We don't have to worry about complex stuff like memory management
        1. The language has abstractions over the small details that we don't have to worry about
            1. It makes the language easy to write and learn
    4. Object-Oriented language: It is mostly based on objects, for storing most kinds of data
    5. Multi-Paradigm language: It is very flexible and versatile and we can use different programming styles (imperative, declarative)
        1. Ways of structuring our code
3. The role of Javascript in web development
    1. HTML, CSS, and JS work together for creation of beautiful, interactive, and dynamic websites / web-applications
        1. HTML - responsible for content of the page
            1. Text
            2. Images
            3. Buttons
            4. ...
        2. CSS - responsible for the presentation of the content
            1. Styling
            2. Layout
        3. JS - real programming language: for building web applications - allows developers to:
            1. Add dynamic, interactive effects to web-page
            2. Manipulate content or CSS
            3. Load data from remote servers
                1. Build entire applications
    2. Roles:
        1. HTML - Nouns
            1. p - paragraph (noun)

```
<p></p>
```

        2. CSS - Adjectives
            1. describes the noun
            2. paragraph is red (adjective)

```
p {color: red}
```

        3. JS - Verbs
            1. Hide (verb) the paragraph - we are doing something

```
p.hide();
```

4. Example: Profile page in Twitter
    1. Rotating spinners
        1. JS might be loading some data
        2. JS hides spinners and displays loaded content when data is available
    2. Modal
        1. JS opens a dialog when we hit the tweet button
        2. JS hides the dialog when we click outside
    3. Display user info
        1. JS shows user info if we hover on any user
    4. Anything that is required can be loaded on the fly
5. Recap:
    1. Add dynamic effects to pages & entire web applications in browser
    2. React/Angular/View - tools that make writing modern, large scale web applications easier
        1. They are based on JS
            1. **We need to be really good at JS before jumping into any framework** (not just after writing 10 lines of JS)
    3. JS and Web browser are two different things
        1. JS can be used on web-server - Node.js
            1. Backend applications
                1. Applications that run on web-servers and interact with databases etc.
    4. What we will learn
        1. Ins and outs of JS
        2. How to use JS in web-browser (front-end web applications)
        3. Node.js
    5. We can use JS to build:
        1. Native mobile applications
            1. React Native
            2. Ionic
        2. Native desktop applications
            1. Electron
6. JS releases/versions:
    1. ES5
    2. ES6/ES2015
        1. Biggest update to the language ever
    3. ES7/ES2016
    4. ES8/ES2017
    5. ES9/ES2018
    6. ES10/ES2019
    7. ES11/ES2020
    8. (New update to JS every single year)
        1. The course will cover modern JS

Linking a JavaScript File

1. How to write JS in a file and run the code
    1. Download starter code from repo
        1. Code > Download ZIP
            1. If not visible:
                1. FAQ: this link
            2. Extract:
                1. Starter code
                    1. Each folder
                        1. final - final version of the code
                        2. starter - project folder
            3. Open project folder in VS Code (starter)
                1. index.html

```
<script>
    let js = 'amazing'; // not mandatory
    if (js === 'amazing') {
        alert('Javascript is FUN!');
    }
</script>
```

                2. Open the file in Google chrome
                    1. Shows an alert
            4. How to see results of our code in browser

```
<script>

    40 + 8 + 23 - 10;
</script>
```

                1. Open inspector
                    1. Console
                        1. Result is not in the console

1. It is a REPL
    1. It doesn't show results of code
2. Solution:

```
console.log(40 + 8 + 23 - 10);
```

Implementation

1. 01-FUNDAMENTALS-PART-1
    1. index.html

```
<script>
    let js = 'amazing'; /* ; tells we are done. It is not mandatory. But it is a good coding practice */
    if (js === 'amazing') {
      alert('Javascript is fun!');
    }

    40 + 8 + 23 - 10; /* Doesn't show anywhere */

    console.log(40 + 8 + 23 - 10); /* Prints on the console */
</script>
```

        1. Maintaining code in separate files is good for maintenability

```
<script src="script.js"></script>
```

Values and Variables

1. Value - a piece of data
    1. The most fundamental unit of information in programming
        1. 'Jonas'

```
console.log('Jonas');
```

2. Variable: Like a box that can hold an object (book or ball)
    1. To re-use a value
    2. Conventions and rules
        1. camelCase - convention
        2. snake_case
    3. A variable cannot start with number
    4. A variable can contain letter, numbers, underscores, dollar ($)
    5. Reserved keywords cannot be used

```
let new = 2; // not allowed
let function = 27; // not allowed
let $function = 27; // allowed
let name = 'Jonas'; // allowed but might cause problems in some cases. Don't use it
```

    6. Don't start with an uppercase letter (convention)
    7. Variables that never change (constants) are written in all uppercase

```
let PI = 3.141592;
```

    8. Use descriptive names (convention)

```
let myFirstJob = 'Programmer';
let myCurrentJob = 'Teacher';
```

3. Errors we make show in console or error icon

Practice Assignments

1. https://codingheroes.io/assignments/javascript-fundamentals-part-1.html

Data Types

1. Values can have different types

Objects and Primitives

1. A value is either an object or a primitive
    1. Object:

```
let me = {
    name: 'Jonas'
}
```

    2. Primitive:

```
let firstName = 'Jonas';
let age = 30;
```

2. There are 7 primitive data types
    1. Number
        1. Floating point numbers
        2. Used for decimals and integers
        3. They always have decimals

        ```
        let age = 23; // same as 23.0
        ```
    2. Boolean
        1. Logical type that can only be `true` or `false`
        2. Used for taking decisions

        ```
        let fullAge = true;
        ```
    3. String
        1. A sequence of characters.
        2. Used for text
        3. Put quotes. Either single quotes or double quotes `'` or `"`
    4. undefined
        1. Value taken by a variable that is not yet defined ("emtpy value")

        ```
        let children; // only declared but not assigned
        ```
    5. null
        1. Also means "empty value"
            1. Used in different circumstances
    6. symbol (ES2015)
        1. Value that is unique and cannot be changed (Not useful for now)
    7. BigInt (ES2020)
        1. Larger integers than the Number type can hold
3. JavaScript has dynamic typing: We do not have to manually define the data type of the value stored in a variable. Instead, data types are determined automatically
    1. **It is the value that has the type and not the variable**
        1. Variables just store values that have a type
            1. We can assign a new value of a different data type to the same variable

            ```
            let x = 24;
            x = 'Jonas'; // allowed
            ```
4. Code commenting:
    1. Single line comment:

    ```
    // Variable name conventions
    // console.log(40 + 8 + 23 - 10);
    ```

        1. ctrl + / (turn on or off comments)
    2. Multiline comments:

    ```
    /*
        This is
        a
        multiline
        comment
    */
    ```

        1. Variable declarations must be before the usage becaus JS executes in sequence
5. `typeof` **(M)** - operator (like +, -, ...)

    ```
    typeof true
    ```
6. Quotes are a must for a string

    ```
    jonas
    ```

    1. `ReferenceError`
7. Dynamic typing:

    ```
    javascriptIsFun = 'YES!'; // `let` is not used
    ```
8. `undefined`: value taken by a variable that is not yet defined

1. Empty value

```
let year;
console.log(year); // undefined
console.log(typeof year); // undefined

year = 1991;
```

let, const, and var

1. Ways to declare variables in JS

let

1. We usually use the `let` keyword to declare variables that can change later (during the execution of the program)

```
let age = 30;
```

    1. Good for declaring on top and assigning later on

const

1. Value in a `const` variable cannot be changed

```
const birthYear = 1991; // It cannot change
birthYear = 1990; // doesn't work
```

    1. Immutable variable
    2. We cannot declare empty `const` variables

```
const job; // doesn't work
```

2. **Use `const` by default but use `let` only if we are sure that the variable value will change during the execution of the program**
    1. It is a good practice to have little variable mutations or changes
        1. Potential to cause bugs

var

1. **This should be completely avoided**
    1. Need to know because it is used in legacy code
2. It is the old way
3. It is similar to `let`

```
var job = 'programmer';
job = 'teacher';
```

4. `let` and `var` are very different under the hood
    1. `var` is **function scoped**
    2. `let` **block scoped**

Variable Declaration

1. It is mandatory to declare variables

```
lastName = 'Schmedtmann';
console.log(lastName);
```

    1. Not a good idea
        1. We did not define the scope
            1. Default: **JS will construct a property in the global scope**
        2. Never write variables without declaring them

Basic Operators

1. Operator: Allows us to transform values or combine multiple values (all with values)
    1. Mathematical operators
    2. Comparison operators
    3. Logical operators
    4. Assignment operators
    5. ...

Mathematical Operators

1. -

```
const ageJonas = 2037 - 1991;
console.log(ageJonas);

const ageSarah = 2037 - 2018;
console.log(ageSarah);
```

    1. Logging multiple values:

```
console.log(ageJonas, ageSarah); // **(M)**
```

2. *, /
3. ** (power)
    1. `2 ** 3` = 2 to the power of 3 = 8
4. Concatenation: +

```
const firstName = "Jonas";
const lastName = "Schmedtmann";
console.log(firstName + lastName);
console.log(firstName, lastName);

console.log(firstName + " " + lastName);
```

5. Assignment operators:

```
let x = 10 + 5;
console.log(x);
x += 10; // 15 + 10 = 25
```

    1. `+=` : `x = x + 10;`
    2. `*=`
    3. `x++`: `x = x + 1;`
    4. `x--`: `x = x - 1;`

6. Comparison operators: used to produce `boolean` values

```
console.log(ageJonas > ageSarah);
console.log(ageSarah >= 18);
```

    1. Variables are available in Console
    2. JS uses operator precedence to perform the calculations

Operator Precedence

1. search: mdn operator presedence (developer.mozilla.org)
    1. **Use MDN**
    2. Just have a general idea as to which operators are executed first
        1. Math operators are executed first as compared to comparison operators
    3. Some operators are executed from left to right and some from right to left
        1. Arithmetic operators - left to right
        2. Assignment operators - right to left

            1. It is one of the lowest in presedence

```
let x, y;
x = y = 25 - 10 - 5;
console.log(x, y); // both are 10
```

                1. - operations are performed first
                2. = are executed next from right to left
                    1. Otherwise

```
x = undefined
y = 10
```

                3. ()

A Note About Challenges

Coding Exercise 1: CHALLENGE #1

CHALLENGE #1: Video Solution

Strings and Template Literals

1. Template literals:

```
const firstName = 'Jonas';
const job = 'teacher';
const birthYear = 1991;

const year = 2037;
const jonas = "I'm " + firstName + ', a ' + (year - birthYear) + ' years old ' + job + '!';
console.log(jonas);
```

    1. Type coersion is used to convert number to a string
    2. Templates string: ``

```
const firstName = "Jonas";
const job = "teacher";
const birthYear = 1991;

const year = 2037;
const jonas =
  "I'm " + firstName + ", a " + (year - birthYear) + " year old " + job + "!";
console.log(jonas);
```

```
const jonasNew = `I'm ${firstName}, a ${year - birthYear} year old ${job}!`;
console.log(jonasNew);

console.log(`Jonas a regular string...`);
```

3. Multi-line string:

```
console.log('String with \n\
multiple \n\
lines');

console.log(`String
multiple
lines`);
```

Taking Decisions: if/else Statements

1. Example:

```
const age = 19;
const isOldEnough = age >= 18;

if (isOldEnough) {
    console.log('Sarah can start driving license <window+.>');
} else {
    const yearsLeft = 18 - age;
    console.log(`Sarah is too young. Wait another ${yearsLeft} years.`);
}
```

    1. `if...else` control structure
        1. Controls blocks of code that execute and blocks that should not execute

Coding Exercise 2: CHALLENGE #2

CHALLENGE #2: Video Solution

Type Conversion and Coercion

1. Converting between types
    1. String into number
    2. Number into boolean
2. JS behaves in a wierd way in some situations
3. Type conversion vs type coersion:
    1. Conversion:
        1. When we manually convert from one to another
        2. Inputs:
            1. Usually come as strings
            2. Conversion:

```
const inputYear = '1991';
console.log(Number(inputYear)); // **(M)**
console.log(Number(inputYear) + 18);
```

                1. Original value is not converted
                    1. `inputYear` is still a string
                2. If we convert a string that cannot be converted:

```
console.log(Number('Jonas')); // NaN (invalid number)

console.log(typeof NaN); // number - it is still a number but it is an invalid number

console.log(String(23)); // **(M)**
```

                3. We cannot convert something to `undefined` or `null`
    2. Coersion:
        1. When JS automatically converts behind the scenes
        2. How is it done?
            1. **If an operator is dealing with two values that have different types**
                1. JS will convert one value into another value
            2. Examples:

```
console.log('I am' + 23 + ' years old');
console.log('23' - '10' - 3); // `-` Converts strings to numbers (opposite)
console.log('23' * '2'); // converted to numbers
console.log('23' / '2'); // converted to numbers
```

                1. How does it work?
                    1. `+` triggers coersion (whenever a string and number is involved, the number will be converted to strings)
                    2. Same with template literals
                2. **Type coersion can introduce bugs if we don't know how it works**
                    1. Advantages:
                        1. Less code
                        2. Readable code

Truthy and Falsy Values

1. Falsy values: They are not `false` but will become `false` when we try to convert them into a boolean
    1. There are only 5 falsy values in JS
        1. 0
        2. ''
        3. undefined
        4. null
        5. NaN
    2. Any other value will be converted into `true`
    3. Examples:

```
console.log(Boolean(0)); // false
console.log(Boolean(undefined)); // false
console.log(Boolean('Jonas')); // true
console.log(Boolean({})); // true
console.log(Boolean('')); // false
```

        1. Coersion can be used:
            1. In two scenarios:
                1. When using logical operators
                2. In a logical context (`if...else` say)

```
const money = 0;
if (money) {
    console.log("Don't spend it all!");
} else {
    console.log('You should get a job!');
}
```

        2. Checking if

Equality Operators: == vs ===

1. Example:

```
const age = 18;
if (age === 18) {
    console.log('You just became an adult');
}
```

    1. `===` - strict equality operator
        1. It doesn't perform type coersion
            1. Both values must exactly be the same
    2. `==` - loose equality operator (it has weird rules and can introduce bugs - avoid this)

```
'18' == 18; // true - type coersion if performed
'18' === 18; // false - no type coersion is performed
```

2. To get inputs: `prompt('<string>')`

```
const favorite = prompt("What's your favorite number?");
console.log(favorite);
```

Boolean Logic

1. A branch of computer science which uses boolean `true` and `false` values for complex logical problems. It uses logical operators.
2. Examples:
    1. A: Sarah has a driver's license (can be true or false)
    2. B: Sarah has good vision (can be true or false)
    3. Combination:
        1. 'Sarah has a driver's license AND good vision'
            1. Truth tables can be used to find out if the combination is `true` or `false`
        2. 'Sarah has a driver's license OR good vision'
            1. `false` only if both are `false`
    4. NOT:
        1. Inverts `true`/`false`
3. Examples:
    1. `age = 16`
        1. Boolean variables:
            1. A: Age is greater or equal to 20 - `false`
            2. B: Age is less than 30 - `true`
        2. NOT:
            1. `!A` - `true`
            2. `A AND B` - `false AND true` = `false`
            3. `A OR B` - `false OR true` = `true`
            4. `!A and B` - `true AND true` = `true`
            5. `A OR !B` - `false OR false` = `false` (`!` has presedence over `OR`)

Logical Operators

1. Example:

```
const hasDriversLicense = true; // A
const hasGoodVision = true; // B
console.log(hasDriversLicense && hasGoodVision);
console.log(hasDriversLicense || hasGoodVision);
console.log(!hasDriversLicense);

const shouldDrive = hasDriversLicense && hasGoodVision;

// if (shouldDrive) {
if (hasDriversLicense && hasGoodVision) {
    console.log('Sarah is able to drive!');
} else {
    console.log('Someone else should drive...');
}

const isTired = true; // C
console.log(hasDriversLicense || hasGoodVision || isTired);
console.log(hasDriversLicense && hasGoodVision && isTired);

if (hasDriversLicense && hasGoodVision && !isTired) {
    console.log('Sarah is able to drive!');
} else {
    console.log('Someone else should drive...');
}
```

Coding Exercise 3: CHALLENGE #3

The switch Statement

1. Example: Activity for each day:

```
const day = 'monday';

switch (day) {
    case 'monday': // day === 'monday'
        console.log('Plan course structure');
        console.log('Go to coding meetup');
        break;
    case 'tuesday':
        console.log('Prepare theory videos');
        break;
    case 'wednesday':
    case 'thursday':
        console.log('Write code examples');
        break;
    case 'friday':
        console.log('Record videos');
        break;
    case 'saturday':
    case 'sunday":
        console.log('Enjoy the weekend!');
    default:
        console.log('Not a valid day!');
}
```

    1. If we don't give `break`, the code continues until it reaches the next break or the end
    2. `switch` does a strict comparison (`===`)
    3. `switch` statement is more readable than `if..else if...` statement
        1. It is not used much these days

Statements and Expressions

1. Expression: It produces a value
    1. `3 + 4`
    2. `1991` (value is an expression)
    3. `true && false && !false`
2. Statement: It does not produce a value on itself
    1. Declarations
        1. Uses expressions (as parts)
    2. Examples:

```
if (23 > 10) { // statement - doesn't produce a value
    const str = '23 is bigger'; // doesn't produce a value - statement
}
```

3. JS expects only statements or expressions in certain places
    1. In template literals, we can only insert expressions but not statements
    2. Example:

```
console.log("I'm ${2037 - 1991} years old ${if (23 > 10) {
    const str = '23 is bigger';
}"); // doesn't work
```

        1. Variable is an expression (it gets replaced with a value)

The Conditional (Ternary) Operator

1. Conditionals:

```
const age = 23;
age >= 18 ? console.log('I like to drink coffee') : console.log('I like to drink milk');
```

1. Only one line of code is allowed in each block
    1. First block is executed if the expression is true
    2. Second block is executed if the expression is false
2. It is called a ternary operator (it has 3 parts)
3. Conditional operator is an operator and it produces a value
    1. Conditional can be an expression
        1. It can return a value which can be assigned to a variable
4. Assigning value conditionally

```
const drink = age >= 18 ? 'coffee' | 'milk';
console.log(`I like to drink ${drink}`);
```

    1. `if...else` is a statement, so it doesn't return a value

```
let drink2;
if (age >= 18) {
    drink2 = 'coffee';
} else {
    drink2 = 'milk';
}
console.log(`I like to drink ${drink2}`);
```

    2. **We can use ternary operator inside a template literal**

```
console.log(`I like to drink ${age >= 18 ? 'coffee' : 'milk'}`); // any expression is accepted
```

        1. Ternary operator is not a replacement
            1. If multiple statements are required in each block

Coding Exercise 4: CHALLENGE #4

CHALLENGE #4: Video Solution

JavaScript Releases: ES5, ES6+ and ESNext

1. JS released / versions
    1. A brief history of JS
        1. Interactive websites were required
            1. Brendan Eich - inventor of JS 1 in 10 days
                1. Called Mocha
            2. Then it was called LiveScript
                1. Then renamed to JS
            3. Microsoft copied JavaScript and called it JScript
            4. ECMA script was released (ES1)
                1. Standard JS
                    1. ECMAScript is the standard
            5. ES5 was released with a lot of features
            6. ES6 (ES2015) was released
                1. Big update
                2. Many new features
            7. A new release every year
                1. Better to keep upto date (instead of big releases)
                2. ES2016, ES2017, ...
2. Backwards compatability: Don't break the Web!
    1. All the way to ES1
    2. If we take the first version of JS and put it in a modern browser, it will work the same
        1. Fundamental feature: Don't break the web
            1. **Old features are never removed**
            2. **Not really new versions, just incremental updates (releases)**
            3. **Websites keep working forever!**
                1. Old websites work even today
                2. Problems:
                    1. There are bugs and wierd things in the older version
3. **Just learn newer versions and ignore the older versions**
4. Forwards compatibility
    1. Modern JavaScript Engine (2020) -> ES2089 Engine
        1. This is not supported
            1. Nothing would work from the future
                1. Reason: The modern JS engines cannot support the future versions of JS
                    1. Problem: How can we use modern JS today? (Users might be using older versions of JS engines)

How to Use Modern JS Today

1. Two distinct scenarios
    1. **Development**
        1. **During development**: Simply use the latest Google Chrome!
            1. Ensures that all the features will work for us
    2. **Production**
        1. **During production**: Use **Babel** to transpile and polyfill your code (converting back to ES5 to ensure browser compatibility for all users)
            1. We can't control what versions of browsers the users will use
            2. It is only required when we ship the app to the users

2. ES5
    1. Fully supported in all browsers (down to IE 9 from 2011)
        1. The reason it is used as a target for transpilation
    2. Ready to be used today
3. ES6/2015 -> ES2020
    1. ES6+: Well supported in all **modern** browsers
    2. No support in **older** browsers
    3. **Can use most features in production with transpiling and polyfilling**
    4. To check what features of ES6+ are supported by the modern browsers:
        1. http://kangax.github.io/compat-table
4. ES2021 - infinity
    1. **ESNext**: Future versions of the language (new feature proposals that reach stage 4);
    2. Can already use **some** features in production with transpiling and polyfilling
        1. Most browsers implement the new features even before they enter the ECMAScript spec
            1. Why?
                1. If new features are proposed, they go through 4 stages
                    1. Stage 1: Submitted
                    2. State 2: ...
                    3. Stage 3: Browsers make sure the features will eventually pass to stage 4
                        1. Browsers implement the feature while it is in stage 3
                    4. Stage 4: Enters the language officially
                        1. Check compatibility table

Modern Javascript From the Beginning

1. Learn **modern JavaScript from the beginning!**
2. But, also learn how some things used to be done **before** modern S (e.g. `const` & `let` vs `var` and function constructors vs ES6 `class`)
    1. `class` - hides implementation details of OOP in JS
3. **3 reasons why we should not forget the Good old JS**
    1. You will better understand how JS actually works
    2. Many tutorials and code you find online today are still in ES5
    3. When working on old codebases, these will be written in ES5

Section 4: JavaScript Fundamentals - Part 2

Section Info

1. JS fundamentals:
    1. Functions
    2. Objects
    3. Arrays
    4. Loops

Activating Strict Mode

1. 02-Fundaments-Part-2 folder
    1. script.js

```
'use strict';
```

        1. Makes it easier for us to write secure javascript code
        2. It must be the very first statement
            1. If we have any code before the statement, strict mode doesn't work
            2. Comments are allowed
        3. We can activate strict mode only for a code block or a function
            1. No use of using that feature
        4. Secure code:
            1. Makes is easier for developers accidental errors (introduction of bugs)
                1. Why?
                    1. Forbids us to do certain things
                    2. Shows visible errors in some situations
                        1. Or else JS will simply fail silently
            2. Example:

```
"use strict"; // catches the issue

let hasDriversLicense = false;
const passTest = true;

if (passTest) hasDriverLicense = true;
if (hasDriversLicense) console.log("I can drive");
```

            3. It has some reserved words that might be added to the language in the future

```
const interface = "Audio"; // unexpected strict mode reserved mode
const private = 545; // doesn't work
```

Functions

1. Fundamental build blocks
    1. Most essential concept
2. What is a function?

1. A piece of code that we can re-use over and over in the code
    1. A function holds one or more lines of code

3. Examples:

```
function logger() {
  console.log("My name is Jonas");
}

logger(); // invoking/running/calling the function
logger();
logger();
```

4. Passing data into a function and it can return data

    1. Functions are like machines
        1. We can put food into a processor
        2. The processor does something
        3. The processor returns something back (juice, say)

5. We can pass arguments to a function that doesn't accept parameters

```
function logger() {
    // ...
}

logger(23);
```

    1. Functions that do not return anything produce `undefined`
    2. DRY - Keep the code DRY
    3. `console.log()` - a function
    4. `Number('23')` - another function

Function Declarations vs. Expressions

1. There are different ways of writing functions
2. Function declaration:

    1. `function` keyword is used - similar to variable declaration

    2. Example:

```
// Function declaration
function calcAge1(birthYear) {
    const age = 2037 - birthYear;
    return age;
}

const age1 = calcAge1(1991);
console.log(age1);

// Function expression
const calcAge2 = function (birthYear) {
    return 2037 - birthYear;
}

const age2 = calcAge2(1991);

console.log(age1, age2);
```

        1. Function without a name (anonymous function)

        2. It is an expression that produces a value

        3. The value is the function

        4. Function expressions are required in some scenarios

        5. **In JS, functions are just values (similar to number, string, or boolean)**

            1. It is not a type but a value (which can be stored in a variable)

        6. **A function can be called before it is declared**

```
const age1 = calcAge1(1991);
function calcAge1(birthYear) {
    return 2037 - birthYear;
}
```

            1. Not possible with expressions
                1. Why? Due to hoisting

        7. Which type of functions should we use?

            1. It is a matter of personal preference
                1. Different devs prefer different formats
                    1. Function expressions:
                        1. Forces us to defined all functions first before calling them (more structured code)
                        2. We can store everything in variables
            2. Both are required in different situations

Arrow Functions

1. Third type of function added to ES6

1. Arrow function

2. Arrow function

    1. A special form of **function expression** that shorter and faster to write

    2. Example:

        ```
        const calcAge3 = birthYear => 2037 - birthYear;
        const age3 = calcAge3(1991);
        console.log(age3);
        ```

        1. We don't need curly braces
        2. Return happens implicitly
            1. The value in the body is returned automatically
                1. Applicable only for one line functions
        3. If the code and parameters become complex, we might lose the advantage of using arrow functions

    3. **Arrow functions do not get the `this` keyword**

    4. Example:

        ```
        const calcAge3 = (birthYear) => 2037 - birthYear;
        const age3 = calcAge3(1991);
        console.log(age3);

        const yearsUntilRetirement = (birthYear, firstName) => {
          const age = 2024 - birthYear;
          const retirement = 58 - age;
          // return retirement;
          return `${firstName} retires in ${retirement} years`;
        };

        console.log(yearsUntilRetirement(1991, "Jonas"));
        console.log(yearsUntilRetirement(1980, "Bob"));
        console.log(yearsUntilRetirement(1985, "Abdullah"));
        ```

Functions Calling Other Functions

1. Example:

    ```
    function cutFruitPieces(fruit) {
        return fruit * 4;
    }

    function fruitProcessor(apples, oranges) {
        const applePieces = cutFruitPieces(apples);
        const orangePieces = cutFruitPieces(oranges);

        const juice = `Juice with ${applePieces} pieces of apple and ${orangePieces) pieces of orange.`;
        return juice;
    }

    fruitProcessor(2, 3);
    ```

Reviewing Functions

1. Example:

    ```
    const calcAge = function (birthYear) {
        return 2037 - birthYear;
    }

    const yearsUntilRetirement = function (birthYear, firstName) {
        const age = calcAge(birthYear);
        const retirement = 65 - age;

        if (retirement > 0) {
            console.log(`${firstName} retires in ${retirement} years`);
            return retirement;
        } else {
            console.log(`${firstName} has already retired`);
            return -1; // Any number that shows that it has no meaning
        }
    }

    console.log(yearsUntilRetirement(1991, 'Jonas'));
    console.log(yearsUntilRetirement(1950, 'Mike'));
    ```

    1. `return` exits the function
        1. It must be the last statement in a code block or else the rest of the code will not get executed

2. Review:

    1. Function declaration:
        1. Function that can be used before it's declared
    2. Function expression:
        1. Essentially a function value stored in a variable
    3. Arrow function
        1. Great for quick one-line function. Has no `this` keyword (more later...)

3. Three different ways of writing functions, but they all work in a similar way: receive input data, tranform data, and then output data.

4. Structure:

    1. Function name

2. Parameters: placeholders to receive input values. Like local variables of a function
3. Function body: block of code that we want to reuse. Processes the function's input data
4. Return statement to output a value from the function and terminate execution
5. Calling, running or invoking the function, using `()`
6. Arguments: actual values of function parameters, to input data
7. Variable to save returned value (function output)
8. `console.log` another function call inside a function

Coding Exercise 5: CHALLENGE #1

CHALLENGE #1: Video Solution

Introduction to Arrays

1. How do we represent 10 friends
    1. We can use arrays
2. Example:

```
const friends = ['Michael', 'Steven', 'Peter']; // literal syntax
console.log(friends);
```

3. Another way:

```
const years = new Array(1991, 1984, 2008, 2020);

console.log(friends[0]); // 0-based
console.log(friends[2]);

console.log(friends.length);
console.log(friends[friends.length - 1]); // any expression is allowed

friends[2] = "Jay";

console.log(friends);
```

    1. An array is not a primitive value
        1. It can be changed even if it is `const`
        2. However, we cannot change the entire array

```
friends = ['Bob', 'Alice'];
```

    2. An array can hold values of different types
        1. In each position JS expects an expression
    3. We can put other arrays inside an array

```
const friends = ['Bob', 'Alice'];
const jonas = [firstName, 'Schmedtmann', 2037 - 1991, 'teacher', friends];
```

Basic Array Operations (Methods)

```
const newLength = friends.push("Jay"); // **(M)** It is a member of the array
console.log(friends);
console.log(newLength);

friends.unshift("John"); // **(M)**

// Remove elements
friends.pop(); // **(M)** - removes the last element
const popped = friends.pop();
console.log(popped);
console.log(friends);

friends.shift(); // First element
console.log(friends);

console.log(friends.indexOf("Steven"));
console.log(friends.indexOf("Bob"));

console.log(friends.includes("Steven")); // ES6 method, uses strict equality

if (friends.includes('Steven')) {
    console.log('You have a friend called Steven');
}
```

Coding Exercise 6: CHALLENGE #2

CHALLENGE #2: Video Solution

Introduction to Objects

1. In arrays, we cannot reference elements by name, but by index values
    1. Solution: Objects
        1. Objects store key-value pairs

```
const jonas = {
    firstName: 'Jonas',
```

```
    lastName: 'Schmedtmann',
    age: 2037 - 1991,
    job: 'teacher',
    friends: [
        'Michael',
        'Peter',
        'Steven'
    ]
};
```

1. Objects are the most fundamental concept in JS
2. The above is called object literal syntax
3. Use objects for unstructured data
4. Use arrays for ordered data

Dot vs. Bracket Notation

1. Example:

```
const jonas2 = {
  firstName: "Jonas",
  lastName: "Schmedtmann",
  age: 2037 - 1991,
  job: "teacher",
  friends: ["Michael", "Peter", "Steven"],
};
console.log(jonas2);
console.log(jonas2.lastName); // retrieves property value for the hardcoded property name
console.log(jonas2["lastName"]); // accepts any expression (not just string)

const nameKey = "Name";
console.log(jonas2["first" + nameKey]);
console.log(jonas2["last" + nameKey]);
```

    1. If we want to compute the property name, we use the `[]` notation
    2. If we want a cleaner approach with not property name computation, use `.` notation

2. Operator precedense:

    1. `.` - member access (left-to-right)
    2. `[...]` - computed member access (left-to-right)
        1. Property names are computed

Object Methods

1. Objects can hold different types of data

    1. Arrays

    2. Objects

    3. Functions

        1. A value can be a functions

    4. Example:

```
const jonas = {
    firstName: 'Jonas',
    lastName: 'Schmedtmann',
    birthYear: 1991,
    job: 'teacher',
    friends: ['Michael', 'Peter', 'Steven'],
    hasDriversLicense: true,

    calcAge: function (birthYear) { // property takes an expression that returns a value
        return 2037 - birthYear;
    }
}

console.log(jonas.calcAge(1991)); // first `jonas.calcAge` is computed
console.log(jonas['calcAge'](1991)); // first `jonas['calcAge']` is computed
```

        1. Any function attached to an object is called a method

        2. Every method gets access to the properties through a special variable called `this`

```
calcAge: function () {
    console.log(this); // prints the entire object calling the method
    return 2037 - this.birthYear;
}
```

            1. `this` is the object calling the method
                1. `jonas.birthYear` also works but violates the DRY principle

        3. If we want to access `calcAge` multiple times and the computation is repeated

            1. Solution: We can store it in the object and retrieve it later

```
calcAge: function() {
    this.age = 2037 - this.birthYear;
    return this.age;
}

console.log(jonas.calcAge());

console.log(jonas.age);
```

```
console.log(jonas.age);
console.log(jonas.age);

// Challenge
// "Jonas is a 46-year old teacher, and he has a driver's license"
```

2. Arrays are also objects
    1. And hence they have methods (built-in methods)

Coding Exercise 7: CHALLENGE #3

```
const mark = {
  fullName: 'Mark Miller',
  mass: 78,
  height: 1.69
};

const john = {
  fullName: 'John Smith',
  mass: 92,
  height: 1.95
};

const calcBMI = function () {
    if (!this.bmi) {
        this.bmi = this.mass / (this.height * this.height);
    }
    return this.bmi;
}

mark.calcBMI = calcBMI;
john.calcBMI = calcBMI;

const summarizeComparison = function (person1, person2) {
    return `${person1.fullName}'s BMI (${person1.calcBMI()}) is higher than ${person2.fullName}'s (${person2.calcBMI()})!`;
}

console.log((mark.calcBMI() > john.calcBMI()) ? summarizeComparison(mark, john) : summarizeComparison(john, mark));
```

CHALLENGE #3: Video Solution

Iteration: The for Loop

1. Loop: Another control structure
    1. Fundamental aspect
        1. They automate repetitive tasks
2. Example:

```
console.log('Lifting weights repitition 1'); // We need to paste 10 times for 10 reptitions (or we don't know how many times if we change later

// for loop keeps running while condition is true
for (let rep = 1; rep <= 10; rep++) { // `let` because the counter is updated in the loop
    // console.log('Lifting weights repetition 1');
    console.log(`Lifting weights repitition ${rep}`);
}
```

Looping Arrays, Breaking and Continuing

1. Looping through Arrays:

```
const jonasArray = [
    'Jonas',
    'Schmedtmann',
    2037 - 1991,
    'teacher',
    ['Michael', 'Peter', 'Steven']
];

const types = [];

for (let i = 0; i < jonasArray.length; i++) {
    console.log(jonasArray[i], typeof jonasArray[i]);

    // types[i] = typeof jonasArray[i];
    types.push(typeof jonasArray[i]);
}
```

continue and break

1. `continue` - Exist the current iteration of the loop and continue to the next one

```
for (let i = 0; i < jonasArray.length; i++) {
  if (typeof jonasArray[i] != "string") continue;

  console.log(jonasArray[i], typeof jonasArray[i]);
}
```

2. `break` - To break out of the whole loop (all subsequent iterations)

```
console.log("--- BREAK WITH NUMBER ---");
for (let i = 0; i < jonasArray.length; i++) {
  if (typeof jonasArray[i] === "number") break;
```

```
  console.log(jonasArray[i], typeof jonasArray[i]);
}
```

Looping Backwards and Loops in Loops

1. Looping backwards

```
const jonasArray = [
  "Jonas",
  "Schmedtmann",
  "teacher",
  1991,
  ["Michael", "Peter", "Steven"],
  true,
];
```

2. Loop inside another loop

```
for (let i = jonasArray.length - 1; i >= 0; i--) {
  console.log(i, jonasArray[i]);
}

for (let exercise = 1; exercise < 4; exercise++) {
  console.log(`======== Starting exercise ${exercise}`);
  for (let rep = 1; rep < 6; rep++) {
    console.log(`Lifting weights repitition ${rep}`);
  }
}
```

The while Loop

1. Used to only specify condition:

```
let rep = 1;
while (rep <= 10) { // while the condition is true
    console.log(`Lifting weights repitition ${rep}`);
    rep++;
}
```

    1. Condition can be anything (sometimes not related to a counter - We may not know how many times a loop has to run)

        1. Example:

```
let dice = Math.trunc(Math.random() * 6); // **(M)**
console.log(dice) + 1;

while (dice !== 6) {
    console.log(`You rolled a ${dice}`);
    dice = Math.trunc(Math.random() * 6) + 1;
    if (dice === 6) console.log('Loop is about to end...');
}
```

            1. While loop is good if we do not know the number of iterations beforehand

Coding Exercise 8: CHALLENGE #4

CHALLENGE #4: Video Solution

Section 4: How to Navigate This Course

Pathways and Section Roadmaps

1. Course Pathways
    1. Which sections to skip
        1. I want to learn everything: Pathway 1: Everything!
        2. I want to learn JavaScript fast: Pathway 2: Only Important Parts
            1. Udemy for Business
        3. I want to learn by building projects with a UI (User Interface): Pathway 3: Only Projects with UI
        4. I only want to leanr the language itself, not the DOM (UI): Pathway 4: Only Language itself (no UI)
        5. I only want the fundamentals to learn React/Angular/Vue next: Pathway 5: Only interested in Frameworks
        6. I only want to prepare for a job interview: Pathway 6: Job Interview Preparation
        7. I took version 1 of the course and only want the updates: Pathway 7: Only What's New in Version 2
2. Roadmap for each section:
    1. Very important sections
    2. Good to know sections
    3. Optional sections

Course Pathways

1. https://www.udemy.com/course/the-complete-javascript-course/learn/lecture/22648329#content

Section 5: Developer Skills & Editor Setup

Section Intro

1. Topics:
    1. How to learn to code
    2. How to think as a developer
    3. How to debug code
    4. How to setup a professional environment on the computer

Section Roadmap

Setting up Prettier and VS Code

1. VS Code editor
    1. Prettier - opinionated code formatter
        1. It makes assumptions about how good code should look like
        2. It makes sure the code is nicely formatted
        3. Install it
        4. Preferences > Settings
            1. Search: default formatter
                1. esbnp.rettier-vscode
            2. Search: Formatonsave
                1. Check
            3. Example:

```
const x = 23;
```

                1. Changing double-quotes to single-quotes
                    1. Google: prettier
                        1. Docs
                            1. Configuring Prettier
                                1. Options
                                    1. Quotes
                                        1. A config file in the project folder:
                                            1. .prettierrc

```
{
    "singleQuote": true
}
```

        5. Configuring snippets
            1. Code > Preferences > User snippets
                1. New Global Snippets File
                    1. jonas

```
Uncomment:

"prefix": "cl"
...
"body": [
    "console.log();"
    // remove second line. For autocomplete to work
```

                2. Setting Sync - syncing settings between computers
                3. TODO Highlight - Highlights in comments

```
// BUG
// FIXME
```

                    1. preferences > Settings
                        1. Open in a JSON format (top right corner)
                            1. "todohighlight.keywords"

Installing Node.js and Setting Up a Dev Environment

1. Reloading the code automatically in the browser
    1. Liver Server
        1. VS Code extension
        2. Node.js and NPM extension (more professional)
            1. Install Node.js
                1. Open terminal
                    1. ctrl + k
                    2. ctrl + (increase window size)
                    3. `node -v` **(M)**
            2. NPM package
                1. `npm install live-server -g` **(M)**
                2. `live-server` **(M)**
                    1. Open `index.html` file by default

Learning How to Code

1. How to fail at learning how to code
    1. Mistakes:
        1. He **didn't have a clear goal** at the beginning of his journey
            1. FIX:
                1. Set a **specific, measurable, realistic**, and **time-based** goal

1. Full-stack Java, DevOps, and Cloud Architect
    1. Front-End - HTML, CSS, JavaScript (March), ReactJS (April), Redux (April), Bootstrap (May)
    2. Back-End - Java, Spring Boot, Spring Cloud, Hibernate (Done - Practice - May)
    3. Cloud - Azure (Done, Practice - May)
    4. DevOps - Docker (Done), Kubernetes (Done), Terraform (May)
2. For Company

2. Know exactly **why** you are learning to code: Switching careers? Finding a better job?
    1. Finding company in top-tier
    2. Better package
3. **Imagine a big project** you want to be able to build!
    1. Keep this in mind and research the technologies needed to build it.
4. Research technologies you need and then learn them
    1. Plan and learn the technologies one by one in a calm way

2. He started by watching courses and reading tutorials, but he would just **copy the code without caring how it works**. Sometimes he would just copy and paste code!
    1. FIX:
        1. Understand the code that you're studying and typing
            1. Do not move forward if you don't understand
        2. **Always type the code**, don't copy-paste!
            1. Training the brain the remember the syntax later
3. He **didn't reinforce** what he was learning by doing small challenges or taking notes
    1. FIX:
        1. After you learn a new feature or concept, **use it immediately**
        2. Take notes
        3. **Challenge yourself** and practice with small coding exercises and challenges
            1. codewars.com
                1. For coding challenges
        4. Don't be in a hurry to complete the course fast!
4. He **didn't practice coding**, and didn't come up with his own project ideas
    1. FIX:
        1. Practicing on our own is the most important thing to do
        2. **This is NOT optional!** Without **practice outside of courses**, you won't go anywhere!
        3. Come up with your own project ideas or copy popular sites or applications, or just parts of them in the beginning
            1. Otherwise, we will never be able to code on our own
        4. Don't be stuck in "tutorial hell"
            1. That is, not being able to leave tutorials
5. He **quickly became frustrated** when his code was not perfectly clean or efficient
    1. FIX:
        1. **Don't get stuck** trying to write the perfect code!
        2. Just write tons of code, **no matter the quality!**
            1. We are still learning
        3. Clean and efficient code will come with time
        4. You can always refactor code later
            1. To improve it
6. He **lost motivation** because he thought he could never know everything
    1. FIX:
        1. Embrace the fact that **you will never know everything**
            1. It is huge
                1. Don't have too many expectations
                2. Don't compare with top developers
                    1. They became experts after a long journey
        2. Just focus on what you need to achieve your goal
            1. Everything is unnecessary
                1. Example: Node.JS is not required if we know Java tech-stack
7. He was **learning in isolation**
    1. FIX:
        1. Explain new concepts to other people. If you can explain it, you truly understand it!
            1. Forces us to understand it first
            2. It is a review for oneself
        2. Share your goals to make **yourself accountable**
            1. To share with others
        3. Share your learning process with the web dev community (#100DaysOfCode, #CodeNewbie, #webdev, etc.)
            1. We get motivation
            2. We get feedback
8. After finishing a couple of courses, **he thought he now was a web developer** and could start applying to jobs. But he couldn't even build an app on his own!
    1. FIX
        1. The **biggest misconception** that people have!
        2. Courses are an amazing starting point, but are only the **beginning of your journey**!
            1. Job readiness:
                1. Everything is awesome (growing confidence)
                    1. Study courses: understand code, take challenges and notes
                2. Cliff of confusion (falling confidence)
                    1. Coding might be harder than I thought we we don't follow a pre-defined path
                        1. Fix: **Stay motivated! Keep writing lots of code on your own, no matter how bad the code is**
                3. Pit of despair (low confidence)
                    1. **We might realize that a lot of practice is required until we build a real application on our own**
                        1. Fix: **Learn with other people, devs and beginners, and share progress**
                        2. Fix: **Keep challenging yourself, run into lots of problems, and fix them**
                            1. Have a mindset of fixing problems
                                1. It is difficult but it is worth-it
                4. Back to awesome (growing confidence)
                    1. Confidence will grow again
                        1. Round up your skillset with best practices and tools (git, testing, task runners, ...)
                        2. Learn new skills
                5. Job Ready (growing confidence)
                    1. Learning never stops

1. Keep constantly adapting to changing technology
2. Prepare a good portfolio

How to Think Like a Developer: Become a Problem Solver!

1. Solving problems
    1. One of the most important thing
2. How to fail at solving problems (real problem with real solutions)
    1. Example: In an array of GPS coordinates, find the two closest points
        1. It is not straight forward
            1. A lot of steps are involved to solve this
        2. There are more complex problems
    2. Problem solving is not just fixing bugs
    3. Whenever John encounters a problem:
        1. He jumps at the problem **without much thinking**
        2. He implements his solution in an **unstructured way** (without a logical approach)
        3. He **gets stressed out** when things don't work
        4. He is **too proud to research** solutions
        5. FIX:
            1. **Stay calm and slow down**, don't jumpt at a problem without a plan
            2. Take a very **logical and rational approach** (programming is just logic, in the end...)
            3. Use the **4-step framework** to solve any problem

4 Steps to Solve Any Problem

1. Make sure you 100% understand the problem. **Ask the right questions** to get a clear picture of the problem
    1. Example:
        1. Project Manager: "We need a function that reverses whatever we pass into it"
            1. What does "whatever" even mean in this context?
            2. What should be reversed?
                1. Answer: It only makes sense to reverse
                    1. Strings
                    2. Numbers
                    3. Arrays
                2. Answer: Objects don't have a well-defined order, so we cannot reverse them
                3. Answer: We cannot reverse `undefined`, `boolean` etc.
            3. What to do if something else is passed in?
                1. How to handle that?
            4. What should be returned?
                1. Should it always be a string, or should the type be the same as passed in?
                    1. We could return the same type as passed in
            5. Solution oriented question:
                1. How to recognize whether the argument is a number, a string, or an array?
                2. How to reverse a number, a string, and an array?
    2. Step back and look at the high-level picture
    3. What are the right questions?
        1. Comes with time and practice
2. **Divide and conquer**: Break a big problem into smaller sub-problems
    1. Break them into as many small problems as possible
        1. The small problems will be a lot easier to solve
    2. This is applicable to understanding the problem
        1. If we break down the question into parts and try to understand each part, it would make it clear
        2. Example:
            1. Sub-problems:
                1. Check if argument is a number, a string, or an array
                    1. We can solve this problem in isolation and move on
                2. Implement reversing a number
                3. Implement reversing a string
                4. Implement reversing an array
                5. Return reversed value
            2. The sub-problems look like a task list that we need to implement
    3. The strategy works for bigger and more complex problems
    4. **This is an essential step for any problem solving**
3. Don't be afraid to do as much **research** as you have to (if we don't know how to solve the sub-problems)
    1. We must try to solve the sub-problems on our own with our coding abilities
    2. If we hit a wall and cannot move on, don't waste time and search Google, Stackoverflow, and mdn web docs
    3. Questions to google:
        1. How to check if a value is a number in JavaScript?
        2. How to check if a value is a string in JavaScript?
        3. How to check if a value is an array in JavaScript?
        4. How to reverse a number in JavaScript?
        5. How to reverse a string in JavaScript?
        6. How to reverse an array in JavaScript?
4. For big problems, **write pseudo-code** before writing the actual code

    1. An informal description of the actual code we are going to write

        1. It is code for humans to understand and not for computers

    2. Example:

```
function reverse(value)
    if value type is !string && !number && !array
        return value

    if value type == string
        reverse string
    if value type == number
```

```
        reverse number
    if value type == array
        reverse array

    return reversed value
```

1. There are no rules for writing pseudocode (it is for ourselves or our team)
2. We can implement the pseudo-code in JavaScript

General Tips

1. Develop a genuine curiosity and passion on how things actually work
    1. Not just programming

Using Google, StackOverflow and MDN

1. Solving a problem:
    1. "Given an array of temperatures of one day, calculate the temperature amplitude. Keep in mind that sometimes there might be a sensor error."
        1. `const temperatures = [3, -2, -6, -1, 'error', 9, 13, 17, 15, 14, 9, 5];`
        2. Steps in action:
            1. Understanding the problem
                1. What is temperature amplitude?
                    1. Answer: Difference between highest and lowest temp
                2. How to compute the max and min temperatures?
                3. What does a sensor errr look like? What to do when it occurs?
                    1. Answer: Ignore the error
            2. Breaking up into sub-problems:
                1. How to ignore errors?
                2. Find max value in temp array
                3. Find minimum value in temp array
                4. Subtract min from max (amplitude)
                5. Return the value
            3. Writing a function - it is a good idea to write a function for a particular task

```
const calcTempAmplitude = function (temps) {
    let max = temps[0];
    let min = temps[0];
    for (let i = 0; i < temps.length; i++) {
        const currentTemp = temps[i];

        if (typeof currentTemp !== 'number') {
            continue;
        }

        if (currentTemp > max) {
            max = currentTemp;
        }
        if (currentTemp < min) {
            min = currentTemp;
        }
    }
    console.log(max, min);
    return max - min;
};

// calcTempAmplitude([3, 7, 4, 1, 8]);
const amplitude = calcTempAmplitude(temperatures);
console.log(amplitude);
```

            4. Research how to find min/max value from an array
                1. Stackoverflow
                    1. We may not understand most of the solutions in the beginning
                    2. Use the solution that we understand
        3. New requirement: Two arrays
            1. Problem 2: Function should now receive 2 arrays of temperatures
            2. Steps:
                1. Understanding the problem:
                    1. Should we implement functionality twice?
                        1. Answer: Just merge two arrays
                2. Breaking up into sub-problems
                    1. How to merge 2 arrays?
                        1. Rest of the logic is the same
                3. Research:
                    1. Search: Javascript merge two arrays
                        1. Stackoverflow:
                            1. `array1.concat(array2)`
                            2. `array1.push(<multiple-elements>)`
                                1. See mdn documentation (complete but unofficial documentation)

4. Code:

```
const calcTempAmplitudeNew = function (t1, t2) {
    const temps = t1.concat(t2);
    console.log(temps);
    // ...
}

const amplitudeNew = calcTempAMplitudeNew([3, 5, 1], [9, 0, 5]);
console.log(amplitudeNew);
```

Debugging (Fixing Errors)

What is a Software Bug?

1. Software bug: Defect or problem in a computer program. Basically, any **unexpected or unintended behavior** of a computer program is a software bug.
2. Bugs are **completey normal** in software development!
    1. Humans make mistakes
    2. Many bugs are quite small and simple to discover
    3. Previous example: "We need a function that reverses whatever we pass into it"
        1. If the reversed array looks scrambled but not reversed
            1. There is a bug in the reverse function
                1. Solution: Debugging - **Process of finding, fixing, and preventing bugs**

The Debugging Process

1. Identify: Become aware that there is a bug (we need to be aware of context in which the bug happened)
    1. During development
    2. Testing software
    3. User reports during production
        1. Worst bugs
            1. Affect real users
    4. Context: Browsers, users, etc.
        1. Certain bugs can happen in certain browsers or only for certain users
2. Find: Isolating where exactly the bug is happening in code
    1. Using Developer console (simple code)
        1. Small bugs
    2. Using Debugger (complex code)
        1. Important for developer
3. Fix: Correct the bug
    1. Replace wrong solution with new correct solution that works
4. Prevent: Preventing it from happening again
    1. Searching for the same bug in similar code
    2. Writing tests using testing software
        1. Bonus section

Debugging with the Console and Breakpoints

1. Simple bugs:
    1. Conversion to kelvin

        1. Add 273 to temperature in celcius

```
const measureKelvin = function () {
  const measurement = {
    type: 'temp',
    unit: 'celsius',

    // C) FIX
    value: Number(prompt('Degrees celsius:')),
  };

  // console.log(measurement); // Input is a string
  // B) FIND
  console.table(measurement); // **(M)**

  console.log(measurement.value); // looks okay
  // console.warn(measurement.value);
  // console.error(measurement.value);
  const kelvin = measurement.value + 273;
  return kelvin;
};

// A) IDENTIFY
console.log(measureKelvin());
```

Using a Debugger in Google Chrome

1. Developer Console
    1. Sources
        1. script.js
            1. Set breakpoint
                1. Click on the left-bar
                    1. Reload the page - execution will stop at the breakpoint
                        1. Shows values of all variables
                    2. Click Resume (top right corner)
                    3. Step in, step over, ...

Coding Challenge #1

1. Problem:

```
Given an array of forecasted maximum temperatures, the thermometer displays a string with these temperatures.

Example: [17, 21, 23] will print "... 17C in 1 days ... 21 C in 2 days ... 23 C in 3 days ..."

Construct a function 'printForecast' which takes in an array 'arr' and logs a string like the above to the console.

Use the problem-solving framework: Understand the problem and break it up into sub-problems!

TEST DATA 1: [17, 21, 23]
TEST DATA 2: [12, 5, -5, 0, 4]
```

The Rise of AI Tools (ChatGPT, Copilot, Cursor AI, etc.)

The Workflo of Using AI for Coding

1. Examples:
    1. ChatGPT
    2. GitHub Copilot
    3. Cursor
2. They are powered by an LLM (Large Language Model) (GPT, Claude)
    1. They generate code based on the prompts the user gives the model
3. Different tools give a different UX
4. We can ask the tools to write code snippets, functions, and even generate entire files
5. Copilot and Cursor have a very powerful autocomplete feature that is aware of the code we are writing, the comments, clipboard, and the entire code-base
    1. We can also chat with our code-base
        1. Ask questions like:
            1. How certain parts of the app work?
            2. ...
    2. They work well sometimes and not so well some other times
6. We need to follow a workflow when we are using an AI tool
    1. Step 1: Make sure you 100% understand the problem. Ask questions to get a clear picture (How to think like a developer) (Don't use an AI tool)
        1. Apply divide and conquer to divide the problem into sub-problems
    2. Step 2: Choose AI (tool) and give it a very specific prompt and enough context (language, style, etc.)
        1. Telling it exactly what we want to get done
            1. Requires feeding the AI the exact context of the problem
                1. What's the topic?
                2. What's the exact goal to be achieved?
                3. What language it has to write the solution in?
                4. What coding style do we want?
                5. ...
                    1. AI generates a solution to the problem and output the code
    3. Step 3: Review and test the output solution. Make sure you introduce no bugs into your app
        1. AI code can contain a lot of bugs which can be hidden
        2. AI can suggest a solution that is more complicated or convoluted than necessary
    4. Step 4: Use the AI to correct or improve the solution
        1. Generate a new specific prompt telling AI what's wrong and want we need fixed.
        2. Repeat the steps 2-4 until the solution is good enough
        3. If we are not able to reach a good solution, we need to stop it
    5. Step 5: Integrate the solution into your app
        1. If we are satisfied with the solution
7. The tools are meant for developers who already know what they are doing

Guidelines for Safe use of AI

1. Before you use AI:
    1. You need to know **how to code on your own**. Fundamental skills are 100% essential!
        1. Don't rely on AI
    2. You need to be able to solve problems on your own
    3. You need to have very **critical thinking** (AI code contains a suprising amount of **bugs or bad code**)
        1. The bugs may be hard to find
        2. The code may not be good enough
    4. You need to have **curiosity and joy** while coding
        1. If you lack the passion for writing code, it is not the right career
2. Don't use AI without knowing what you're doing. Otherwise, AI will turn you into a terrible programmer (or not a programmer at all)
    1. You might become a copy-paster
        1. Who cannot generate a good app
        2. Who cannot fix bugs
        3. Who cannot maintain code
        4. ...
3. Use AI as an assistant, not a replacement!
    1. Save time on repititive and boring tasks (like boilerplace code - things that don't require much thinking). It's also great for **learning!**
        1. Ask programming questions
        2. Ask for code solutions and analyze the solutions step by step
        3. We can learn new tools and techniques

Incorporate AI Code

1. When you could have **written the code yourself**
    1. Think about the steps and JS features we might need to solve the problem
        1. Better to imagine the code in the mind
        2. AI generates code faster than you typing the code by hand
2. When you **truly understand** the generated code
3. When you have ensured the code is **100% correct**
    1. Ensures errors made by AI don't make it into the codebase
4. When you're not using the code for **mission-critical parts** of your apps

1. AI can be used to be faster at non-crucial tasks even if we could have done it ourselves (we must be able to fix code if it breaks)

Will AI Take Your Job?

1. Will AI take my job?
    1. No
        1. Why?
            1. **There is a lot of hype around AI right now!** In the past, tools like Dreamweaver, Wordpress, or Wix were supposed to replace web developers...
                1. AI only helps developers
            2. **Things will change**: a portion of apps will be written by AI. You might write less code in the future, but that's not a bad thing!
                1. AI is good for mundane tasks
            3. **You will still be needed!** You're the one who maintains control and implements the app architecture. **You** are the one who asks AI to write the code in the first place, reviews it, corrects it, and integrates it into the app
                1. Developer will be in control of implementing and adapting the application's architecture, and writing the core business logic
                2. Developer is the one who asks AI to generate the code, test it, and integrate it into the app (in control)
                3. Developers also decide how much of AI code to incorporate in projects
            4. Software developers do a **lot more than just writing code**: maintain the bigger picture of huge projects / think about software / implement complex design principles / are creative / they collaborate with other developers and clients
                1. They are many things that AI is not capable of solving
                    1. Humans are more creative than AI
            5. AI-generated code is **still buggy**, and AI is **not very good at debugging** (non-trivial problems that span multiple functions or files)
                1. Solutions might make the code worse each time we ask it to improve the solution
            6. The world **needs more software** than can be built by humans right now!
                1. AI can improve bad or non-existing software
                2. If AI makes us 10 times faster in the future, we might be ble to write 10 times more software
            7. **AI will help us, not replace use!**
        2. There may be even more jobs in the future

Solve CHALLENGE #2 With AI

1. Question:
    1. Let's say you're building a time tracking application for freelancers. At some point in building this app, you need a function that receives daily work hours for a certain week, and returns:
        1. Total hours worked
        2. Average daily hours
        3. The day with the most hours worked
        4. Number of days worked
        5. Whether the week was full-time (worked 35 hours or more)
    2. TEST DATA: [7.5, 8, 6.5, 0, 8.5, 4, 0]
2. AI Assistant tool: ChatGPT

```
I'm building a time-tracking application for freelancers. Write a function called analzeWorkWeek that receives an array of daily work hours for a cert
1. Total hours worked
2. Average daily hours
3. The day with the most hours worked
4. Number of days worked
5. Whether the week was full-time (worked 35 hours or more)

function analyzeWorkWeek(dailyHours) {
    // Ensure input is valid
    if (!Array.isArray(dailyHours) || dailyHours.length !== 7) {
        throw new Error("Input must be an array of 7 numbers representing daily work hours.");
    }

    // Calculate total hours worked
    const totalHours = dailyHours.reduce((sum, hours) => sum + hours, 0);

    // Calculate average daily hours
    const averageHours = +(totalHours / dailyHours.length).toFixed(1);

    // Find the day with the most hours worked
    const maxHours = Math.max(...dailyHours);
    const maxHoursDay = dailyHours.indexOf(maxHours);

    // Count the number of days worked
    const daysWorked = dailyHours.filter(hours => hours > 0).length;

    // Determine if the week was full-time
    const isFullTime = totalHours >= 35;

    // Return the result as an object
    return {
        totalHours,
        averageHours,
        maxHoursDay,
        daysWorked,
        isFullTime
    };
}

// Example usage:
const weekHours = [8, 7.5, 6, 8.5, 0, 4, 5];
console.log(analyzeWorkWeek(weekHours));

1. We need to test the code properly before incorporating it into the application
```

2. Iteration 2: Change maxHoursDay to an actual work day like Monday.

```
function analyzeWorkWeek(dailyHours) {
    // Ensure input is valid
    if (!Array.isArray(dailyHours) || dailyHours.length !== 7) {
        throw new Error("Input must be an array of 7 numbers representing daily work hours.");
    }
```

```
    // Days of the week
    const daysOfWeek = ["Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday", "Sunday"];

    // Calculate total hours worked
    const totalHours = dailyHours.reduce((sum, hours) => sum + hours, 0);

    // Calculate average daily hours
    const averageHours = +(totalHours / dailyHours.length).toFixed(1);

    // Find the day with the most hours worked
    const maxHours = Math.max(...dailyHours);
    const maxHoursDay = daysOfWeek[dailyHours.indexOf(maxHours)];

    // Count the number of days worked
    const daysWorked = dailyHours.filter(hours => hours > 0).length;

    // Determine if the week was full-time
    const isFullTime = totalHours >= 35;

    // Return the result as an object
    return {
        totalHours,
        averageHours,
        maxHoursDay,
        daysWorked,
        isFullTime
    };
}

// Example usage:
const weekHours = [8, 7.5, 6, 8.5, 0, 4, 5];
console.log(analyzeWorkWeek(weekHours));
```

3. Iteration 3: The function should require the input array to have exactly 7 days. Update the function accordingly.
    1. Already taken care of.
4. To learn more:
    1. What does the `filter` method do? Why is it useful? Give me a few examples.
    2. What is a callback? Explain the filter method in simple terms to a JavaScript beginner.
5. Iteration 4: Add JavaScript docs to the function.

```
/**
 * Analyzes a week of work hours and provides insights.
 *
 * @param {number[]} dailyHours - An array of 7 numbers representing daily work hours (Monday to Sunday).
 * @returns {Object} - An object containing the analysis of the work week:
 *   - {number} totalHours: Total hours worked in the week.
 *   - {number} averageHours: Average daily hours worked (rounded to one decimal place).
 *   - {string} maxHoursDay: The day of the week with the most hours worked.
 *   - {number} daysWorked: The number of days with more than 0 hours worked.
 *   - {boolean} isFullTime: Whether the total hours worked is 35 or more.
 *
 * @throws {Error} - Throws an error if the input is not a valid array of 7 numbers.
 */
function analyzeWorkWeek(dailyHours) {
    // Ensure input is valid
    if (!Array.isArray(dailyHours) || dailyHours.length !== 7) {
        throw new Error("Input must be an array of 7 numbers representing daily work hours.");
    }

    // Days of the week
    const daysOfWeek = ["Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday", "Sunday"];

    // Calculate total hours worked
    const totalHours = dailyHours.reduce((sum, hours) => sum + hours, 0);

    // Calculate average daily hours
    const averageHours = +(totalHours / dailyHours.length).toFixed(1);

    // Find the day with the most hours worked
    const maxHours = Math.max(...dailyHours);
    const maxHoursDay = daysOfWeek[dailyHours.indexOf(maxHours)];

    // Count the number of days worked
    const daysWorked = dailyHours.filter(hours => hours > 0).length;

    // Determine if the week was full-time
    const isFullTime = totalHours >= 35;

    // Return the result as an object
    return {
        totalHours,
        averageHours,
        maxHoursDay,
        daysWorked,
        isFullTime
    };
}

// Example usage:
const weekHours = [8, 7.5, 6, 8.5, 0, 4, 5];
console.log(analyzeWorkWeek(weekHours));
```

6. Iteration 5: Write unit tests for the function covering all the possible edge cases, and verify if all the test cases pass.

```
/**
 * Analyzes a week of work hours and provides insights.
 *
 * @param {number[]} dailyHours - An array of 7 numbers representing daily work hours (Monday to Sunday).
 * @returns {Object} - An object containing the analysis of the work week:
 *   - {number} totalHours: Total hours worked in the week.
 *   - {number} averageHours: Average daily hours worked (rounded to one decimal place).
 *   - {string} maxHoursDay: The day of the week with the most hours worked.
 *   - {number} daysWorked: The number of days with more than 0 hours worked.
```

```
 *   - {boolean} isFullTime: Whether the total hours worked is 35 or more.
 *
 * @throws {Error} - Throws an error if the input is not a valid array of 7 numbers.
 */
function analyzeWorkWeek(dailyHours) {
    // Ensure input is valid
    if (!Array.isArray(dailyHours) || dailyHours.length !== 7) {
        throw new Error("Input must be an array of 7 numbers representing daily work hours.");
    }

    // Days of the week
    const daysOfWeek = ["Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday", "Sunday"];

    // Calculate total hours worked
    const totalHours = dailyHours.reduce((sum, hours) => sum + hours, 0);

    // Calculate average daily hours
    const averageHours = +(totalHours / dailyHours.length).toFixed(1);

    // Find the day with the most hours worked
    const maxHours = Math.max(...dailyHours);
    const maxHoursDay = daysOfWeek[dailyHours.indexOf(maxHours)];

    // Count the number of days worked
    const daysWorked = dailyHours.filter(hours => hours > 0).length;

    // Determine if the week was full-time
    const isFullTime = totalHours >= 35;

    // Return the result as an object
    return {
        totalHours,
        averageHours,
        maxHoursDay,
        daysWorked,
        isFullTime
    };
}

// Unit Tests
function runTests() {
    const testCases = [
        { input: [8, 8, 8, 8, 8, 0, 0], expected: { totalHours: 40, averageHours: 5.7, maxHoursDay: "Monday", daysWorked: 5, isFullTime: true } },
        { input: [4, 5, 6, 7, 8, 0, 0], expected: { totalHours: 30, averageHours: 4.3, maxHoursDay: "Friday", daysWorked: 5, isFullTime: false } },
        { input: [0, 0, 0, 0, 0, 0, 0], expected: { totalHours: 0, averageHours: 0.0, maxHoursDay: "Monday", daysWorked: 0, isFullTime: false } },
        { input: [10, 10, 10, 10, 10, 10, 10], expected: { totalHours: 70, averageHours: 10.0, maxHoursDay: "Monday", daysWorked: 7, isFullTime: true
        { input: [0, 0, 0, 0, 0, 8, 9], expected: { totalHours: 17, averageHours: 2.4, maxHoursDay: "Sunday", daysWorked: 2, isFullTime: false } },
    ];

    testCases.forEach(({ input, expected }, index) => {
        const result = analyzeWorkWeek(input);
        const passed = JSON.stringify(result) === JSON.stringify(expected);
        console.log(`Test Case ${index + 1}: ${passed ? "PASSED" : "FAILED"}`);
        if (!passed) {
            console.log("  Input:", input);
            console.log("  Expected:", expected);
            console.log("  Received:", result);
        }
    });
}

runTests();
```

Section 6: [OPTIONAL] HTML & CSS Crash Course

Section Info

Basic HTML Structure and Elements

Attributes, Classes and IDs

Basic Styling with CSS

Introduction to the CSS Box Model

Section 7: JavaScript in the Browser: DOM and Events Fundamentals

Section Intro

1. How JavaScript interacts with web-pages

Section Roadmap

1. JavaScript in the Browser: DOM and Events Fundamentals

PROJECT #1: Guess My Number!

1. Retro design
    1. Guess a secret number
    2. Score decreases by 1 if we fail
    3. Highscore
    4. Again - resets (except for high-score)
2. 05-Guess...-Number folder

1. We can select elements based on class

```
console.log(document.querySelector('.message').textContent);
```

What's the DOM and DOM Manipulation

1. DOM Minipulation: JS interacting with web-page

What is the DOM?

1. DOM: Document Object Model
    1. A structured representation of HTML documents. Allows JS to access HTML elements and styles to manipulate them.
        1. Change text
        2. Change HTML attributes
        3. CSS styles
    2. It is a connection between HTML elemements and JS code
    3. DOM is automatically instantiated by the browser as soon as HTML page loads
        1. It is stored as a tree structure

            1. Each HTML element in the tree is an object

            2. Example:

```
Document
    Element (html)
        Element (head)
        Element (body)
```

            1. `Document`: root object
                1. A special object
                2. The object serves as an entry point into the DOM
                    1. `document.querySelector()`
                        1. `querySelector` method is available on the `document` object
                3. The first child elmenent of `document` is `html` element
                    1. Root of all web-pages
                    2. `html` element has two children
                        1. `head`
                        2. `body`
                    3. `head` and `body` are siblings
                4. A DOM is a complete representation of an HTML document

DOM !== JavaScript

1. DOM Methods and Properties for DOM manipulation
    1. Not part of JS
        1. Example: `document.querySelector()`
        2. It is not part of ECMA script specification
    2. They are part of WEB APIs
        1. Libraries that browsers implement (written in JS)
        2. They are accessible in JS code
            1. No need to import
        3. API - Application Programming Interface
            1. There is an official DOM spec that browsers implement
        4. Other WEB APIs
            1. Timers
            2. Fetch

Selecting and Manipulating Elements

1. Setting content:

```
document.querySelector('.message').textContent = '🎉 Correct Number!'; // Windows + .
```

    1. We did DOM manipulation

2. Get secret number:

```
document.querySelector('.number').textContent = 13;
document.querySelector('.score').textContent = 20;

document.querySelector('.guess').value = 23;
console.log(document.querySelector('.guess').value);
```

Handling Click Events

1. Code reacting to something happening to DOM

    1. We need to use event-listener
        1. Event: Something happening to a page
            1. Mouse click
            2. Mouse moving
            3. Key-press
            4. ...
        2. EventListener: Waits for a certain event to happen and then reacts to it

2. Code:

```
document.querySelector('.check').addEventListener('click', function () {
  // console.log(document.querySelector('.guess').value);
```

```
  const guess = Number(querySelector('.guess').value);

  if (!guess) {
    // 0 - falsy value
    document.querySelector('.message').textContent = '⛔ No Number!';
  }
}); // **(M)**
// JS engine calls the function as soon as the specified event happens
```

Implementing the Game Logic

```
const secretNumber = Math.trunc(Math.random() * 20) + 1;
document.querySelector('.number').textContent = secretNumber;
let score = 20; // Not relying on DOM to hold the data. It is part of application state

document.querySelector('.check').addEventListener('click', function () {
  // console.log(document.querySelector('.guess').value);
  const guess = Number(querySelector('.guess').value);

  if (!guess) {
    // 0 - falsy value
    document.querySelector('.message').textContent = '⛔ No Number!';
  } else if (guess === secretNumber) {
    document.querySelector('.message').textContent = '🎉 Correct Number!';
  } else if (guess > secretNumber) {
    if (score > 1) {
      document.querySelector('.message').textContent = '📈 Too high!';
      score--;
      document.querySelector('.score').textContent = score;
    } else {
      document.querySelector('.message').textContent = '💥 You lost the game!';
      document.querySelector('.score').textContent = 0;
    }
  } else {
    if (score > 1) {
      document.querySelector('.message').textContent = '📉 Too low!';
      score--;
      document.querySelector('.score').textContent = score;
    } else {
      document.querySelector('.message').textContent = '💥 You lost the game!';
      document.querySelector('.score').textContent = 0;
    }
  }
}); // **(M)**
// JS engine calls the function as soon as the specified event happens
```

Manipulating CSS Styles

```
document.querySelector('body').style.backgroundColor = '#60b347'; // Applied as inline style

document.querySelector('.number').style.width = '30rem'; // Always a string & must have a unit, applied as inline style
// When guess is too high
```

Coding Challenge #1

1. Steps:
    1. Select the element with the 'again' class and attach a click event handler
    2. In the handler function, restore initial values of the score and number variables
    3. Restore the initial conditions of the message, number, score and guess input field
    4. Also restore the original background color (#222) and number width (15rem)

Implementing Highscores

```
let highScore = 0;

if (score > highScore) {
  highScore = score;
  document.querySelector('highscore').textContent = highScore;
}
```

Refactoring Our Code: The DRY Principle

1. Why DRY?
    1. Duplication code?
        1. If we have to change some functionality, we need to change in multiple places
            1. It is not manageable with big code-bases
        2. We can start out with duplicate code
        3. Then we refactor - restructuring without changing how it works
            1. Steps:
                1. **Identify duplicate or almost duplicate code**
                2. **Consolidate the duplicate code**
2. Code:

```
let secretNumber = Math.trunc(Math.random() * 20) + 1;
let score = 20; // Not relying on DOM to hold the data. It is part of application state
let highScore = 0;

const displayMessage = function (message) {
  document.querySelector('.message').textContent = message;
};

document.querySelector('.check').addEventListener('click', function () {
  // console.log(document.querySelector('.guess').value);
```

```
  const guess = Number(document.querySelector('.guess').value);

  // When there is no input
  if (!guess) {
    // 0 - falsy value
    // document.querySelector('.message').textContent = '⛔ No Number!';
    displayMessage('⛔ No Number!');

    // When player wins
  } else if (guess === secretNumber) {
    // document.querySelector('.message').textContent = '🎉 Correct Number!';
    displayMessage('🎉 Correct Number!');

    document.querySelector('body').style.backgroundColor = '#60b347'; // Applied as inline style

    document.querySelector('.number').style.width = '30rem'; // Always a string & must have a unit, applied as inline style

    document.querySelector('.number').textContent = secretNumber;

    if (score > highScore) {
      highScore = score;
      document.querySelector('.highscore').textContent = highScore;
    }

    // When guess is not equal
  } else if (guess !== secretNumber) {
    if (score > 1) {
      // document.querySelector('.message').textContent = guess > secretNumber ? '📈 Too high!' : '📉 Too low!';
      displayMessage(guess > secretNumber ? '📈 Too high!' : '📉 Too low!');
      score--;
      document.querySelector('.score').textContent = score;
    } else {
      // document.querySelector('.message').textContent = '💥 You lost the game!';
      displayMessage('💥 You lost the game!');
      document.querySelector('.score').textContent = 0;
    }
  }
}); // **(M)**
// JS engine calls the function as soon as the specified event happens

document.querySelector('.again').addEventListener('click', function () {
  // Anonymous handler function
  secretNumber = Math.trunc(Math.random() * 20) + 1;
  score = 20;

  document.querySelector('.guess').value = '';
  // document.querySelector('.message').textContent = 'Start guessing...';
  displayMessage('Start guessing...');
  document.querySelector('.number').textContent = '?';
  document.querySelector('.score').textContent = 20;
  document.querySelector('body').style.backgroundColor = '#222';
  document.querySelector('.number').style.width = '15rem';
});
```

PROJECT #2: Modal Window

1. First UI component:
    1. Click on any button. Modal window opens
    2. Click on x, outside the window, esc key
2. Solution: Done by manipulating classes
3. 06-Modal
    1. Select all the elements we need at the top
    2. Store the elements into variables to use them over and over again

Working With Classes

1. Reacting to a click on each button
2. We can add and remove classes
    1. Classes can be used to aggregate properties
        1. We can activate and de-activate multiple styles at once
3. Code:

```
const modal = document.querySelector('.modal');
const overlay = document.querySelector('.overlay');
const btnCloseModal = document.querySelector('.close-modal');
// const btnsOpenModal = document.querySelector('.show-modal'); // It only returns the first element
// console.log(btnsOpenModal);

const btnsShowModal = document.querySelectorAll('.show-modal');
console.log(btnsShowModal);

const openModal = function () {
  console.log('Button clicked');
  // modal.classList.remove('hidden', '<class>');
  modal.classList.remove('hidden'); // **(M)** just the name of the class
  overlay.classList.remove('hidden'); // .hidden can have many properties and everything will be taken care of at once
};

const closeModal = function () {
  modal.classList.add('hidden');
  overlay.classList.add('hidden');
};

for (let i = 0; i < btnsShowModal.length; i++) {
  // console.log(btnsShowModal[i].textContent);
  btnsShowModal[i].addEventListener('click', openModal);
```

```
}

btnCloseModal.addEventListener('click', closeModal);
overlay.addEventListener('click', closeModal);
```

Handling an "Esc" Keypress Event

1. Hitting ESC key
    1. Responding to keyboard events
        1. Keyboard events are **global events** (they do not happen on a specific element) - We listen on the whole document

```
document.addEventListener('keydown', function (event) {
    console.log(event);
}); // No matter where the event happens on the page, they will trigger this event
```

            1. There are 3 types of keyboard events:
                1. `keydown` **(M)** - fired when we press down our finger on a key
                2. `keypress` **(M)** - fired continously as we keep our finger on a key
                3. `keyup` **(M)** - fired when we lift our finger off the key
            2. The key information is stored in the event (object) as soon as the event occurs
                1. event object - JS generates it
                    1. It contains all the info about the object and we can access the object in the event-handler function

2. Code:

```
document.addEventListener('keydown', function (event) {
  // console.log(event.key);

  if (event.key === 'Escape' && !modal.classList.contains('hidden')) {
    closeModal();
  }
});
```

PROJECT #3: Pig Game

1. Live game: https://pig-game-v2.netlify.app
    1. Roll and current score gets accumulated
    2. When we click on hold, current score gets added to the total score
    3. When we roll a 1, the current score becomes 0 and the next player gets a chance to roll
    4. Click New Game to reset
    5. Flow chart: A representation of everything that can happen in the application
        1. Flow 1:
            1. User rolls dice
            2. Generate random dice roll
            3. Display dice roll
            4. Is it a 1?
                1. No:
                    1. Add dice roll to current score
                    2. Display new score
                2. Yes:
                    1. Switch player
        2. Flow 2:
            1. User hods score
            2. Add current score to total score
            3. Score >= 100?
                1. No:
                    1. Switch player
                2. Yes:
                    1. Current player wins!
        3. Flow 3:
            1. User resets game
            2. Set all scores to 0
            3. Set player 1 as starting player

Rolling the Dice

Switching the Active Player

Holding Current Score

Resetting the Game

Section 8: How JavaScript Works Behind the Scenes

Section Intro

1. How everyting we learnt works under the hood
    1. Helps us write better code
    2. Helps us understand other developers' code
    3. Makes a better, more confident, and more successful developer
    4. Sets us apart from the competition
        1. Many junior developers do not know much about this stuff

Section Roadmap

A High-Level Overview of JavaScript

1. Covers most of the things we need to learn to eventually master JS
2. Definition 1:
    1. **JavaScript is a high-level, object-oriented, multi-paradigm programming language** (tip of the iceberg)
3. Definition 2:
    1. **JavaScript is a high-level, prototype-based object-oriented, multi-paradigm, interpreted or just-in-time compiled, dynamic, single-threaded, garbage-collected programming language with first-class functions and a non-blocking event loop concurrency model.** - unreadable but true.
        1. Important features
            1. High-level
            2. Prototype-based object-oriented
            3. Multi-paradigm
            4. Interpreted or just-in-time compiled
            5. Dynamic
            6. Single-threaded
            7. Garbage-collected
            8. First-class functions
            9. Non-blocking
            10. Event loop concurrency model

Deconstructing The Monster Definition

1. High-Level
    1. Any computer program needs resources:
        1. Memory
        2. CPU
        3. ...
    2. There are low-level languages
        1. C - We need to manage the resources
            1. We need to explicitly ask the computer for memory to define a new variable
        2. JS, Python - Developer does NOT have to worry, everything happens automatically
            1. Abstractions take away the resource management work away from us
            2. Advantages:
                1. Easier to learn
                2. Easier to use
            3. Disadvantages:
                1. Programs will never be as optimized or fast as C programs
2. Garbage-Collected
    1. It takes memory-management away from us
    2. It is an algorithm that removes old and un-used objects on computer memory
        1. To avoid clogging it up with unnecessary stuff
        2. It cleans our memory from time-to-time (we don't have to do it manually)
3. Interpreted or Just-in-Time Compiled
    1. Computer processor only understands 0s and 1s
        1. Every program must be in machine code (0s and 1s)
            1. It is not practical to write
                1. Solution: We write human readable JS code which is eventually translated to machne code
                    1. Two ways:
                        1. Compiling
                        2. Interpreting
                            1. This happens inside the JS engine
4. Multi-Paradigm
    1. Paradigm: An approach and midset of structuring code, which will direct your coding style and technique (depends on what paradigm the project is using)
        1. Popular paradigms:
            1. Procedural programming
                1. Organizing code in a linear way with some functions in between
            2. Object-oriented programming (OOP)
            3. Functional programming (FP)
        2. Other paradigms:
            1. Imperative
            2. Declarative
    2. Programming languages are usually just procedural, OOP, or FP.
        1. JS does all of them
            1. Very flexible and verstatile
                1. We can use whatever paradigm we want
5. Prototype-Based Object-Oriented
    1. Almost everything in JS is an object except the primitive types (numbers, strings, ...)
        1. Arrays
        2. ...
    2. Why does any array support the `push` method?
        1. **Due to prototypal inheritance**
            1. Each array is created from an array blueprint (a template) - called prototype
                1. Prototype - contains all array methods
                    1. `Array.prototype.push`
                    2. `Array.prototype.indexOf`
                2. **Prototype contains all the array methods**
                    1. Arrays in the code inherit the methods from the blueprint (to enable their usage)

6. First-Class Functions
    1. In a language with **first-class functions**, functions are simply **treated as variables**. We can pass them into other functions, and return them from functions
        1. Very powerful
            1. Allows us to use powerful techniques
            2. Allows us to do functional programming
    2. Example:

```
const closeModel = () => {
    modal.classList.add("hidden");
    overlay.classList.add("hidden");
};

overlay.addEventListener("click", closeModal); // passing a function into another function as an argument: first-class functions!
```

7. Dynamic
    1. Dynamically-typed language:
        1. No data types are assigned to variables
            1. Types become known at runtime
    2. Data type of variable is automatically changed when we re-assign

```
let x = 23;
let y = 19;
x = "Jones";
```

        1. They become known only when the JS executes our code
    3. In other programming languages, we need to manually assign types: (C, Java, Ruby)
        1. Strongly-typed languages prevent certain bugs
            1. Solution: TypeScript
8. Single-Threaded
    1. Concurrency Model:
        1. How JS engine handles multiple tasks happening at the same time.
            1. Why do we need that?
                1. JS runs in one **single thread**
                    1. So it can only do one thing at a time
                        1. How do we handle multiple things happening at the same time?
        2. Thread:
            1. It is a set of instructions executed in the computer's CPU
            2. It is where our code is executed in the machine's processor
        3. What about a long-running task? (fetching data from a remote server say)
            1. It would block the single thread.
                1. **We want a non-blocking behavior!**
                    1. How do we achieve that?
                        1. Solution: Event-loop
9. Non-Blocking Event Loop
    1. By using an **event loop**:
        1. Takes long running tasks, executes them in the "background", and puts them back in the main thread once they are finished

The JavaScript Engine and Runtime

1. What is a JavaScript Engine?
    1. A computer program that executes JavaScript code
        1. There are many steps involved in doing that
    2. Every browser has its JS engine
        1. V8 Engine - powers google chrome & node.js (for server-side applications)
    3. Components:
        1. Callstack
            1. Where our code is executed
                1. Uses **execution contexts**
        2. Heap
            1. Unstructured memory pool which stores all the objects that our application needs
    4. How is code executed?
        1. Computer Science Sidenote: Compilation vs Interpretation
            1. Every single computer program needs to be converted into machine code
                1. Using:
                    1. Compilation
                        1. Entire source-code is converted into machine code at once, and written to a binary file that can be executed by a computer
                        2. Flow:

```
Source code
    |
    Step 1
    Compilation
    |
    v
Portable file:
machine code
    |
    Step 2
    Execution
(can happen way after compilation)
    |
    v
Program running
```

2. Interpretation
    1. Interpreter runs through the source code and executes it line by line

```
Source code
    |
Execution line by line
(Code still needs to be
converted to machine code)
    |
    v
Program running
```

        1. Disadvantages: Interpreted languages are much slower than compiled languages
            1. Low performance is no longer acceptable in modern applications
            2. JS:
                1. It is not interpreted language anymore
                    1. Modern JS engines use a mix of compilation and interpretation (JIT)
                        1. JIT compilation
                            1. Just-in-time (JIT) compilation: **Entire code is converted into machine code at once, then executed immediately**

3. Just-in-time (JIT) compilation: Entire code is converted into machine code at once, then executed immediately

```
Source code
    |
    Step 1
    Compilation
    |
    v
Machine code
(not a portable file)
    |
    Step 2
    Execution
    |
    v
Program running
```

    1. A lot faster

5. Modern Just-In-Time Compilation of JavaScript:
    1. When JS code enters the engine, it parses the code (reads the code)
        1. The code is parsed into a data-structure called the AST (abstract syntax tree)
            1. Parses each line of code into pieces that are meaningful to the language (`const`, `function`, ...)
                1. Saves the pieces in a tree
            2. The step checks if there are any syntax errors
            3. The tree will be used to generate machine code
            4. Example:

```
const x = 23;

AST:

- VariableDeclaration {
    start: 0
    end: 13
    - declaration: [
        - VariableDeclarator {
            start: 6
            end: 12
            - id: Identifier {
                start: 6
                end: 7
                name: "x"
            }
            - init: Literal = $node {
                start: 10
                end: 12
                value: 23
                raw: "23"
            }
        }
        kind: "const"
    ]
}
```

2. Compilation: JS Engine takes the generated AST and compiles into machine code
    1. The machine code gets executed right-away (JIT)
        1. Execution happens in JS **call-stack**
    2. Clever execution strategy
        1. Initially un-optimized version of machine code in the beginning (to enable faster execution)
        2. The code is them optimized and then re-compiled during the already running program execution
            1. Each time it is repeated, the un-optimized code is swapped with a more optimized code (makes V8 fast)
            2. Happens in special threads that we can't access from code
                1. Separate from main thread running in the call-stack

2. What is a JavaScript Runtime?
    1. Runtime in the Browser
        1. A big container which includes all the things that we need to use JavaScript (in this case in the browser)
            1. The heart of a runtime is: JavaScript Engine
            2. Web APIs: Functionalities provided to the engine (not part of JS language), accessible on `window` object (JS gets access to the APIs)
                1. DOM
                2. Timers
                3. Fetch API
                4. ...
            3. Callback Queue
                1. Data-structure that contains all the callback functions that are ready to be executed
                    1. Example: Callback function from DOM event listener ('click', ...)
                        1. Callback function is called when an event occurs
                2. Event-Loop:
                    1. It takes callback function from the callback queue and puts them into the call-stack so that they can be executed
                    2. It is essential for non-blocking concurrency model
        2. Runtime in Node.JS
            1. Similar but instead of Web APIs, we get **C++ bindings & thread pool**

3. How is JS code translated to machine code?

Execution Contexts and The Call Stack

1. What is an execution context?
    1. Assume the code is compiled
    2. Then The following happens:

```
Creation of global execution context (for top-level code)
(Top-level code: Code not inside any code - only code outside functions are executed)
    Example:

            const name = 'Jonas';

            const first = ...;

    Execution Context: Environment in which a piece of JavaScript is executed. Box that stores all the necessary information for some code
        Exectution: Pizza in a box that comes with cutlery, receipt (to pay)
            Eating of Pizza happens inside the box

        Exactly one global execution context (EC) (for any JS project): Default context (it exists as), created for code that is not insic

Execution of top-level code (inside global EC)
    It is just CPU processing the machine code

Execution of functions and waiting for callbacks
    For each and every function call, a new execution context is created containing all the information necessary to run that function (sa

    The execution contexts make up the call-stack - one execution context per function: For each function call, a new execution context is

    When all functions are done executing, the engine waits for callback functions to arrive
        Callback function associated with a click event (event-loop provides the callback functions)
```

Execution Context Detail

1. What's inside execution context?
    1. Variable environment
        1. `let`, `const`, and `var` declarations are stored
        2. Functions
        3. `arguments` object
            1. Contains all the arguments passed into the function that the current execution context belongs to
                1. Each function gets its own execution context as soon as it is called
            2. All the variables declared in a function will end up in the execution context
        4. A function can also access variables outside the function
            1. Works due to **scope chain**
    2. Scope chain
        1. It consists of references to variables located outside the current function
        2. To keep track of the scope chain, it is stored in each execution context
    3. `this` keyword
        1. Each context gets `this` variable
2. Execution context is generated during creation phase (happens right before execution)
    1. Execution contexts belonging to arrow functions do not get their own `arguments` object and `this` keyword
        1. Instead they can use `arguments` and `this` keyword from their closest regular function parent
3. Example: Creation phase

```
const name = 'Jonas';

const first = () => {
    let a = 1;
    const b = second(7, 9);
    a = a + b;
    return a;
}

function second(x, y) {
    var c = 2;
    return c;
}

const x = first();
```

1. Global execution context
    1. name = 'Jonas'
    2. first = - literally the function code
    3. second = - literally the function code
    4. x = - needs to run `first()` first (mared as unknown)
        1. Technically, values only become known during execution
2. `first()` execution context
    1. a = 1
    2. b =
3. `second()` execution context
    1. c = 2
    2. arguments = [7, 9]
        1. Array of passed arguments. Available in all "regular" functions (not arrow)

4. The Call Stack

    1. Answers:

        1. How will engine keep track of the order in which functions are called?
        2. How will it know where it is currently in the execution

    2. Call stack along with memory heap makes up the JS engine

        1. "Place" where execution contexts get stacked on top of each other, to keep track of where we are in the execution
            1. The execution context on top of the stack is the one currently running
            2. When it finishes execution, the execution context will be removed from the stack, and previous execution context will resume execution
        2. It is like stacking pizza boxes to keep track of which pizza belongs to which friend

    3. Example:

        ```
        const name = 'Jonas';

        const first = () => {
            let a = 1;
            const b = second(7, 9);
            a = a + b;
            return a;
        }

        function second(x, y) {
            var c = 2;
            return c;
        }

        const x = first();
        ```

        1. A global execution context is created for the top-level code
            1. Code outside of any function is executed
            2. The global execution context is put into the call-stack
        2. When `first()` function is called, it gets its own execution context and it is pushed on top the current context (new current execution context)
        3. When `second()` function is called, it gets its own context and it is pushed on top of the current context
            1. Only this function is executed and no other function will be executed
                1. Since JS has only one thread of execution
            2. `return` statement makes execution context to pop out of the text
                1. It might keep living in memory (later)
        4. The previous execution context will be active again
            1. Automatically goes back to the function from which the new execution context was created
                1. It ensures order of execution never gets lost
        5. When the `first()` function returns, the current execution context pops off the stack and previous context is now the current context (global execution context)
            1. Returned value is assigned to x
            2. Execution is finished
                1. The execution context stays until it is eventually finished
                    1. Happens only when be close the browser tab or browser window
                        1. Only in this case the global execution context is popped

Scope and The Scope Chain

Scoping and Scope in JavaScript: Concepts

1. Scope Concepts:
    1. **Scoping**: How our program's variables are organized and accessed. "Where do variables live?" or "Where can we access a certain variable, and where not?"
        1. **Lexical scoping:** Scoping (the way variables are organized and accessed) is controlled by **placement** of functions and blocks in the code (where we write functions or code-blocks)
            1. Example: A function written inside another function has access to variables in the parent function

2. **Scope**: Space or environment in which a certain variable is **declared** (variable environment in case of functions). There is **global** scope, **function** scope, and **block** scope
3. **Scope of a variable**: Region of our code where a certain variable can be **accessed** (Not the same as **scope**)

The 3 Types of Scope

1. Scope is applicable for variables and functions (stored as values in variables)
2. Scopes:
    1. Global Scope
        1. Outside of any function or block
        2. Variables declared in global scope are accessible **everywhere** (all functions and all blocks)
        3. Examples:

```
const me = 'Jonas';
const job = 'teacher';
const year = 1989;
```

    2. Function Scope
        1. Variables are accessible only **inside function, NOT** outside
        2. Also called local scope
        3. Example:

```
function calcAge(birthYear) {
    const now = 2037;
    const age = now - birthYear;
    return age;
}

console.log(now); // ReferenceError
```

            1. Not accessible outside the function (similar to variable environment but there is block-scope as well)
            2. Function delcarations, function expressions, and arrow functions define their own scope

    3. Block Scope (ES6)
        1. Variables are accessible only **inside block** (block scoped) (only in which they were created)
        2. **However**, this only applies to `let` and `const` variables! (`var` variables are accessible outside the block - scoped to current function or global scope)
        3. Functions are **also block scoped** (only in strict mode)
        4. Example:

```
if (year >= 1991 && year <= 1996) {
    const millenial = true;
    const food = 'Avacado toast';
}

console.log(millenial); // ReferenceError
```

            1. ES5 had only global and function scopes
            2. ES6 has introduced block scope
                1. In ES6, all functions are block-scoped (in strict mode)
                    1. If a function is declared inside a block, it is accessible only in the block
                    2. Common in other programming languages
                        1. Function scope is wierd for beginners and hence block-scope was introduced

The Scope Chain

1. Example:

```
const myName = 'Jonas';

function first() {
    const age = 30;

    if (age >= 30) { // true
        const decade = 3;
        var millenial = true;
    }

    function second() {
        const job = 'teacher';

        console.log(`${myName} is a ${age}-old ${job}`);
        // Jonas is a 30-old teacher
    }

    second();
}

first();
```

    1. Global scope (considering only variable declarations)

```
myName = 'Jonas'
```

    2. `first(0)` scope

```
age = 30
millenial = true; // var is function scoped in this case
```

1. `second()` scope

   ```
   job = 'teacher'

   console.log(`${myName} is a $age-old ${job}`);
   ```

   1. **Every scope always has access to all the variables from all outer scopes**
      1. `second()` scope can access `age`, `myName`
         1. Since it has access to variables in the `first()` scope (applicable to arguments as well)
      2. `first()` can access `myName`
   2. If a scope is unable to find a variable in the current scope, it will look up in the scope-chain to see if it can find in one of the parent scopes (**variable lookup**) (variables are not copied from scope to scope, they are just looked up until they find the variable in the scope chain) (it doesn't work the other-way around - `first()` will never get access to the `second()` scope) (blocks get access to all outer scopes)
      1. If it can, it will use the variable
      2. If it cannot, there will be an error

2. `if` block scope

   ```
   decade = 3
   ```

3. Global scope variables are accessible everywhere because they are at the top of the scope-chain
   1. They are called global variables
4. `if` block scope has no access to `second()` scope variables and vice-versa (sibling-scopes) (they are not inside one another)
   1. Why? Lexical scoping
      1. Access to variables depends on where the scope is placed (where it is written in the code)

Scope Chain vs. Call Stack

1. Example:

```
const a = 'Jonas';
first();

function first() {
    const b = 'Hello!';
    second();

    function second() {
        const c = 'Hi!';
        third();
    }
}

function third() {
    const d = 'Hey!';
    console.log(d + c + b + a);
    // ReferenceError
}
```

   1. Call stack: (order in which functions were called)

      ```
      third() EC
          d = "Hey!"

      second() EC
          c = "Hi!"

      first() EC
          b = "Hello!"
          second = <function> // this is variable

      Global EC
          a = "Jonas"
          first = <function>
          third = <function>
      ```

   2. Scope chain:

      ```
      Global Scope
          a = "Jonas"
          first = <function>
          third = <function>
              ^                                       ^
              |                                       |
      first() scope                           third() scope
          b = "Hello!"                            d = "Hey!"
          second = <function>                     -------------------
          -------------------                     a = "Jonas"
          a = "Jonas"                                     first = <function>
          first = <function>                  third = <function>
          second = <function>
              ^
              |
      second() scope (has access to all parent scopes)
          c = "Hi!"
          -------------------
          b = "Hello!"
          second = <function>
          -------------------
          a = "Jonas"
          first = <function>
          third = <function>
      ```

        1. Scope chain - order in which functions are **written in the code**
            1. Has **nothing** to do with order in which functions were **called!**
                1. Has nothing to do with order of execution contexts in the callstack
        2. `third()` is a global function so it is accessible everywhere (even in `second()`)
        3. `c` is not in the scope chain of `third()`
        4. `b` is not in the scope chain of `third()`

    2. Summary:

        1. Scoping asks the question "Where do variables live?" or "Where can we access a certain variable, and where not?"
        2. There are 3 types of scope in JavaScript:
            1. The global scope
            2. Scopes defined by functions
            3. Scopes defined by blocks
        3. Only `let` and `const` variables are block-scoped.
            1. Variables declared with `var` end up in closest function scope
        4. In JavaScript, we have lexical scoping, so the rules of where we can access variables are based on exactly where in the code functions and blocks are written
        5. Every scope always has access to all the variables from all its outer scopes. This is the scope chain!
        6. When a variable is not in the current scope, the engine looks up in the scope chain until it finds the variable it's looking for. This is called variable lookup
        7. The scope chain is a one-way street: a scope will never, ever have access to the variables of an inner scope (only of outer scopes)
        8. The scope chain in a certain scope is equal to adding together all the variable environments of all the parent scopes
        9. The scope chain has nothing to do with the order in which functions were called. It does not affect the scope chain at all!

Scoping in Practice

1. 08-Behind-the-Scenes

```
'use strict';

function calcAge(birthYear) {
  // deined in global scope
  // function scope with VE
  const age = 2037 - birthYear;
  console.log(firstName); // in the scope chain (global variable), gets printed
  // `firstName` was defined after this function but not a problem

  function printAge() {
    let output = `${firstName}, You are ${age}, born in ${birthYear}`; // `age` and `birthYear` are in parent scope
    // Scope of a variable is the entire region of the code the variable is accessible
    // `firstName` is in the global scope
    console.log(output);

    if (birthYear >= 1981 && birthYear <= 1996) {
      var millenial = true;
      // Constructing NEW variable with same name as outer scope's variable
      const firstName = 'Steven'; // Lookup is from current-scope up, so JS will use the one in the current scope (because the lookup is from b
      // They are different variables with the same name. Mechanism used for parameters.
      const str = `Oh, and you're a millenial, ${firstName}`;
      console.log(str);

      function add(a, b) {
        return a + b;
      }

      // Reassiging outer scope's variable
      output = 'NEW OUTPUT!';
    }

    console.log(str); // `str` is in block scoped in the `if` block
    console.log(millenial); // `var` variables are function scoped (they ignore the block), older code-bases use this type of variable
    // add(2, 3); // doesn't work because functions are block-scoped in strict mode
    console.log(output); // We get 'NEW OUTPUT!' since parent-scoped variable is manipulated in the inner-scope
  }

  printAge();

  return age;
}

const firstName = 'Jonas';
calcAge(1991);

// console.log(age); // not accessible (it is in the inner scope)
// printAge(); // no access
```

Variable Environment: Hoisting and the TDZ

1. Hoisting in JavaScript

    1. An execution context contains 3 parts:

        1. Variable environment
        2. Scope chain
        3. `this` keyword

    2. **Hoisting**: Makes some types of variables accessible/usable in the code before they are actually declared. "Variables lifted to the top of their scope" (hoisting is not lifting up variables to the top of the scope behind the scenes)

        1. Behind the scenes:
            1. **Before execution**, code is scanned for variable declarations (creation phase of the execution context), and for each variable, a new property is created in the **variable environment object**
        2. Hoisting works differently for different kind of variables
            1. Function declarations
                1. Hoisted?

1. Yes
    1. It is because of hoisting
2. Initial value (in VE)
    1. Set to the actual function
        1. Enables us to use function decl
3. Scope
    1. Block (For strict mode. Otherwise: function scoped!)

2. `var` variables
    1. Hoisted?
        1. Yes
    2. Initial value
        1. `undefined`
            1. If we try to access `var` variable before it is declared in the code, we get `undefined` (not error)
                1. Common source of bugs in JS (hence we don't use `var`)
    3. Scope
        1. Function
3. `let` and `const` variables
    1. Hoisted?
        1. No
            1. Technically, yes, But not in practice
                1. Initialized to , TDZ
                2. It is as good as hoisting not happening
    2. Initial value
        1. , TDZ
            1. Placed in TDZ - Temporal Dead Zone
                1. Ensures that we cannot access the variable from beginning of the scope to the place where it is declared
            2. We get an error if we try to access
    3. Scope
        1. Block
            1. They exist in the block there were created
4. `function` expressions and arrows
    1. Hoisted?
        1. It depends on using `var` or `let`/`const`
            1. They are similar to variables
                1. `var` - Hoisted to `undefined`
                2. `let`/`const` - Not usable before declared in the code
    2. Initial value
        1. It depends on using `var` or `let`/`const`
    3. Scope
        1. It depends on using `var` or `let`/`const`

3. Temporal Dead Zone, Let and Const

```
const myName = 'Jonas';

if (myName === 'Jonas') {
    console.log(`Jonas is a ${job}`); // This is in TDZ - ReferenceError: Cannot access 'job' before initialization (Engine knows that 'jc
    const age = 2037 - 1989;
    console.log(age);
    // it is TDZ upto this point
    const job = 'teacher'; // accessible only from here
    console.log(x); // ReferenceError: x is not defined
}
```

    1. Why TDZ?
        1. **Makes it easier to avoid and catch errors**: accessing varables before declaration is bad practice and should be avoided.
        2. **Makes `const` variables actually work**
            1. We cannot assign `undefined` first and then assign real value later (it should not be re-assigned by definition)
    2. Why Hoisting?
        1. For using functions before actual declaration
            1. Enables certain programming techniques such as mutual recursion
            2. It can make code more readable
        2. `var` hoisting is just a byproduct of hoisting functions
            1. JS was not intended to become the huge programming language it has become today

Hoisting and TDZ in Practice

1. Example:

```
console.log(me);
console.log(job);
console.log(year);

var me = 'Jonas';
let job = 'teacher';
const year = 1991;

console.log(addDecl(2, 3)); // works
console.log(addExpr(2, 3)); // it is just a variable and it is in TDZ
console.log(addArrow(2, 3)); // this is in TDZ

function addDecl(a, b) {
  return a + b;
}

const addExpr = function (a, b) {
  return a + b;
};

const addArrow = (a, b) => a + b;
```

2. Pitfall of hoisting:

```
if (numProducts) deleteShoppingCart(); // `numProducts` is undefined at this place

var numProducts = 10;

function deleteShoppingCart() {
  console.log('All products deleted!');
}
```

1. All products are deleted - due to hoisting
2. **Tips to avoid pitfalls:**
    1. Don't use `var`
    2. Use `const` and `let` only if we need to change the value
    3. Define variables at the top of each scope (for clean code)
    4. Always declare functions first, and only use them after the declaration (all types of functions)

3. Another difference:

```
var x = 1;
let y = 2;
const z = 3;

// Open `window` in console
// We can see `x = 1` - It is `var`
// Property is created on `window` object if we use `var`

console.log(x === window.x); // true
```

The this Keyword

How the this Keyword Works

1. `this` keyword/variable: Special variable that is created for every execution context (every function) (it is one of the 3 components of EC). Takes the value of (points to) the "owner" of the function in which the `this` keyword is used.
2. `this` is NOT static. It depends on how the function is called, and its value is only assigned when the function **is actually called**.
3. 4 different ways a function can be called:
    1. **Method**: `this` =
        1. Function attached to an object
        2. It points to the object calling the method
        3. Example:

```
const jonas = {
    name: 'Jonas',
    year: 1989,
    calcAge: function () {
        return 2037 - this.year;
    }
}
jonas.calcAge(); // 48
```

            1. Way better solution
    2. **Simple function call**: `this` = `undefined`
        1. Only for strict mode
            1. If not, it will point to global object (`window` object in browser)
    3. **Arrow functions**: `this` = <this of surrounding unction (lexical `this`)>
        1. They don't get their own `this` keyword
            1. It gets picked up from outer lexical scope of the arrow function
    4. **Event listener**: `this` =
    5. **new, call, apply, bind**: later
    6. `this` does not point to the function itself
    7. `this` is also NOT its variable environment

The this Keyword in Practice

1. Example:

```
calcAgeArrow(1980);

const jonas = {
  year: 1991,
  calcAge: function () {
    console.log(this);
    console.log(2037 - this.year); // references the object
  },
};

jonas.calcAge(); // `jonas` object. jonas was the object calling the method

const matilda = {
  year: 2017,
};

matilda.calcAge = jonas.calcAge; // method borrowing
matilda.calcAge();

const f = jonas.calcAge;

f(); // `undefined`
```

Regular Functions vs. Arrow Functions

1. When to use when:

```
var firstName = 'Matilda';

const jonas = {
  firstName: 'Jonas',
  year: 1991,
  calcAge: function () {
    console.log(this);
    console.log(2037 - this.year); // references the object

    const self = this; // self or that
    const isMillenial = function () {
      console.log(this);
      console.log(this.year >= 1981 && this.year <= 1996); // `this` is `undefined`, inside a regular function call, `this` must be
    };
    console.log(self.year >= 1981 && self.year <= 1996); // it is in the scope chain
    isMillenial();

    // Solution 2
    const isMillenialArrow = () => {
      console.log(this);
      console.log(this.year >= 1981 && this.year <= 1996); // `this` is from surrounding function which is `jonas`
    };
  },

  greet: () => console.log(`Hey ${this.firstName}`),
}; // This is not a code-block. It is only object literal

jonas.greet(); // `undefined` - Parent scope is global scope
console.log(this);
console.log(this.firstName); // returns `undefined` - if no `firstName` on `window` object. If defined, then returns the property va

// if not in strict mode, `this.firstName` returns `Matilda`

// Never use arrow function as a method

// Function inside a method

// Arguments - functions also get access to `arguments` keyword. Only available in regular functions

const addExpr = function (a, b) {
  console.log(arguments); // useful when we want function to accept more arguments than we specified. We can pass more arguments tha
  return a + b;
};
addExpr(2, 5); // stored in `arguments` keyword
addExpr(2, 5, 8, 12); // all arguments are stored in the `arguments` array

var addArrow = (a, b) => {
  console.log(arguments); // `arguments` was not defined
  a + b;
}; // does not have `arguments` array

// `arguments` is not important in modern JS
```

Primitives vs. Objects (Primitive vs. Reference Types)

1. Example:

```
// Primitives vs Objects
let age = 30;
let oldAge = age;
age = 31;
console.log(age);
console.log(oldAge); // keeps the old value

const me = {
  name: 'Jonas',
  age: 30,
};

const friend = me;
friend.age = 27;
console.log('Friend:', friend);
console.log('Me:', me); // `age` changed to 27
```

Review: Primitives, Objects, and the Javascript Engine

1. Primitives:
    1. Types:
        1. Number
        2. String
        3. Boolean
        4. Undefined
        5. Null
        6. Symbol
        7. BigInt
    2. Memory management
        1. Primitive types
            1. Stored in call-stack
                1. In execution contexts in which they are declared
2. Objects
    1. Types:
        1. Object literal
        2. Arrays
        3. Functions
        4. ...

    2. Memory management
        1. Reference types
            1. They get stored in memory heap

3. Example:

```
let age = 30;
let oldAge = age;
age = 31;
console.log(age); // 31
console.log(oldAge); // 30

Call Stack:
    Id     | Address | Value
    age -->| 0001    | 30
              ^
              |
    oldAge----+


    Id      | Address  | Value
    age     | 0001     | 30
        \     ^
         +----|--+
       +------+  |     |
       |         v     |
    oldAge  |   0002   | 31
```

```
1. age - equals memory address that holds the value 30
2. `oldAge` points to the same memory address
3. When the value of `age` is changed, the memory is immutable, so a new memory is allocated and `age` points to new address
```

4. Example: Objects

```
const me = {
  name: 'Jonas',
  age: 30,
};

const friend = me;
friend.age = 27;
console.log('Friend:', friend);
console.log('Me:', me); // `age` changed to 27


Call Stack:            Heap:
                 Address | Value
         +----->        D30F     {
         |                                      name: 'Jonas';
         |                                      age: 30;
me 0003 D30F                     }
    ^
    |
friend
```

1. A reference to memory address in heap in created in Call stack
    1. Why?
        1. Objects might be too large to be stored in the stack (heap - almost unlimited memory pool)
2. `friend` is exactly same as the `me` object
3. `friend.age` to 27 changes the value in the heap
    1. Allows manipulating objects in heap (even if it is `const`)
    2. Changes `me` object's `age` as well
4. Copying points the second variable point to the same memory address

How Javascript Works Behind the Scenes Topics for Later

1. **Prototypal inheritance** - Object Oriented Programming (OOP) with JavaScript
2. **Event Loop** - Asynchronous JavaScript: Promises, Async/Await and AJAX
3. **How the DOM Really Works** - Advanced DOM and Events

Primitives vs. Objects in Practice

1. Example:

```
// Primitive types
let lastName = 'Williams';
let oldLastName = lastName;
lastName = 'Davis';

console.log(lastName, oldLastName);

// Reference types
const jessica = {
  firstName: 'Jessica',
  lastName: 'Williams',
  age: 27,
};

const marriedJessica = jessica;
marriedJessica.lastName = 'Davis';

console.log('Before marriage: ', jessica);
console.log('After marriage: ', marriedJessica); // Both show `lastName` as `Davis`, only the value in the stack is `const`

// marriedJessica = {}; // not allowed, reference will change which cannot be changed in stack

// Copying objects
const jessica2 = {
  firstName: 'Jessica',
```

```
  lastName: 'Williams',
  age: 27,
  family: ['Alice', 'Bob'],
};

const jessicaCopy = Object.assign({}, jessica2); // merges the two objects and returns a new object, **(M)**, all properties were co
// It only works at first-level. Doesn't copy inner objects - only shallow copy (not a deep clone)

jessicaCopy.lastName = 'Davis';

console.log(jessica2);
console.log(jessicaCopy);

jessicaCopy.family.push('Mary', 'John');

console.log(jessica2);
console.log(jessicaCopy);

// Deep-clone: Using lodash
```

Memory Management: Garbage Collection

Review: The Memory Lifycycle

1. Allocate memory: `let temp = 23.7`
2. Use memory:

```
temp = temp + 5;
round(temp);
```

3. `temp` is removed from memory
    1. How is it done?

Garbage Collection

1. How is memory freed up after we no longer need a value?
    1. Call Stack
        1. Simple:
            1. Variable environment is **simply deleted when EC pops off stack**
            2. Global variables will never be deleted (stay in stack for ever)
    2. Heap
        1. Garbage collection (central memory managemet tool)
            1. It's an engine that runs automatically whenever it sees fit
            2. Mark-and-Sweep Algorithm:
                1. Suppose stack has references to objects in the heap
                2. The objects in the heap may have references to other objects in the heap
                3. **Mark**: Mark all objects that are reachable from a root as "alive"
                    1. Root: Starting points
                        1. Global EC
                        2. Function EC
                        3. Active event listener or timer
                            1. Reachable from an event listener or timer
                        4. Closure
                4. **Sweep**: Delete un-marked (**unreachable**) objects and reclaim memory for future allocations
            3. When does the GC happen?
                1. It depends on
                    1. How much memory is available
                    2. How much memory is being consumed
                    3. Which JS engine the browser is using
                    4. ...
            4. The objects referenced from the Global Execution Context can never be deleted
        2. Memory leak:
            1. When objects that are no longer needed are **incorrectly still reachable**, and therefore **not being garbage collected**
                1. Objects are marked alive and no longer deleted
                    1. Source:
                        1. Unnecessary event listeners
                            1. Needs to be de-activated when no longer needed.
                        2. Unnecessary timers
                            1. Developer needs to actively delete the timer when it is no longer needed (or else the objects created in the timer are always reachable)
                        3. Large global objects
                            1. Avoid declaring unnecessary large global objects (they can never be garbage collected)
                                1. Can lead to memory leaks
            3. In reality, memory management is a huge topic:
                1. There are multiple heaps.
                2. Objects are moved from one heap to another depending on their age
                3. GC has multiple algorithms for different heaps
                    1. Generational GC
                    2. ...
2. How JS Works Behind the Scenes Topics for Later:
    1. Closures - A closer look at functions
    2. Prototypal Inheritance - Object Oriented Programming (OOP) with JS
    3. Event Loop - Asynchronous JS: Promises, Async/Await, and AJAX
    4. How the DOM Really Works - Advanced DOM and Events

Section 9: Data Structures, Modern Operators and Strings

Section Info

1. Basic JS features and Syntax
    1. More modern JS
        1. Objects
        2. Maps
        3. Arrays
    2. ES6+ operators
        1. Structuring
        2. Optional chaining
    3. Strings

Section Roadmap

Destructuring Arrays

1. 09-Data-Structures-Operators

2. Destructuring:

    1. ES6 feature.
    2. A way of un-packing values from an array or an object into separate variables
        1. Breaking a complex data-structure into smaller data-structures

3. Example:

```
// Data needed for first part of the section
const restaurant = {
  name: 'Classico Italiano',
  location: 'Via Angelo Tavanti 23, Firenze, Italy',
  categories: ['Italian', 'Pizzeria', 'Vegetarian', 'Organic'],
  starterMenu: ['Focaccia', 'Bruschetta', 'Garlic Bread', 'Caprese Salad'],
  mainMenu: ['Pizza', 'Pasta', 'Risotto'],

  openingHours: {
    thu: {
      open: 12,
      close: 22,
    },
    fri: {
      open: 11,
      close: 23,
    },
    sat: {
      open: 0, // Open 24 hours
      close: 24,
    },
  },

  order: function (starterIndex, mainIndex) {
    return [this.starterMenu[starterIndex], this.mainMenu[mainIndex]];
  },
};

const arr = [2, 3, 4];
const a = arr[0];
const b = arr[1];
const c = arr[2];

const [x, y, z] = arr; // [] because we are destructuring an array, [] destructuring assignment
// The original array is not affected
console.log(x, y, z);

const [first, second] = restaurant.categories; // we don't have to take all elements of an arrays
console.log(first, second);

let [primary, , secondary] = restaurant.categories; // We can skip in-between, and items will be read in that order

// Without destructuring
// const temp = primary;
// primary = secondary;
// secondary = temp;
// console.log(primary, secondary);

[secondary, primary] = [secondary, primary]; // no need for temporary variable
console.log(primary, secondary);

// Function can return an array and we can destructure the array into different variables - allows returning multiple values from a

const [starter, mainCourse] = restaurant.order(2, 0);
console.log(starter, mainCourse);

// Nested destructuring
const nested = [2, 4, [5, 6]];
// const [i, , j] = nested;
const [i, , [j, k]] = nested;
console.log(i, j, k);

// Default values for variables - useful when we don't know the length
const [p, q, r] = [8, 9];
console.log(p, q, r); // we get `undefined` for `r`

const [s = 1, t = 1, u = 1] = [8, 9];
console.log(s, t, u);
```

Destructuring Objects

1. Example:

```
// Data needed for first part of the section
const restaurant = {
  name: 'Classico Italiano',
  location: 'Via Angelo Tavanti 23, Firenze, Italy',
  categories: ['Italian', 'Pizzeria', 'Vegetarian', 'Organic'],
  starterMenu: ['Focaccia', 'Bruschetta', 'Garlic Bread', 'Caprese Salad'],
  mainMenu: ['Pizza', 'Pasta', 'Risotto'],

  openingHours: {
    thu: {
      open: 12,
      close: 22,
    },
    fri: {
      open: 11,
      close: 23,
    },
    sat: {
      open: 0, // Open 24 hours
      close: 24,
    },
  },

  order: function (starterIndex, mainIndex) {
    return [this.starterMenu[starterIndex], this.mainMenu[mainIndex]];
  },

  orderDelivery: function ({
    starterIndex = 1,
    mainIndex = 0,
    time = '20:00',
    address,
  }) {
    // as we receive the object, we immediately do destructuring (the names must match), the order doesn't have to be matched
    console.log(
      `Order received! ${this.starterIndex[starterIndex]} and ${this.mainMenu[mainIndex]} will be delivered to ${address} at ${time}
    );
  },
};

restaurant.orderDelivery({
  time: '22:30',
  address: 'Via del Sole, 21',
  mainIndex: 2,
  starterIndex: 2,
});

restaurant.orderDelivery({
  address: 'Via del Sole, 21',
  starterIndex: 1,
}); // Default values are taken by variables that are not specified in the object
// Good technique for a lot of params

const arr = [2, 3, 4];
const a = arr[0];
const b = arr[1];
const c = arr[2];

const [x, y, z] = arr; // [] because we are destructuring an array, [] destructuring assignment
// The original array is not affected
console.log(x, y, z);

const [first, second] = restaurant.categories; // we don't have to take all elements of an arrays
console.log(first, second);

let [primary, , secondary] = restaurant.categories; // We can skip in-between, and items will be read in that order

// Without destructuring
// const temp = primary;
// primary = secondary;
// secondary = temp;
// console.log(primary, secondary);

[secondary, primary] = [secondary, primary]; // no need for temporary variable
console.log(primary, secondary);

// Function can return an array and we can destructure the array into different variables - allows returning multiple values from a

const [starter, mainCourse] = restaurant.order(2, 0);
console.log(starter, mainCourse);

// Nested destructuring
const nested = [2, 4, [5, 6]];
// const [i, , j] = nested;
const [i, , [j, k]] = nested;
console.log(i, j, k);

// Default values for variables - useful when we don't know the length
const [p, q, r] = [8, 9];
console.log(p, q, r); // we get `undefined` for `r`

const [s = 1, t = 1, u = 1] = [8, 9];
console.log(s, t, u);

// Destructuring Objects
const { name, openingHours, categories } = restaurant; // variable names must exactly match the property names
console.log(name, openingHours, categories);
// Useful when we deal with the result of an API call
// Allows us to write much less code

// Renaming properties
const {
  name: restaurantName, // **(M)**
  openingHours: hours,
```

```
  categories: tags,
} = restaurant;
console.log(name, openingHours, tags);

const { menu = [], starterMenu: starters = [] } = restaurant;
console.log(menu, starters);

// Mutating objects
let d = 111;
let e = 999;
const obj = { d: 23, e: 7, f: 14 };

// {d, e} = obj; // doesn't work because `{` `}` is a code-block
({ d, e } = obj);
console.log(d, e);

// Nested objects
const {
  fri: { open: op, close: cl },
} = openingHours;
// console.log(open, close);
console.log(op, cl);

// Data needed for first part of the section
const restaurant = {
  name: 'Classico Italiano',
  location: 'Via Angelo Tavanti 23, Firenze, Italy',
  categories: ['Italian', 'Pizzeria', 'Vegetarian', 'Organic'],
  starterMenu: ['Focaccia', 'Bruschetta', 'Garlic Bread', 'Caprese Salad'],
  mainMenu: ['Pizza', 'Pasta', 'Risotto'],

  openingHours: {
    thu: {
      open: 12,
      close: 22,
    },
    fri: {
      open: 11,
      close: 23,
    },
    sat: {
      open: 0, // Open 24 hours
      close: 24,
    },
  },

  order: function (starterIndex, mainIndex) {
    return [this.starterMenu[starterIndex], this.mainMenu[mainIndex]];
  },

  orderDelivery: function ({
    starterIndex = 1,
    mainIndex = 0,
    time = '20:00',
    address,
  }) {
    // as we receive the object, we immediately do destructuring (the names must match), the order doesn't have to be matched
    console.log(
      `Order received! ${this.starterIndex[starterIndex]} and ${this.mainMenu[mainIndex]} will be delivered to ${address} at ${time}
    );
  },
};

restaurant.orderDelivery({
  time: '22:30',
  address: 'Via del Sole, 21',
  mainIndex: 2,
  starterIndex: 2,
});

restaurant.orderDelivery({
  address: 'Via del Sole, 21',
  starterIndex: 1,
}); // Default values are taken by variables that are not specified in the object
// Good technique for a lot of params
```

The Spread Operator (...)

1. The spread operator works on all iterables (not just arrays)
    1. Iterables: arrays, strings, maps, sets, (not objects)
2. Example:

```
// Data needed for first part of the section
const restaurant = {
  name: 'Classico Italiano',
  location: 'Via Angelo Tavanti 23, Firenze, Italy',
  categories: ['Italian', 'Pizzeria', 'Vegetarian', 'Organic'],
  starterMenu: ['Focaccia', 'Bruschetta', 'Garlic Bread', 'Caprese Salad'],
  mainMenu: ['Pizza', 'Pasta', 'Risotto'],

  openingHours: {
    thu: {
      open: 12,
      close: 22,
    },
    fri: {
      open: 11,
      close: 23,
    },
    sat: {
```

```
    open: 0, // Open 24 hours
    close: 24,
  },
},

order: function (starterIndex, mainIndex) {
  return [this.starterMenu[starterIndex], this.mainMenu[mainIndex]];
},

orderDelivery: function ({
  starterIndex = 1,
  mainIndex = 0,
  time = '20:00',
  address,
}) {
  // as we receive the object, we immediately do destructuring (the names must match), the order doesn't have to be matched
  console.log(
    `Order received! ${this.starterMenu[starterIndex]} and ${this.mainMenu[mainIndex]} will be delivered to ${address} at ${time}`
  );
},

orderPasta: function (ing1, ing2, ing3) {
  console.log(
    `Here is your delicious pasta with ${ing1}, ${ing2}, and ${ing3}`
  );
},
};

const ingredients = [
  prompt("Let's make pasta! Ingredient 1?"),
  prompt("Let's make pasta! Ingredient 2?"),
  prompt("Let's make pasta! Ingredient 3?"),
];
console.log(ingredients);

restaurant.orderPasta(...ingredients); // good for long arrays

// spread operator works on objects in ES2018 (even though objects are not iterables)
const newRestaurant = { foundedIn: 1998, ...restaurant, founder: 'Guiseppe' };

console.log(newRestaurant);

const restaurantCopy = { ...restaurant }; // makes a copy
restaurantCopy.name = 'Ristorante Roma';
console.log(restaurantCopy.name); // has the new value
console.log(restaurant.name); // has the old value
```

Rest Pattern and Parameters

1. Similar to spread operator but does the opposite of spread operator
    1. Spread operator use-cases:
        1. For creation of new arrays
        2. For passing multiple parameters into functions
    2. Rest operator uses exactly the same operator to condense elements into an array
2. Example:

```
...
  orderPizza: function (mainIngredient, ...otherIngredients) {
    console.log(mainIngredient);
    console.log(otherIngredients);
  },
};

// REST - Takes the remaining elements of an array and put them in a new array

const [pizza, , risotto, ...otherFood] = [
  ...restaurant.mainMenu,
  ...restaurant.starterMenu,
];
console.log(pizza, risotto, otherFood); // Doesn't include skipped elements, only after the last variable. It should always be the l
// There can be only one rest element in a destructuring assignment

// Works for objects
const { sat, ...weekdays } = restaurant.openingHours;
console.log(weekdays); // Fri, Thu - rest of the elements

// Functions
const add = function (...numbers) {
  // accepts both arguments, and also an array (with spread operator)
  // rest arguments
  // stored in array
  // accepts any number of parameters
  console.log(numbers);
  let sum = 0;
  for (let i = 0; i < numbers.length; i++) {
    sum += numbers[i];
  }
  console.log(sum);
};
add(2, 3);
add(5, 3, 7, 2); // any number of arguments
add(8, 2, 5, 3, 2, 1, 4);

const x1 = [23, 5, 7];
add(...x1); // spread here and collected in the function

restaurant.orderPasta('mushroom', 'onion', 'olives', 'spinach'); // mainIngredient: 'mushroom', otherIngredients: ['onion', 'olives'
restaurant.orderPasta('mushroom'); // otherIngredients: []
```

Short Circuiting (&& and ||)

1. Example:

```
console.log(3 || 'Jonas'); // result is 3 - it doesn't have to be boolean
```

2. Properties of logical operators:
    1. They can use any data-type
    2. They can return any data-type
    3. They do short-circuiting (short-circuit evaluation)
        1. Short-circuiting:
            1. If the first value is a truthy value (||), then it returns immediately (JS will not evaluate the second value)
                1. Example:

```
console.log('' || 'Jonas'); // 'Jonas' (first operand is a falsy value
console.log(true || 0); // true
console.log(undefined || null); // null (the last value is returned)
console.log(undefined || 0 || '' || 'hello' || 23 || null); // hello (first truthy value)
```

                1. It returns truthy value
                2. The result is true if any one of the values is true

3. Example:

```
// Short-circuiting
const guests = restaurant.numGuests ? restaurant.numGuests : 10;
console.log(guests);

const guests1 = restaurant.numGuests || 10;
console.log(guests1);

// AND operator
console.log(0 && 'Jonas'); // AND operator short-circuits when the first value is falsy and then immediately returns the falsy value

console.log(7 && 'Jonas'); // Jonas. Evaluation continues and the last value is returned
// AND is true if only all the operands are true

console.log('Hello' && 23 && null && 'jonas'); // null - first falsy value

if (restaurant.orderPizza) {
  restaurant.orderPizza('mushrooms', 'spinach');
}

restaurant.orderPizza && restaurant.orderPizza('mushrooms', 'spinach'); // short-circuits if it does not exist. Caveat - may be hard
```

The Nullish Coalescing Operator (??)

1. Example:

```
// Nullish coelescing operator (ES2020)
restaurant.numGuests = 0;
const guests2 = restaurant.numGuests || 10;
console.log(guests2);

const guestsCorrect = restaurant.numGuests ?? 10;
console.log(guestsCorrect);

// Works with `null`ish values instead of falsy values: `undefined`, `null` only (0, '', are considered as truthy values)
```

Logical Assignment Operators

1. Example:

```
// ES2021 additions
const rest1 = {
  name: 'Capri',
  numGuests: 20,
};

const rest2 = {
  name: 'La Piazza',
  owner: 'Giovanni Rossi',
};

rest1.numGuests = rest1.numGuests || 10;
rest2.numGuests = rest2.numGuests || 10;

console.log(rest1);
console.log(rest2);

// OR assignment operator
rest1.numGuests ||= 10;
rest2.numGuests ||= 10;

console.log(rest1);
console.log(rest2);

rest1.numGuests = 0;

rest1.numGuests ??= 10; // nullish assignment operator
rest2.numGuests ??= 10;

rest1.owner = rest1.owner && '<ANONYMOUS>'; // undefined
rest2.owner = rest2.owner && '<ANONYMOUS>'; // <ANONYMOUS>

rest1.owner &&= '<ANONYMOUS>';
rest2.owner &&= '<ANONYMOUS>';
```

Coding Challenge #1

1.

Looping Arrays: The for-of Loop

1. Example:

```
// for-of loop:
const menu1 = [...restaurant.starterMenu, ...restaurant.mainMenu];

for (const item of menu1) {
  // counters and conditions are not required
  // **(M)**
  console.log(item);
  // We can use continue and break statements
}

for (const item of menu1.entries()) {
  // **(M)**
  console.log(item);
  console.log(`${item[0] + 1}: ${item[1]}`);

  for (const [i, el] of menu1.entries()) {
    console.log(`${i + 1}: ${el}`);
  }
}

console.log(menu1.entries()); // Array Iterator
console.log([...menu1.entries()]); // New array of arrays
```

Enhanced Object Literals

1. ES6 has introduced 3 ways to write object literals:
    1. Way 1:

    ```
    // openingHours: openingHours, // Prior to ES6
    openingHours, // ES6 enhanced object literals - constructs a property name with exactly the variable name
    ```

    2. Way 2:

    ```
    // order: function (starterIndex, mainIndex) {
    // return [this.starterMenu[starterIndex], this.mainMenu[mainIndex]];
    // },

    order(starterIndex, mainIndex) {
    // ES6 syntax - same as: order: function (starterIndex, mainIndex) {...}
    return [this.starterMenu[starterIndex], this.mainMenu[mainIndex]];
    },
    ```

    3. Way 3: Computing property names instead of writing them manually

    ```
    const weekdays = ['mon', 'tue', 'wed', 'thu', 'fri', 'sat', 'sun'];

    const openingHours = {
      [weekdays[3]]: {
        // Accepts any expression inside `[]`
        open: 12,
        close: 22,
      },
      [weekdays[4]]: {
        open: 11,
        close: 23,
      },
      [`day-${2 + 4}`]: {
        open: 0, // Open 24 hours
        close: 24,
      },
    };
    ```

Optional Chaining (?.)

1. Newer feature of objects and arrays
2. Example:

```
console.log(restaurant.openingHours.mon.open); // Assuming we don't know if restaurant open on monday or not
// Returns an error - `undefied`.open
// We need to check if `mon` exists

if (restaurant.openingHours.mon) console.log(restaurant.openingHours.mon.open); // more unreadable or messy. What if `openingHours`

if (restaurant.openingHours && restaurant.openingHours.mon)
  console.log(restaurant.openingHours.mon.open);

// ES2020 introduced optional chaining - If a property does not exist, then undefined is returned immediately

console.log(restaurant.openingHours?.mon?.open); // Only if the property before ? exists, then `open` is invoked. If property does n

const days = ['mon', 'tue', 'wed', 'thu', 'fri', 'sat', 'sun'];

// ES2020
for (const day of days) {
  // console.log(day);
  const open = restaurant.openingHours[day]?.open ?? 'closed';
  console.log(`On ${day}, we open at ${open}`);
```

```
}

// Methods - we can check if a method exists before we call it
console.log(restaurant.order?.(0, 1) ?? 'Method does not exist');
console.log(restaurant.orderRisotto?.(0, 1) ?? 'Method does not exist');

// Arrays
const users = [{ name: 'Jonas' }, { email: 'hello@jonas.io' }];

console.log(users[0]?.name ?? 'User array empty'); // Tests if value on the left exists or returns `undefined`
```

Looping Objects: Object Keys, Values, and Entries

1. Options based on what we want to loop over (property names, values, or both) - done in an indirect way
2. Example:

```
for (const day of Object.keys(openingHours)) {
  // **(M)**
  console.log(day);
}

console.log(`We are open on ${properties.length} days`);

let openStr = `We are open on ${properties.length} days`;
for (const day of properties) {
  openStr = `${day}, `;
}
console.log(openStr);

// Property values
const values = Object.values(openingHours); // **(M)**
console.log(values);

// Entries - names + values
const entries = Object.entries(openingHours); // **(M)**
console.log(entries); // key, value

for (const [key, { open, close }] of entries) {
  // console.log(x);
  console.log(`On ${key} we open at ${open} and close at ${close}`);
}
```

Coding Challenge #2

Sets

1. Sets and Maps were introduced in ES6
2. Set:
    1. A collection of unique values
        1. They cannot have any duplicates
    2. Easy to interact with sets (straight-forward methods)
    3. Not as important as arrays
3. Example:

```
const ordersSet = new Set([
  'Pasta',
  'Pizza',
  'Pizza',
  'Risotto',
  'Pasta',
  'Pizza',
]); // **(M)** We need to pass an iterable, Set can hold mixed data-types, Order of elements is irrelevant
console.log(ordersSet);
console.log(new Set('Jonas')); // since string is also an iterable
console.log(new Set()); // empty set
console.log(ordersSet.size); // **(M)**
console.log(ordersSet.has('Pizza')); // **(M)**
ordersSet.add('Garlic Bread'); // **(M)**
ordersSet.delete('Risotto'); // **(M)**
// There are no way to get values out of a set, Array can be used instead
// ordersSet.clear(); // **(M)**

for (const order of ordersSet) console.log(order);

// Example
const staff = ['Waiter', 'Chef', 'Waiter', 'Manager', 'Chef', 'Waiter'];
const staffUnique = [...new Set(staff)]; // an iterable, since Set is an iterable which supports spread operator
console.log(staffUnique);
console.log(
  new Set(['Waiter', 'Chef', 'Waiter', 'Manager', 'Chef', 'Waiter']).size
);

console.log(new Set('jonasschmedtmann').size);
```

Maps: Fundaments

1. Map: To map values to keys
    1. Data is stored in (key, value) pairs in maps
    2. Difference between Objects and Maps?
        1. Maps: Keys can be of any type
            1. It can be objects, arrays, other maps, etc.
        2. Objects: Keys can only be strings

2. Example:

```
const rest = new Map();
rest.set('name', 'Classico Italiano'); // **(M)**
rest.set(1, 'Firenze, Italy');
console.log(rest.set(2, 'Lisbon, Portugal')); // sets and returns the map - allows chaining

// chaining
rest
  .set('categories', ['Italian', 'Pizzeria', 'Vegetarian', 'Organic'])
  .set('open', 11)
  .set('close', 23)
  .set(true, 'We are open!')
  .set(false, 'We are closed!');

console.log(rest.get('name'));
console.log(rest.get(true)); // If key does not exist, `undefined` is returned

const time = 21;
console.log(rest.get(time > rest.get('open') && time < rest.get('close')));

console.log(rest.has('categories'));
rest.delete(2);
console.log(rest);
console.log(rest.size);
rest.clear();
console.log(rest);
console.log(rest.size);

rest.set([1, 2], 'Test');
console.log(rest);

console.log(rest.get([1, 2])); // doesn't work, because the arrays are not the same objects

const arr = [1, 2];
rest.set(arr, 'Test');

console.log(rest.get(arr)); // This works
// Useful for DOM elements

rest.set(document.querySelector('h1'), 'Heading');
console.log(rest);
```

Maps: Iteration

1. Code:

```
const question = new Map([
  ['question', 'What is the best programming language in the World?'],
  [1, 'C'],
  [2, 'Java'],
  [3, 'JavaScript'],
  ['correct', 3],
  [true, 'Correct 🎉'],
  [false, 'Try again!'],
]); // array of arrays of key/value pairs

console.log(question);
console.log(Object.entries(openingHours)); // Returns array or arrays

// Convert Object to Map
const hoursMap = new Map(Object.entries(openingHours));

console.log(question.get('question'));

// Maps are iterables and hence the for loop works

for (const [key, value] of question) {
  if (typeof key === 'number') console.log(`Answer ${key}: ${value}`);
}

const answer = Number(prompt('Your answer'));
console.log(answer);

console.log(question.get(question.get('correct') === answer));

// Converting Map to an array
console.log([...question]); // array of arrays
console.log(question.entries());
console.log(question.keys());
console.log(question.values());
```

Summary: Which Data Structure to Use?

1. Data-structures to choose

Data Structures Overview

1. Sources of Data
    1. From the program itself: Data written directly in source code (e.g. status messages)
    2. From the UI: Data input from the user or data written in DOM (e.g. tasks in todo app, expenses in a budget app)
    3. From external sources: Data fetched for example from web API (e.g. recipe objects)
        1. API: Application Programming Interface
            1. We can get data from other applications
                1. API to get weather data
                2. API to get about movies
                3. API to get currency conversion
2. No matter where the data comes from, we will have a collection of data that we want to store somewhere

        1. Where do we store?
            1. Data structures
                1. There are 4 data-structures
    3. How to choose?
        1. If simple list of values?
            1. Choose Arrays or Sets (no descriptions)
        2. If key/value pairs?
            1. Choose objects or maps (key allows us to describe values)
    4. Data from web-api comes as JSON
        1. JSON: A long string that can be converted to JS objects
            1. Since it has the same formatting as the JS objects and arrays
                1. If we don't need description of each recipe, so we store them as values in a list (array)
    5. Other data-structures
        1. Built-in:
            1. `WeekMap`
            2. `WeekSet`
        2. Non-built-in:
            1. Stacks
            2. Queues
            3. Linked lists
            4. Trees
            5. Hash tables

Arrays vs. Sets and Objects vs. Maps

1. Arrays vs Sets
    1. Arrays
        1. Use when you need **ordered** list of values (might contian duplicates)
        2. Use when you need to **manipulate** data (there are methods)
    2. Sets
        1. Use when you need to work with **unique** values
        2. Use when **high-performance** is really important
            1. Searching from an item or deleting an item is much faster in sets
        3. Use to **remove duplicates** from arrays
2. Objects vs Maps
    1. Objects
        1. More "traditional" key/value store ("abused" objects)
        2. Easier to write and access values with . and []
        3. Use-cases:
            1. Use when you need to include **functions** (methods)
                1. We can use `this` keyword when accessing properties (not possible with maps)
            2. Use when working with JSON (can convert to map)
    2. Maps
        1. Better performance
            1. For simple key-value stores
        2. Keys can have **any** data type
        3. Easy to iterate
        4. Easy to compute size
        5. Use-cases:
            1. Use when you simply need to map key to value
            2. Use when you need keys that are **not** strings

Coding Challenge #3

Working with Strings - Part 1

1. String methods
2. Examples:

```
const airline = 'TAP Air Portugal';
const plane = 'A320';

console.log(plane[0]); // char access using index
console.log(plane[1]);
console.log(plane[2]);

console.log('B737'[0]); // B
console.log(airline.length);

console.log(airline.indexOf('r'));
console.log(airline.lastIndexOf('r'));
console.log(airline.indexOf('Portugal')); // case-sensitive search

// Use of indexes - for slice (for extracting substrings)
console.log(airline.slice(4)); // **(M)**, 4 - begin param
console.log(airline.slice(4, 7)); // 7 - exclusive (length - 7 - 4 = 3)

console.log(airline.slice(0, airline.indexOf(' '))); // first word
console.log(airline.slice(airline.lastIndexOf(' ') + 1)); // last word

console.log(airline.slice(-2)); // start index = (length - 2)
console.log(airline.slice(1, -1)); // end index = (length - 1)

const checkMiddleSeat = function (seat) {
  const s = seat.slice(-1);
  if (s === B || s === E) {
    console.log('You got the middle seat 💥');
  } else {
    console.log('You got lucky 🎉');
  }
```

```
};

checkMiddleSeat('11B'); // B & E are middle seats
checkMiddleSeat('23C');
checkMiddleSeat('3E');
```

3. Why do strings have methods?
    1. **Whenever we call a method on a string primitive, JS will automatically convert the primitive into a string object with the same content** (Boxing)
    2. **After the operation is done, the object is converted into a primitive**
    3. All methods return string primitives (even if called on an object)

Working with Strings - Part 2

1. Code:

```
console.log(new String('jonas')); // Object
console.log(typeof new String('jonas')); // 'object'

console.log(typeof new String('jonas').slice(1)); // 'string'

console.log(airline.toLowerCase());
console.log(airline.toUpperCase());
console.log('airline'.toLowerCase());

// Fix capitalization in name
const passenger = 'jOnAS';
const passengerLower = passenger.toLowerCase();
const passengerCorrect =
  passengerLower[0].toUpperCase() + passengerLower.slice(1);
console.log(passengerCorrect);

const correctCapitalization = function (name) {
  const nameLower = name.toLowerCase();
  const nameCorrect = nameLower[0].toUpperCase() + nameLower.slice(1);
  return nameCorrect;
};

console.log(correctCapitalization('jonas'));
console.log(correctCapitalization('JONAS'));
console.log(correctCapitalization('jOnAs'));
console.log(correctCapitalization('abdullah'));

// Comparing emails
const email = 'hello@jonas.io';
const loginEmail = '    Hello@Jonas.Io \n'; // it is correct and valid

const lowerEmail = loginEmail.toLowerCase();
const trimmedEmail = lowerEmail.trim(); // **(M)**
console.log(trimmedEmail);

const normalizedEmail = loginEmail.toLowerCase().trim(); // chaining
console.log(normalizedEmail);
console.log(email === normalizedEmail);

// ES2019: trimStart(), trimEnd()
// Replacing
const priceGB = '288,97L'; // , is decimal separator
const priceUS = priceGB.replace('L', '$').replace(',', '.');
console.log(priceUS);

const announcement =
  'All passengers come to boarding door 23. Boarding door 23!';
console.log(announcement.replace('door', 'gate')); // only replaces first occurrance
console.log(announcement.replaceAll('door', 'gate')); // **(M)**

// Regex
console.log(announcement.replace(/door/g, 'gate')); // **(M)**

// Booleans
const plane1 = 'A320neo';
console.log(plane1.includes('A320')); // **(M)**
console.log(plane1.includes('Boeing'));

console.log(plane1.startsWith('Air')); // **(M)**
console.log(plane1.endsWith('neo')); // **(M)**

if (plane1.startsWith('Airbus') && plane1.endsWith('neo')) {
  console.log('Part of the new Airbus family');
}

// Practice exercise
const checkBaggage = function (items) {
  const baggage = items.toLowerCase();
  if (baggage.includes('knife') || baggage.includes('gun')) {
    console.log('You are not allowed on board');
  } else {
    console.log('Welcome aboard');
  }
};

checkBaggage('I have a laptop, some food and a pocket Knife');
checkBaggage('Socks and camera');
checkBaggage('Got some snacks and a gun for protection');
```

Working with Strings - Part 3

1. Split a string into multiple parts based on a divider string:

2. Go to mdn
    1. Search: mdn string replace
        1. We can see all string methods
3. Code:

```
console.log('a+very+nice+string'.split('+')); // **(M)** Stores result in an array
console.log('Jonas Schmedtmann'.split(' '));

const [firstName, lastName] = 'Jonas Schmedtmann'.split(' ');
console.log(firstName, lastName);

const newName = ['Mr.', firstName, lastName.toUpperCase()].join(' ');
console.log(newName);

const capitalizeName = function (name) {
  const names = name.split(' ');
  let namesUpper = [];
  for (const n of names) {
    // namesUpper.push(n[0].toUpperCase() + n.slice(1));
    namesUpper.push(n.replace(n[0], n[0].toUpperCase()));
  }
  console.log(namesUpper.join(' '));
};

const passenger1 = 'jessica ann smith davis';
capitalizeName(passenger1);

// Padding a string - adding characters until a string has the desired length
const message = 'Go to gate 23!';
console.log(message.padStart(25, '+').padEnd(30, '+')); // **(M)** total lenth is 30

// Credit card number masking
const maskCreditCard = function (number) {
  const str = number + '';
  const last = str.slice(-4);
  return last.padStart(str.length, '*');
};

console.log(maskCreditCard(4353857398231234));

// Repeat
const message2 = 'Bad weather... All Departures Delayed... ';
console.log(message2.repeat(5));

const planesInLine = function (n) {
  console.log(`There are ${n} plaes in line ${'✈️'.repeat(n)}`);
};

planesInLine(5);
planesInLine(3);
planesInLine(12);
```

Coding Challenge #4

1. Solution:

```
const snakeCaseToCamelCase = function (str) {
  const words = str.split('_');
  const capitalizedWords = [];
  for (const word of words) {
    capitalizedWords.push(word.replace(word[0], word[0].toUpperCase()));
  }
  return capitalizedWords.join('');
};

console.log(snakeCaseToCamelCase('hello_world'));
console.log(snakeCaseToCamelCase('this_is_a_big_ball_of_mud'));
```

String Methods Practice

Section 10: A Closer Look at Functions

Section Intro

1. Advanced topics
    1. Higher-order methods
    2. Bind method
    3. Closures

Section Roadmap

Default Parameters

1. 10-functions
2. Functions with default params
    1. Default values can contain any expression
        1. `price = 199 * 1.2`
    2. Default values can use values of other parameters we defined before it (not after it)
        1. `price = 199 * numPassengers`
    3. We cannot skip values in-between and set later values
        1. Work-around: pass `undefined`

3. Code:

```
const bookings = [];

const createBooking = function (
  flightNum,
  numPassengers = 1,
  price = 199 * numPassengers
) {
  // Above: ES6
  // Below: ES5
  // numPassengers ||= 1;
  // price ||= 199;

  const booking = {
    flightNum,
    numPassengers,
    price,
  };
  console.log(booking);
  bookings.push(booking);
};

createBooking('LH123');
// numPassengers: undefined, price: undefined (if no default values specified)

createBooking('LH123', 2, 800);
createBooking('LH123', 2);
createBooking('LH123', 5);
createBooking('LH123', undefined, 200);
```

How Passing Arguments Works: Value vs. Reference

1. Primitive types vs reference types for functions
2. Passing by value vs passing by reference
    1. JS does not have passing by reference
        1. Only passing by value
            1. C++ can pass a reference to any value instead of values (works even with primitives - even values outside the function can be changed)
            2. Reference itself is a value (it just contains a memory address)
                1. Reference itself is passed.
                2. We do not pass by a reference

3. Code:

```
const flight = 'LH123';
const jonas = {
  name: 'Jonas Schmedtmann',
  passport: 34243423,
};

const checkIn = function (flightNum, passenger) {
  // don't change params of a function - not a good practice
  flightNum = 'LH999'; // It is a copy of the original value
  // similar to: flightNum = flight (variables are different)
  passenger.name = 'Mr. ' + passenger.name;
  // only reference to the object in the heap is copied (both references point to the same object in the heap)
  // similar to: passenger = jonas

  if (passenger.passport === 34243423) {
    alert('Check in');
  } else {
    alert('Wrong passport');
  }
};

checkIn(flight, jonas);
console.log(flight); // primitive. Not changed
console.log(jonas); // name changed because object

const newPassport = function (person) {
  person.passport = Math.trunc(Math.random() * 1000000000);
};

newPassport(jonas);
checkIn(flight, jonas); // passport number is chnaged
```

First-Class and Higher-Order Functions

1. First-class functions
    1. Enables us to write higher-order functions

First-Class vs. Higher-Order Functions

1. First-Class Functions
    1. JavaScript treats functions as **first-class citizens**
    2. This means that functions are **simply values**
    3. Functions are just another **type of object**
        1. Enables use to:
            1. Store them in variables
            2. Store them in object properties

3. Pass functions as arguments to other functions
    1. Event-listener to DOM objects
4. **Return functions FROM functions**
5. Call methods on Functions (since functions are a type of objects like an array or an object):

```
counter.inc.bind(someOtherObjects);
```

2. Higher-Order Functions
    1. A function that **receives** another function as an argument, or that **returns** a new function, or **both**
        1. Example:

```
const greet = () => console.log('Hey Jonas');
btnClose.addEventListener('click', greet);
```

            1. `addEventListener` - higher-order function
            2. `greet` - callback function
                1. It will be called later by the higher-order function
                    1. Example: It is called when click event happens (call back when you can)
        2. Function that returns new function

```
function count() { // higher-order function
    let counter = 0;
    return function () { // returned function
        counter++;
    };
}
```

            1. Harder to understand
    2. This is only possible because of first-class functions
        1. It just means that all functions are values (a concept)
            1. Higher-order functions are possible because of first-class functions

Functions Accepting Callback Functions

1. Why are callback functions used so much?
    1. Makes it easy to split up code into more re-usable and interconnected parts
    2. Allows us to define abstractions
        1. Abstraction: We hide the detail of a code implementation because we do not care about that detail
            1. Thinking about problems at a higher more abstract level
            2. Example:
                1. `transform` doesn't care how tansformation is done (higher-level abstraction)
                    1. Delegates transformation to lower-level functions
    3. Higher order functions do not know exactly what to do, so we pass callback functions to make them do exactly what we want them to do.
2. Code:

```
const oneWord = function (str) {
  return str.replace(/ /g, '').toLowerCase();
};

const upperFirstWord = function (str) {
  const [first, ...others] = str.split(' ');
  return [first.toUpperCase(), ...others].join(' ');
};

// Higher-order function
const transformer = function (str, fn) {
  console.log(`Original string: ${str}`);
  console.log(`Transformed string: ${fn(str)}`);

  console.log(`Transformed by: ${fn.name}`); // **(M)** functions have properties
};

transformer('JavaScript is the best!', upperFirstWord);

transformer('JavaScript is the best!', oneWord);

const high5 = function () {
  console.log('⚡');
};
document.body.addEventListener('click', high5);

['Jonas', 'Martha', 'Adam'].forEach(high5); // higher-order function that takes a callback function
```

Functions Returning Functions

1. Code:

```
const greet = function (greeting) {
  return function (name) {
    console.log(`${greeting} ${name}`);
  };
};

const greeterHey = greet('Hey');
greeterHey('Jonas');
greeterHey('Steven'); // Why is `greeting` variable work here? because of closure
```

```
greet('Hello')('Jonas'); // Calling immediately

// Useful for functional programming

// Arrow functions
const greetArrow = greeting => name => console.log(`${greeting} ${name}`); // Closures work with arrow functions as well

const greeterHiArrow = greetArrow('Hi');
greeterHiArrow('Steven');

greetArrow('Hello')('Jonas');
```

The call and apply Methods

1. Telling JS what the `this` should be?
    1. Method 1: Using `call()` **(M)** - a method that belongs to a function

        ```
        book.call(eurowings, 23, 'Sarah Williams'); // `this` is set to `eurowings`
        console.log(eurowings);
        ```

    2. Method 2: Using `apply()` **(M)**
        1. Difference?
            1. Does not receive any argument list after the `this` keyword but an array

                ```
                const flightData = [583, 'George Cooper'];
                book.apply(swiss, flightData); // not used anymore
                console.log(swiss);
                ```

2. Code:

```
book.call(eurowings, 23, 'Sarah Williams'); // `this` is set to `eurowings`
console.log(eurowings);

book.call(lufthansa, 239, 'Mary Cooper');
console.log(lufthansa);

const swiss = {
  airline: 'Swiss Air Lines',
  iataCode: 'LX',
  bookings: [],
};

book.call(swiss, 583, 'Mary Cooper');
console.log(swiss);

// Apply method
const flightData = [583, 'George Cooper'];
book.apply(swiss, flightData); // not used anymore
console.log(swiss);

book.call(swiss, ...flightData); // this is used instead
console.log(swiss);
```

The bind Method

1. Method 3: using `bind()` **(M)**
    1. Used to manually set the `this` keyword
        1. It does not call the function immediately, but returns another function with `this` keyword bound (set to value passed into `bind()`)
    2. Example:

        ```
        const bookEW = book.bind(eurowings);
        const bookLW = book.bind(lufthansa); // define once, and use anywhere
        bookEW(23, 'Steven Williams'); // `this` keyword set already
        bookLW(34, 'John Miller');

        // Multiple arguments - to set other params

        const bookEW23 = book.bind(eurowings, 23);
        bookEW23('Jonas Schmedtmann'); // Only needs the `name` argument
        bookEW23('Martha Cooper');

        // With Event listeners
        lufthansa.planes = 300;
        lufthansa.buyPlane = function () {
          console.log(this);
          this.planes++;
          console.log(this.planes);
        };

        document.querySelector('.buy').addEventListener('click', lufthansa.buyPlane); // `this` points to element the handler is attacl
        // lufthansa.buyPlane();

        document
          .querySelector('.buy')
          .addEventListener('click', lufthansa.buyPlane.bind(lufthansa)); // `this` inside the `buyPlane` functions points to `lufthans

        // Partial application
        const addTax = (rate, value) => value + value * rate;
        console.log(addTax(0.1, 200));

        const addVAT = addTax.bind(null, 0.23); // `this` is set to `null`
        console.log(addVAT(200));
        console.log(addVAT(23));
        ```

```
// Creation of specific function from a more generic function

// Returning the `addVAT` function
const getAddVAT = function (rate) {
  return function (value) {
    return addTax(rate, value);
  };
};

const addVAT2 = getAddVAT(0.23);
console.log(addVAT2(100));
console.log(addVAT2(23));
```

Coding Challenge #1

1. Code:

```
const poll = {

}
```

Immediately Invoked Function Expression (IIFE)

1. Sometimes, we want a function that we need to be executed only once and not again
    1. Function that disappears right after it is called.
        1. Needed for `async-await`, say
2. Example:

```
const runOnce = function () {
  console.log('This will never run again');
};
runOnce(); // we can invoke the function anywhere again

// IIFE
(function () {
  // expression
  console.log('This will never run again');
})(); // transformed into expression and called immediately

(() => console.log('This will never run again'))();

{
  const isPrivate = 23; //
  var notPrivate = 46; // accessible outside
} // outside cannot access the `const` variable. Doesn't require IIFE because `const` and `let` variables are used

console.log(notPrivate);

// IIFE is now used only for one time execution
```

3. Why was the pattern invented?
    1. A scope has no access to inner scopes (scope chain works the other way around)
        1. All data inside a scope is **private**
            1. The data is **encapsulated**
                1. Protects data from being modified by external scripts, or other parts of the program
                    1. Hence, the IIFE pattern was invented

Closures

1. Closures bring the previous concepts together
2. A closure is not a feature we explicitly use
    1. We don't define closure like we define a new array or new function
        1. It happens automatically in certain situations
3. Code:

```
const secureBooking = function () {
  const passengerCount = 0;

  return function () {
    passengerCount++;
    console.log(`${passengerCount} passengers`);
  };
};

const booker = secureBooking(); // inner function
booker(); // 1 passengers
booker(); // 2 passengers
booker(); // 3 passengers

console.dir(booker); // **(M)** shows `scopes` property (VE of the function)
// Shows `closure (secureBooking)` & `passengerCount`
// [[...]] - internal property that we cannot access from code
```

    1. Global Scope:
        1. Gloal EC:

```
secureBooking = <f>
booker = <f>
```

2. `secureBooking()` EC:

```
passengerCount = 0 (VE - Scope of the function)
```

2. Scope Chain:

1. `secureBooking()` scope: (gets access to all variables in parents scope)

```
passengerCount = 0
------------------
secureBooking = <f>
booker = <f>
```

3. When we execute the `secureBooking()` function, it's scope is pushed and then popped of the stack

1. How can we still update the variable using inner function?
1. Solution: Closure
1. Allows a function to remember all variables at its birthplace

4. When `booker()` function is called, a new `booker()` EC is created and put on the call-stack (VE is empty in this function)

1. Scope chain:

1. `booker()` scope - a child scope of global scope

```
booker() scope
<empty>
------------------
secureBooking = <f>
booker = <f>
```

1. Scope chain doesn't contain `passengerCount` variable
1. Solution: Closure - any function has access to the VE of the EC in which the function was created (even after the EC is gone)
1. **The connection between the VE of the EC in which a function was created and the function is called closure**
2. Definition: **VE attached to the function, exactly as it was at the time and place the function was created**
1. The scope-chain is preserved through the closure even when a scope is already been destroyed (VE continuous living in the engine)
2. `booker()` function closed over its parent scope (or parent VE)
3. A closed over VE stays with a function for ever

2. Updated lookup:

1. If a variable is not in the current scope, JS looks immediately in its closure to find the variable
1. Closure has priority over the scope-chain

4. Closures Summary:

1. A closure is the closed-over **variable environment** of the execution context **in which a function was created**, even **after** that execution context is gone
2. A closure gives a function access to all the variables **of its parent function**, even **after** that parent function has returned. The function keeps a **reference** to its outer scope, which **preserves** the scope chain throughout time
3. A closure makes sure that a function doesn't loose connection to **variables that existed at the function's birth place**
4. A closure is like a **backpack** that a function carries around wherever it goes. This backpack has all the **variables that were present in the environment where the function was created**
5. We do not have to manually construct closures.
1. It is a JS feature that happens automatically
2. We can't access closed-over variables explicitly
3. A closure is **NOT** a tangible JS object

More Closure Examples

1. Example:

```
let f;

const g = function () {
  const a = 23;
  f = function () {
    // f is closed over variables in the EC in which it is defined (even if `f` is not created in the VE)
    console.log(a * 2);
  };
};

const h = function () {
  const b = 777;
  f = function () {
    // re-birth
    // assigned a second function
    console.log(b * 2);
  };
};

g();
f();
console.dir(f); // it has value of `a`

// re-assigned function
h();
f();

console.dir(f); // it has value of `b` but not the value of `a` (loses the previous closure)

// Example 2
```

```
const boardPassengers = function (n, wait) {
  // closure includes arguments (because they are local variables to the function)
  const perGroup = n / 3;

  setTimeout(function () {
    console.log(`We are now boarding all ${n} passengers`);
    console.log(`There are 3 groups, each with ${perGroup} passengers`);
  }, wait * 1000); // function is executed after the time specified (in ms)

  console.log(`Will start boarding in ${wait} seconds`);
};

const perGroup = 1000; // not used, because closure has priority over scope-chain
boardPassengers(180, 3); // callback function was executed independently but has access to all variables in the VE in which it was c
```

Coding Challenge #2

1. Code:

```
(function () {
  const header = document.querySelector('h1');
  header.style.color = 'red';

  document.querySelector('body').addEventListener('click', function () {
    // function is created in the parent EC and has access to the VE of it's parent
    header.style.color = 'blue';
  });
})();
```

Section 11: Working with Arrays

Section Intro

1. Arrays are most used data structure
    1. It has array methods
2. Topics:
    1. How to use methods
    2. When to use methods

Section Roadmap

Simple Array Methods

1. 11-Array-Bankist
2. Why do arrays have methods?
    1. They are functions we call on objects (attached to objects)
        1. Since arrays are objects
            1. The functions are attached to all arrays (prototypal inheritance)
3. Methods:
    1. `slice` - used to extract part of an array
        1. `slice()` - shallow copy
            1. Usually used when we want to chain methods
    2. `splice` - similar to `slice` but modifies the original array
        1. mdn array splice - more examples
            1. Second param: `deleteCount` (not the end param)
4. Code:

```
let arr = ['a', 'b', 'c', 'd', 'e'];

console.log(arr.slice(2)); // **(M)** `c` to the end

console.log(arr.slice(2, 4));

console.log(arr.slice(-2)); // last two elements
console.log(arr.slice(-1)); // last element

console.log(arr.slice(1, -2)); // except the last two

console.log(arr.slice()); // a shallow copy is created.

// Splice - similar to slice. It mutates the original array

console.log(arr.splice(2));
console.log(arr); // changed. Splice deleted the other elements
arr.splice(-1); // last element

arr.splice(1, 2); // delete 2 elements from position 1 - [a, d] (b, c are deleted)

// Reverse
arr = ['a', 'b', 'c', 'd', 'e'];
const arr2 = ['j', 'i', 'h', 'g', 'f'];

console.log(arr2.reverse()); // mutates the original array
console.log(arr2);

// Concat
const letters = arr.concat(arr2); // **(M)**. Doesn't mutate
console.log(letters);
console.log([...arr, ...arr2]); // does't mutate
```

```
// Join
console.log(letters.join(' - ')); // **(M)**
```

The new at Method

1. When to use `at` method?
    1. To count from the end of an array
    2. To do method chaining (to combine multiple methods)
2. Code:

```
const arr1 = [23, 11, 64];
console.log(arr1[0]);
console.log(arr1.at(0)); // using a method. **(M)**

// Uses - Getting the last element
console.log(arr1[arr.length - 1]);
console.log(arr1.slice(-1)[0]);
console.log(arr1.at(-1)); // negative index - starts counting from the right-side

// `at` on strings
console.log('jonas'.at(0)); // **(M)**
console.log('jonas'.at(-1));
```

Looping Arrays: forEach

1. `forEach` - it is different
2. When to use `forEach`?
    1. We cannot break out of a `forEach` loop (`break` and `continue` will not work)
        1. Solution: for-of loop
3. Example:

```
const movements = [200, 450, -400, 3000, -650, -130, 70, 1300];

for (const movement of movements) {
  if (movement > 0) {
    console.log(`You deposited ${movement}`);
  } else {
    console.log(`You withdrew ${Math.abs(movement)}`);
  }
}

console.log('----FOREACH----');
movements.forEach(function (movement) {
  if (movement > 0) {
    console.log(`You deposited ${movement}`);
  } else {
    console.log(`You withdrew ${Math.abs(movement)}`);
  }
}); // higher-order function which requires a callback function. In each iteration, it executes the callback function and passes the
// 0: function (200)
// 1: function (450)
// 2: function (400)
// ...

for (const [i, movement] of movements.entries()) {
  if (movement > 0) {
    console.log(`Movement ${i + 1}: You deposited ${movement}`);
  } else {
    console.log(`Movement ${i + 1}: You withdrew ${Math.abs(movement)}`);
  }
}

console.log('-----FOREACH-----');
movements.forEach(function (mov, i, arr) {
  if (mov > 0) {
    console.log(`Movement ${i + 1}: You deposited ${mov}`);
  } else {
    console.log(`Movement ${i + 1}: You withdrew ${Math.abs(mov)}`);
  }
});
```

forEach with Maps and Sets

1. Code:

```
// Map
const currencies = new Map([
  ['USD', 'United States dollar'],
  ['EUR', 'Euro'],
  ['GBP', 'Pound sterling'],
]);

currencies.forEach(function (value, key, map) {
  console.log(`${key}: ${value}`);
});

// Set
const currenciesUnique = new Set(['USD', 'GBP', 'USD', 'EUR', 'EUR']);

console.log(currenciesUnique);

currenciesUnique.forEach(function (value, key, set) {
  console.log(`${key}: ${value}`);
```

```
}); // key is same as the value
// A set doesn't have keys and it doesn't have indexes. Hence, key is set to the value
```

PROJECT: "Bankist" App

1. Application: https://bankist.netlify.app (simple bank application)
    1. user: js, pin: 1111
    2. Request loan: 1000
        1. Appears as deposit
    3. Transfer money: 5000
    4. user: js, pin: 2222
        1. Has deposit
2. Bankist flowchart (Complete after next section)

Constructing DOM Elements

1. HTML
    1. `nav`
    2. `main` - entire app
2. Code:

```
// Map
const currencies = new Map([
  ['USD', 'United States dollar'],
  ['EUR', 'Euro'],
  ['GBP', 'Pound sterling'],
]);

currencies.forEach(function (value, key, map) {
  console.log(`${key}: ${value}`);
});

// Set
const currenciesUnique = new Set(['USD', 'GBP', 'USD', 'EUR', 'EUR']);

console.log(currenciesUnique);

currenciesUnique.forEach(function (value, key, set) {
  console.log(`${key}: ${value}`);
}); // key is same as the value
// A set doesn't have keys and it doesn't have indexes. Hence, key is set to the value

// It is a good practice to pass data into function instead of using global variables
const displayMovements = function (movements) {
  containerMovements.innerHTML = ''; // Make container empty
  // textContent - only text

  movements.forEach(function (mov, i) {
    const type = mov > 0 ? 'deposit' : 'withdrawal';

    const html = `
    <div class="movements__row">
      <div class="movements__type movements__type--${type}">${
      i + 1
    } ${type}</div>
      <div class="movements__value">${mov}</div>
    </div>
    `;

    containerMovements.insertAdjacentHTML('afterbegin', html); // **(M)**. Right after the beginning tag of the element, placed on t
  });
};
displayMovements(account1.movements);

// console.log(containerMovements.innerHTML);
```

Coding Challenge #1

1. Code:

```
const juliasData1 = [3, 5, 2, 12, 7];
const katesData1 = [4, 1, 15, 8, 3];
const juliasData2 = [9, 16, 6, 8, 3];
const katesData2 = [10, 5, 6, 1, 4];

const juliasDogs1 = juliasData1.slice(1, -2);
const juliasDogs2 = juliasData2.slice(1, -2);

console.log(juliasData1);
console.log(juliasDogs1);
console.log(juliasData2);
console.log(juliasDogs2);

const dogs1 = [...juliasDogs1, ...katesData1];
const dogs2 = juliasDogs2.concat(katesData2);

const checkDogs = function (dogs) {
  dogs.forEach(function (dog, i) {
    console.log(`Dog ${i} is ${dog < 3 ? 'a puppy' : 'an adult'}`);
  });
};

console.log('----- DATA 1 -----');
checkDogs(dogs1);
console.log('----- DATA 2 -----');
checkDogs(dogs2);
```

Data Transformations: map, filter, reduce

1. Big array methods to perform data-transformations (new arrays are created)
2. Methods:
    1. Map
        1. Used to loop over arrays
        2. Makes a new array based on the original array
        3. It takes an array, loops over the array, and in each iteration, it applies a callback function (we specify in our code) to the current array element
            1. Example: current * 2 - multiplies each element by 2 and places the result in the new array
    2. Filter
        1. Filters the elements in the original array which satisfies a condition
            1. Example: `current > 2` - only the elements that satisfy the condition will be added to a new filtered array
        2. `filter` returns a new array containing the array elements that passed a specified test condition
            1. Other elements are filtered out
    3. Reduce
        1. `reduce` boils ("reduces") all array elements down to one single value (e.g. adding all elements together)
        2. We need an accumulator variable
            1. Example: `acc + current`
                1. In each iteration, reduce method keeps adding the current element to the `acc` (to get the total sum)
        3. Final value is returned (only the value and not an array)

The map Method

1. Code:

```
const movements = [200, 450, -400, 3000, -650, -130, 70, 1300];

// EUR to USD
const eurToUsd = 1.1;

// Functional programming
const movementsUSD = movements.map(function (mov) {
  return mov * eurToUsd;
});

console.log(movements);
console.log(movementsUSD);

const movementsUSDFor = [];
for (const mov of movements) {
  movementsUSDFor.push(mov * eurToUsd);
}
console.log(movementsUSDFor);

const movementsUSDArrow = movements.map(mov => mov * eurToUsd);
console.log(movementsUSDArrow);

const movementDescriptions = movements.map(
  (mov, i, arr) =>
    `Movement ${i + 1}: You ${mov > 0 ? 'deposited' : 'withdrew'} ${Math.abs(
      mov
    )}`
); // No side-effects (functional programming)

console.log(movementDescriptions);
```

Computing Usernames

1. Code:

```
const constructUsername = accs =>
  accs.forEach(acc => {
    // `forEach` is used for side-effects
    acc.username = acc.owner
      .toLowerCase()
      .split(' ')
      .map(name => name.at(0))
      .join('');
  });

const user = 'Steven Thomas Williams'; // stw - username

constructUsername(accounts);
console.log(accounts);
```

The filter Method

1. Filter elements based on a certain condition
2. Code:

```
const deposits = movements.filter(mov => mov > 0); // boolean
console.log(movements);
console.log(deposits);

const withdrawals = movements.filter(mov => mov < 0);
console.log(withdrawals);
```

The reduce Method

1. Used to boil all elements in an array to single result
2. Code:

```
const calcDisplayBalance = function (movements) {
  const balance = movements.reduce((acc, mov) => acc + mov, 0);
  labelBalance.textContent = `${balance} EUR`;
};

calcDisplayBalance(account1.movements);

// Maximum value
const max = movements.reduce((acc, mov) => Math.max(acc, mov), movements.at(0));
console.log(max);
const min = movements.reduce((acc, mov) => Math.min(acc, mov), movements.at(0));
console.log(min);
```

Coding Challenge #2

1.

Chaining Methods

1. Code:

```
const calcDisplayBalance = function (movements) {
  const balance = movements.reduce((acc, mov) => acc + mov, 0);
  labelBalance.textContent = `${balance} EUR`;
};

calcDisplayBalance(account1.movements);

// Maximum value
const max = movements.reduce((acc, mov) => Math.max(acc, mov), movements.at(0));
console.log(max);
const min = movements.reduce((acc, mov) => Math.min(acc, mov), movements.at(0));
console.log(min);

const dogAges = [5, 2, 4, 1, 15, 8, 3];

const humanAges = dogAges.map(dogAge =>
  dogAge <= 2 ? 2 * dogAge : 16 + dogAge * 4
);
console.log(humanAges);

// Pipeline
const totalDepositsUSD = movements
  .filter(mov => mov > 0)
  .map(mov => mov * eurToUsd)
  .reduce((acc, mov) => acc + mov, 0); // chaining is allowed only if an array is returned
console.log(totalDepositsUSD);

// It is hard to debug if some intermediate step in the pipeline goes wrong
// Solution: Get the `array` element and check inside the next method

const totalDepositsUSD1 = movements
  .filter(mov => mov > 0)
  .map((mov, i, arr) => {
    console.log(arr); // Good for debugging
    return mov * eurToUsd;
  })
  .reduce((acc, mov) => acc + mov, 0); // chaining is allowed only if an array is returned
console.log(totalDepositsUSD1);

const calcDisplaySummary = movements => {
  const incomes = movements
    .filter(mov => mov > 0)
    .reduce((acc, mov) => acc + mov, 0);
  labelSumIn.textContent = `${incomes}💵`;

  const outcomes = movements
    .filter(mov => mov < 0)
    .reduce((acc, mov) => acc + mov, 0);
  labelSumOut.textContent = `${Math.abs(outcomes)}💵`;
};
calcDisplaySummary(account1.movements);
```

1. Huge chains can reduce performance
    1. Optimize by reducing the length of the chain
2. It is a bad practice to chain methods that mutate the original array

Coding Challenge #3

1.

The find Method

1. `find` - used to retrieve one element of an array based on a condition
    1. It returns only the first matching element
    2. It returns only an element and not an array
2. Code:

```
const firstWithdrawal = movements.find(mov => mov < 0);
console.log(movements);
console.log(firstWithdrawal);

const account = accounts.find(acc => acc.owner === 'Jessica Davis'); // Great use-case
console.log(account);

let account5 = undefined;
```

```
for (let acc of accounts) {
  if (acc.owner === 'Jessica Davis') {
    account5 = acc;
    break;
  }
}
console.log(account5);
```

Implementing Login

1. Code:

```
// Event handler
let currentAccount;

btnLogin.addEventListener('click', function (e) {
  e.preventDefault(); // **(M)**
  console.log('LOGIN');

  currentAccount = accounts.find(
    acc => acc.username === inputLoginUsername.value
  );
  console.log(currentAccount);
  if (currentAccount?.pin === Number(inputLoginPin.value)) {
    console.log('LOGIN');
    // Display UI and message
    labelWelcome.textContent = `Welcome back, ${currentAccount.owner
      .split(' ')
      .at(0)}`;
    containerApp.style.opacity = 100;

    // Clear input fields
    inputLoginUsername = inputLoginPin.value = '';
    inputLoginPin.blur(); // field loses focus

    // Display movements
    displayMovements(currentAccount.movements);

    // Display balance
    calcDisplayBalance(currentAccount.movements);

    // Display summary
    calcDisplaySummary(currentAccount);
  }
});
```

Implementing Transfers

1. Code:

```
btnTransfer.addEventListener('click', function (e) {
  e.preventDefault(); // **(M)**
  const amount = Number(inputTransferAmount.value);
  const receiverAcc = accounts.find(
    acc => acc.username === inputTransferTo.value
  );
  inputTransferAmount.value = inputTransferTo.value = '';
  console.log(amount, receiverAcc);

  if (
    amount > 0 &&
    receiverAcc &&
    currentAccount.balance >= amount &&
    receiverAcc?.username !== currentAccount.username
  ) {
    // console.log('Transfer valid');
    currentAccount.movements.push(-amount);
    receiverAcc.movements.push(amount);

    updateUI(currentAccount);
  }
});

const updateUI = function (acc) {
  // Display movements
  displayMovements(acc.movements);

  // Display balance
  calcDisplayBalance(acc);

  // Display summary
  calcDisplaySummary(acc);
};
```

The findIndex Method

1. `findIndex()` - **(M)** returns index of found element
    1. `splice` needs index to delete an element
2. Code:

```
btnClose.addEventListener('click', function (e) {
  e.preventDefault(); // **(M)**
  console.log('Delete');

  if (
    inputCloseUsername?.value === currentAccount.username &&
    Number(inputClosePin?.value) === currentAccount.pin
  ) {
```

```
    console.log('Matched');
    const index = accounts.findIndex(
      acc => acc.username === currentAccount.username
    ); // supports complex condition (not just `===`). Gets access to current index, and current array

    // Delete account
    accounts.splice(index, 1); // remove 1 element at `index` location

    // Hide UI
    containerApp.style.opacity = 0;

    inputCloseUsername.vaue = inputClosePin.value = '';
  }
});
```

some and every

1. We can use any condition
    1. `some` - returns `true` if at-least one element in an array satisfies the given condition
    2. `every` - returns `true` if every element in an array satsifies the given condition
2. Code:

```
btnClose.addEventListener('click', function (e) {
  e.preventDefault(); // **(M)**
  console.log('Delete');

  if (
    inputCloseUsername?.value === currentAccount.username &&
    Number(inputClosePin?.value) === currentAccount.pin
  ) {
    console.log('Matched');
    const index = accounts.findIndex(
      acc => acc.username === currentAccount.username
    ); // supports complex condition (not just `===`). Gets access to current index, and current array

    // Delete account
    accounts.splice(index, 1); // remove 1 element at `index` location

    // Hide UI
    containerApp.style.opacity = 0;

    inputCloseUsername.vaue = inputClosePin.value = '';
  }
});

console.log(movements);
console.log(movements.includes(-130)); // only tests for equality

const anyDeposits = movements.some(mov => mov > 0); // **(M)**
console.log(anyDeposits);
const anyDepositAbove5k = movements.some(mov => mov > 5000);
console.log(anyDepositAbove5k);

btnLoan.addEventListener('click', function (e) {
  e.preventDefault();

  const amount = Number(inputLoanAmount.value);

  if (amount > 0 && currentAccount.movements.some(mov => mov >= amount * 0.1)) {
    // Add movement
    currentAccount.movements.push(amount);

    // Update UI
    updateUI(currentAccount);
  }

  inputLoanAmount.value = '';
});

// Every
console.log(movements.every(mov => mov > 0)); // all movements are deposits? `false`
console.log(account4.movements.every(mov => mov > 0)); // all movements are deposits? `true`

// We can write condition function separately and pass it as a callback
const deposit = mov => mov > 0;
console.log(movements.some(deposit));
console.log(movements.every(deposit));
console.log(movements.filter(deposit));
```

flat and flatMap

1. For nested arrays
2. `flatMap` - compines `map(...).flat(...)` into one method
    1. Why? For performance (goes only one level deep)
3. Code:

```
const arr = [[1, 2, 3], [4, 5, 6], 7, 8];
console.log(arr.flat()); // **(M)** (ES2019), no callback

const arrDeep = [[[1, 2], 3], [4, 5, 6], 7, 8];
console.log(arrDeep.flat()); // contains inner arrays - it goes only one level deep (doesn't go deep)
console.log(arrDeep.flat(2)); // `depth` = 2 levels deep

const accountMovements = accounts.map(acc => acc.movements);
console.log(accountMovements);
```

```
const allMovements = accountMovements.flat();
console.log(allMovements);
const overallBalance = allMovements.reduce((acc, mov) => acc + mov, 0);
console.log(overallBalance);

const overallBalance2 = accounts
  .map(acc => acc.movements)
  .flat()
  .reduce((acc, mov) => acc + mov, 0);

// Map & flat is common operation - solution: `flatMap`
console.log(overallBalance2);

const overallBalance3 = accounts
  .flatMap(acc => acc.movements)
  .reduce((acc, mov) => acc + mov, 0);

console.log(overallBalance3);
```

Sorting Arrays

1. Code:

```
// Strings
const owners = ['Jonas', 'Zach', 'Adam', 'Martha'];
console.log(owners.sort()); // **(M)** - mutates the original array

// Numbers
console.log(movements);
console.log(movements.sort()); // it does sorting based on strings (not numbers). It converts everything to strings and then sorts

// return < 0 => A, B
// return > 0 => B, A
movements.sort((a, b) => {
  if (a > b) return 1;
  else return -1;
}); // `a`, `b` are numbers in the condition
console.log(movements);

// Descending order
movements.sort((a, b) => b - a);
console.log(movements);

// Not recommended for soriting mixed elements
```

2. Code:

```
const displayMovements = function (movements, sort = false) {
  containerMovements.innerHTML = ''; // Make container empty
  // textContent - only text

  const movs = sort ? movements.slice().sort((a, b) => a - b) : movements;

  movs.forEach(function (mov, i) {
    const type = mov > 0 ? 'deposit' : 'withdrawal';

    const html = `
    <div class="movements__row">
      <div class="movements__type movements__type--${type}">${
      i + 1
    } ${type}</div>
      <div class="movements__value">${mov}</div>
    </div>
    `;

    containerMovements.insertAdjacentHTML('afterbegin', html); // **(M)**. Right after the beginning tag of the element, placed on t
  });
};
displayMovements(account1.movements);

let sorted = false;
btnSort.addEventListener('click', function (e) {
  e.preventDefault();

  sorted = !sorted;
  displayMovements(currentAccount.movements, sorted);
});
```

More Ways of Constructing and Filling Arrays

1. Programmatically generating arrays
    1. Example:
2. `Array.from` - invented to instantiate arrays from array-like structures (iterables - strings, sets, maps, ...) (Array from other things)
3. Code:

```
const x = new Array(7);
console.log(x); // empty array of size 7
// We cannot use the array for anything - `map` cannot be called
// console.log(x.map(() => 5)); // doesn't do anything

x.fill(1); // **(M)** mutates the underlying array
x.fill(1, 3); // starts at index 3
x.fill(1, 3, 5); // starts at index 3 and ends at 4 (exclusive) (similar to slice)

arr.fill(23, 2, 6);

// Array.from
const y = Array.from({ length: 7 }, () => 1); // **(M)**
```

```
console.log(y);

const z = Array.from({ length: 7 }, (_, i) => i + 1);
console.log(z); // `_` - throw away parameter. It will not be used

const diceRolls = Array.from({ length: 100 }, () =>
  Math.round(Math.random() * 100)
);
console.log(diceRolls);

labelBalance.addEventListener('click', function () {
  const movementsUI = Array.from(
    document.querySelectorAll('.movements__value'),
    el => Number(el.textContent.replace('€', ''))
  );
  console.log(movementsUI);

  // const movementsUI2 = [...document.querySelectorAll('.movements__value')];
});
```

Summary: Which Array Method to Use?

1. Which array method to use?
    1. Q: What do I want from this method?
        1. To mutate original array?
            1. Add to original:
                1. `.push`
                2. `.unshift`
            2. Remove form original:
                1. `.pop`
                2. `.shift`
                3. `.splice`
            3. Others
                1. `.reverse`
                2. `.sort`
                3. `.fill`
        2. A new array?
            1. Compute from original
                1. `.map`
            2. Filtered using condition
                1. `.filter`
            3. Portion of original
                1. `.slice`
            4. Adding original to other
                1. `.concat`
            5. Flattening the original
                1. `.flat`
                2. `.flatMap`
        3. An array index?
            1. Based on value:
                1. `.indexOf`
            2. Based on test condition:
                1. `.findIndex`
        4. An array element?
            1. Based on test condition
                1. `.find`
        5. Know if array includes an element?
            1. Based on value:
                1. `.includes`
            2. Based on test condition:
                1. `.some`
                2. `.every`
        6. A new string
            1. Based on separator string:
                1. `.join`
        7. To transform to value
            1. Based on accumulator:
                1. `.reduce`
                    1. Boil down array to single value of any type: number, string, boolean, or even new array or object
                        1. Some of the other methods can be replaced with `.reduce`
        8. To just loop over an array
            1. Based on callback:
                1. `.forEach`
                    1. Does not construct a new array, just loops over it

Array Methods Practice

1. How much has been deposited in-total in the bank (across all accounts)

```
const bankDepositSum = accounts
  .flatMap(acc => acc.movements)
  .filter(mov => mov > 0)
  .reduce((acc, mov) => acc + mov, 0);
console.log(bankDepositSum);
```

2. Count the number of deposits with at-least 1000.

```
const bankDeposit1000 = accounts
    .flatMap(acc => acc.movements)
    .filter(mov => mov >= 1000)
```

```
    .reduce((acc, mov) => acc + 1, 0);
console.log(bankDeposit1000);
```

3. New object with the sum of deposits and withdrawals

```
const { deposits, withdrawals } = accounts
  .flatMap(acc => acc.movements)
  .reduce(
    (sums, mov) => {
      // mov > 0 ? (acc.deposits += mov) : (acc.withdrawals += mov);
      sums[mov > 0 ? 'deposits' : 'withdrawals'] += mov;
      return sums;
    },
    {
      deposits: 0,
      withdrawals: 0,
    }
  );
console.log(deposits, withdrawals);
```

4. Convert any string to titlecase (All words in a sentence are capitalized except for some of them (exceptions))

    1. `this is a nice title -> This Is a Nice Title`

    2. Code:

```
const convertTitleCase = function (title) {
  const capitalize = str => str[0].toUpperCase() + str.slice(1);

  const exceptions = ['a', 'an', 'the', 'and', 'but', 'or', 'on', 'in', 'with'];

  const titleCase = title
    .toLowerCase()
    .split(' ')
    .map(word =>
      exceptions.includes(word)
        ? word
        : word.at(0).toUpperCase() + word.slice(1)
    )
    .join(' ');
  return capitalize(titleCase);
};
console.log(convertTitleCase('this is a nice title'));
console.log(convertTitleCase('this is a LONG title'));
console.log(convertTitleCase('and here is another title with an EXAMPLE'));
```

Coding Challenge #4

Section 12: Numbers, Dates, Intl and Timers

Section Intro

1. Less important but other representations of data

Section Roadmap

Converting and Checking Numbers

1. 12-Numbers-Dates-Timers-Bankist

2. Code:

```
console.log(23 === 23.0); // `true` - All numbers are represented in 64-base-2 (always in binary format)
// Binary base 2 - 0 1
console.log(0.1 + 0.2); // 0.30000000000000004
// Hence JS is not good for precise scientific calculations

console.log(+'23'); // converts all operands to numbers

// Parsing
console.log(Number.parseInt('30px')); // symbols are ignored (input must start with a number)
console.log(Number.parseInt('1011', 2));

console.log(Number.parseFloat('2.5rem')); // even whitespaces around do not matter
console.log(parseFloat('2.5rem')); // global functions is old-school
// `Number` is called namespace

// Check if value is NaN
console.log(Number.isNaN(20));
console.log(Number.isNaN('20')); // `false`
console.log(Number.isNaN(+'20X'));
console.log(Number.isNaN(20 / 0)); // `false` - returns `Infinity` (special value)

// Checking if value is a number
console.log(Number.isFinite(20)); // **(M)**
console.log(Number.isFinite('20')); // `false` (since it is NaN)
console.log(Number.isFinite(+'20X'));
console.log(Number.isFinite(20 / 0)); // `false` - good for floating point numbers

console.log(Number.isInteger(23)); // **(M)**
console.log(Number.isInteger(23.0));
```

Math and Rounding

1. `Math` - namespace

2. Code:

```
console.log(Math.sqrt(25));
console.log(25 ** (1 / 2)); // sqrt
console.log(8 ** (1 / 3)); // 2

console.log(Math.max(5, 18, 23, 11, 2)); // does type-coersion but does not parseInt ('20px' does not work)
console.log(Math.min(5, 18, 23, 11, 2));

console.log(Math.PI * Number.parseFloat('10px') ** 2);

console.log(Math.trunc(Math.random() * 6) + 1);

const randomInt = (min, max) => Math.trunc(Math.random * (max - min) + 1) + min;

console.log(randomInt(10, 20));

// Rounding integers
console.log(Math.trunc(23.3));

console.log(Math.round(23.9)); // 24

console.log(Math.ceil(23.3));
console.log(Math.ceil(23.9));

console.log(Math.floor(23.3));
console.log(Math.floor(23.9)); // type coersion is applied

// `floor` and `trunc` do work the same for positive numbers
console.log(Math.trunc(-23.3));
console.log(Math.floor(-23.3)); // rounds the other way (-24)

// Rounding decimals
console.log((2.7).toFixed(0)); // **(M)** '3' (JS does boxing and unboxing internally)
console.log((2.7).toFixed(3)); // '2.700' - 3 decimals
console.log((2.345).toFixed(2)); // '2.35'
console.log(+(2.345).toFixed(2)); // 2.345
```

The Remainder Operator

1. Remainder operator returns remainder of a division

```
console.log(5 % 2); // 1
console.log(8 % 3); // 2
console.log(6 % 2); // 0 - even
console.log(7 % 2); // 1 - odd

labelBalance.addEventListener('click', function () {
  [...document.querySelectorAll('.movements__row')].forEach(function (row, i) {
    if (i % 2 === 0) row.style.backgroundColor = 'orangered';
    if (i % 3 === 0) row.style.backgroundColor = 'blue';
  });
});
```

Numeric Separators

1. For formatting numbers for readability and understandability
    1. Especially for large numbers
2. Code:

```
// Numeric separators

const diameter = 287_460_000_000; // 287,460,000,000
console.log(diameter); // The engine ignores `_`s
// We can place the `_` anywhere

const priceCents = 345_99;
console.log(priceCents);

const transferFee1 = 15_00;
const transferFee2 = 1_500;

// const PI = 3._1415; // not allowed
// const PI = _3.1415; // not allowed
// const PI = 3.1415_; // not allowed
// const PI = 3.14__15; // not allowed

console.log(Number('23000'));
console.log(Number('230_000')); // NaN - not allowed (only allowed for numbers)
console.log(parseInt('230_000')); // We get only 230
```

Working with BigInt

1. Introduced in ES2020 (usually 64 bits were used before)
    1. Integers: 53 bits to store the number and 11 bits for exponent value

       ```
       console.log(2 ** 53 - 1); // biggest number
       ```
2. BigInt is a primitive used to store a number as large as we want.
3. Operations: All operators work the same
4. Code:

```
console.log(2 ** 53 - 1);
console.log(Number.MAX_SAFE_INTEGER); // **(M)** (any number greater will be set to the max value)
```

```
console.log(
  948509823049832098290850392840932840983209832840932849083593284093284093284093n
);

console.log(
  BigInt(
    948509823049832098290850392840932840983209832840932849083593284093284093284093n
  )
);

console.log(
  309839489589384938592384932849859839483298982398593284932898923849328498324 9n
);

const sum =
  948509823049832098290850392840932840983209832840932849083593284093284093284093n +
  309839489589384938592384932849859839483298982398593284932898923849328498324 9n;
console.log(sum);

const huge = 935934935934934937534379384938593493849 3n;
const num = 23;
// console.log(huge + num); // script.js:379 Uncaught TypeError: Cannot mix BigInt and other types, use explicit conversions

console.log(huge + BigInt(num)); // works

// Exceptions
console.log(20n > 15); // works
console.log(20n === 20); // type-coersion doesn't work (false)

console.log(typeof 20n); // bigint
console.log(20n == '20'); // `true` - type coersion

console.log(huge + ' is REALLLY big!!!'); // converted to string

// Math operations will not work
// console.log(Math.sqrt(16n));

// Division
console.log(10n / 3n); // 3n - since 'bigint` is integer
```

Constructing Dates

1. 4 ways of creation of dates in JS (different params)
2. Code:

```
// 1st way:
const now = new Date();
console.log(now); // current date and time

// 2nd way:
console.log(new Date('Aug 02 2020 12:05:41'));
console.log(new Date('May 22 1985 06:00:00')); // not a good idea - it can be unreliable
// Good if string is created in JS
console.log(new Date(account1.movementsDates[0]));
// Z - UTC - In London, No day-light savings

console.log(new Date(2037, 10, 19, 15, 23, 5)); // Month is 0 based (10 - 11th month)

console.log(new Date(2037, 10, 31)); // Nov 31 - auto-corrected to Dec 1

// 3rd way
console.log(new Date(0)); // Start of unix time
console.log(new Date(3 * 24 * 60 * 60 * 1000)); // passing timestamp

// Date methods can be used to get or set components of a date

// Working with dates
const future = new Date(2037, 10, 19, 15, 23);
console.log(future);
console.log(future.getFullYear()); // don't use `getYear`
console.log(future.getMonth());
console.log(future.getDate()); // Dau of the month
console.log(future.getDay()); // Day of the week
console.log(future.getHours());
console.log(future.getMinutes());
console.log(future.getSeconds());
console.log(future.toISOString()); // **(M)**
console.log(future.getTime()); // **(M)** timestamp
console.log(new Date(future.getTime()));

console.log(Date.now()); // **(M)**

future.setFullYear(2040);
console.log(future);
```

Adding Dates to "Bankist" App

1. Code:

```
currentAccount.movementsDates.push(new Date().toISOString());
receiverAcc.movementsDates.push(new Date().toISOString());
```

Operations With Dates

1. Difference between dates
2. `moment.js` - library for handling complex edge cases

3. Code:

```
const formatMovementDate = function (date) {
  const calcDaysPassed = (date1, date2) =>
    Math.round(Math.abs(date2 - date1) / (1000 * 60 * 60 * 24));

  const daysPassed = calcDaysPassed(new Date(), date);
  console.log(daysPassed);

  if (daysPassed === 0) return 'Today';
  if (daysPassed === 1) return 'Yesterday';
  if (daysPassed <= 7) return `${daysPassed} days ago`;

  const day = `${date.getDate()}`.padStart(2, 0);
  const month = `${date.getMonth() + 1}`.padStart(2, 0);
  const year = date.getFullYear();

  return `${day}/${month}/${year}`;
};

const displayMovements = function (acc, sort = false) {
    // ...
    const displayDate = formatMovementDate(date);
    // ...
}

// ...

const future1 = new Date(2037, 10, 19, 15, 23);
console.log(Number(future)); // timestamp in milliseconds
console.log(+future); // timestamp in milliseconds

const calcDaysPassed = (date1, date2) =>
  Math.abs(date2 - date1) / (1000 * 60 * 60 * 24);

const days1 = calcDaysPassed(new Date(2037, 3, 14), new Date(2037, 3, 4));
console.log(days1);
```

Internationalizing Dates (Intl)

1. Internationalization API - Allows us to format numbers and strings according to different languages

Internationalizing Numbers (Intl)

Timers: setTimeout and setInterval

1. `setTimeout` - **(M)** Runs once after a defined time
    1. We can cancel the timeout before 3s passed
2. `setInterval` **(M)** Runs forever after a certain interval until we stop it
3. Code:

```
const ingredients = ['olives', 'spinach'];
const pizzaTimer = setTimeout(
  (ing1, ing2) => console.log(`Here is your pizza ${ing1} and ${ing2} 🍕`),
  3000,
  ...ingredients
); // time in ms, the callback is registered and the code execution continues
console.log('Waiting...'); // asynchronous JS

if (ingredients.includes['spinach']) clearTimeout(pizzaTimer); // **(M)**

// setInterval
setInterval(function () {
  const now = new Date();
  console.log(now);
}, 1000);

const startLogOutTimer = function () {
  // Set time to 5 minutes
  let time = 100;

  // Call the timer every second
  setInterval(function () {
    // In each call, print the remaining time to UI
    labelTimer.textContent = time;

    // Decrease 1s
    time--;

    // When 0 seconds, stop timer and log out user
  }, 1000);
};
```

Implementing a Countdown Timer

Section 13: Advanced DOM and Events

Section Intro

1. Advanced DOM manipulation techniques
2. After this section we can build any visual component or effect we can imagine

Section Roadmap

PROJECT: "Bankist" Website

1. Marketing website for fictional bank

How the DOM Really Works

Review: What is the DOM?

1. DOM is the interface between JS code and browser (HTML documents rendered by the browser)
2. How does DOM help?
    1. Allows us to make JS interact with the browser
    2. We can write JS to construct, modify, and delete HTML elements; set styles, classes and attributes; and listen and respond to events;
        1. Works because, DOM tree is generated from an HTML document, which we can then interact with
            1. How does the interaction work?
                1. DOM is very complex API that contains lots of methods and properties to interact with the DOM tree
                    1. `.querySelector()`
                    2. `.addEventListener()`
                    3. `.createElement()`
                    4. `.innerHTML`
                    5. `.textContent`
                    6. `.children`
                    7. etc.
        2. Types of nodes:
            1. HTML elements
            2. Text

How the DOM API is Organized Behind the Scenes

1. Every single node in a DOM tree is of type `Node`
    1. Each node is represented by JS as an Object
        1. Object gets access to `Node` methods and properties such as:
            1. `.textContent`
            2. `.childNodes`
            3. `.parentNodes`
            4. `.cloneNode()`
            5. etc.
    2. `Node` type has a few child types
        1. `Element`
            1. It internally has `HTMLElement` child type
                1. It has exactly one child type for each element that exists in HTML (because each HTML element can have unique properties - example: `img` has `src` attribute, `link` and `a` have `href` attributes)
                    1. `HTMLButtonElement`
                    2. `HTMLDivElement`
                    3. ...
        2. `Text`
            1. `<p>Paragraph</p>`
                1. A text inside an element gets its own node (`Node`)
                    1. The type will be `Text`
                2. Same with `<!-- Comment -->` (Type is `Comment`)
                3. `<p>...</p>` get type as `Element`
                    1. Gives access to many HTML properties
                        1. `.innerHTML`
                        2. `.classList`
                        3. `.children`
                        4. `.parentElement`
                        5. `.append()`
                        6. `.remove()`
                        7. `.insertAdjacentHTML()`
                        8. `.querySelector()`
                        9. `.closest()`
                        10. `.matches()`
                        11. `.scrollIntoView()`
                        12. `.setAttribute()`
                        13. etc.
        3. `Comment`
        4. `Document`
2. Inheritance - all child types will get access to methods and properties of their parent types
    1. Inheritance of methods and properties
        1. Example: Any `HTMLElement` will have access to:
            1. `.addEventListener()`
            2. `.cloneNode()`
            3. `.closest()`
            4. And everything from `Node` type (grand-parent and other ancestors)
3. `Document` - a type of node
    1. Contains:
        1. `.querySelector()` (available on `Document` and `Element` type)
        2. `.createElement()`
        3. `.getElementById()`
4. Events:
    1. `EventTarget` (abstract type) - parent of `Node` type and `Window` node type (Global object, lots of methods and properties, many unrelated to DOM)
        1. Contains
            1. `.addEventListener()` (every element type supports - because `Node` inherits from this type)
            2. `.removeEventListener()` **(M)**
5. MDN web docs - has more info

Selecting, Creating, and Deleting Elements

1. Code:

```
console.log(document.documentElement); // **(M)** selects the document element (`document` is not real DOM element)
console.log(document.head);
console.log(document.body);

const header = document.querySelector('.header'); // returns first element that matches the selector
const allSections = document.querySelectorAll('.section'); // returns multiple elements
console.log(allSections);

document.getElementById('section--1');
const allButtons = document.getElementsByTagName('button'); // all buttons (elements having the given name)
console.log(allButtons); // returns `HTMLCollection` - live collection (if DOM changes, collection is immediately updated automatica

console.log(document.getElementsByClassName('btn')); // returns a live HTML collection

// Constructing and inserting elements
// .insertAdjacentHTML - quick and easy way

const message = document.createElement('div'); // returns a DOM element
message.classList.add('cookie-message');
message.textContent =
  'We use cookies for improved functionality and analytics.';
message.innerHTML =
  'We use cookies for improved functionality and analytics. <button class="btn btn--close-cookie">Got it!</button>';

header.prepend(message); // **(M)** adds the element as the first child
header.append(message); // **(M)** adds the element as the last child (inserted only once - not inserted at multiple places). Moves
// header.append(message.cloneNode(true)); // **(M)** `true` - all child elements will be copied

header.before(message); // as a sibling before the element
header.after(message); // as a sibling after the element

// Delete elements
document
  .querySelector('.btn--close-cookie')
  .addEventListener('click', function () {
    message.remove(); // **(M)**
    // message.parentElement.removeChild(message); // older way
  });
```

Styles, Attributes and Classes

1. Code:

```
// Styles,
message.style.backgroundColor = '#37383d'; // set as inline style
message.style.width = '120%';

console.log(message.style.height); // doesn't return anything - works only for inline styles we set
console.log(message.style.backgroundColor);

console.log(getComputedStyle(message)); // **(M)**
console.log(getComputedStyle(message).color);
console.log(getComputedStyle(message).height);

message.style.height =
  Number.parseFloat(getComputedStyle(message).height, 10) + 40 + 'px';

console.log(getComputedStyle(message).height);

// CSS custom properties (variables)
// :root - equivalent to `document.documentElement`

document.documentElement.style.setProperty('--color-primary', 'orangered'); // **(M)**, can be used to set any property

// Attributes
const logo = document.querySelector('.nav__logo');
console.log(logo.alt); // directly accessible (JS adds the properties to the object automatically). Works only on standard propertie
console.log(logo.src); // returns absolute URL
console.log(logo.className); // `class`

// Non-standard
console.log(logo.designer); // doesn't work
console.log(logo.getAttribute('designer')); // **(M)**

logo.alt = 'Beautiful minimalist logo';
console.log(logo.alt);
logo.setAttribute('company', 'Bankist'); // a new attribute is created

// returns only the relative path specified
console.log(logo.getAttribute('src'));

const link = document.querySelector('.nav__link--btn');
console.log(link.href);
console.log(link.getAttribute('href'));

// Data attributes - special kind of attributes
console.log(logo.dataset.versionNumber); // `data-<name>` are always stored in `dataset` object
// Used to store data in UI (in HTML)

// Classes
logo.classList.add('c', 'j');
logo.classList.remove('c', 'j');
logo.classList.toggle('c');
logo.classList.contains('c'); // Not includes

logo.className = 'jonas'; // Don't use (overrides all existing class. It allows only one class on an element)
```

Implementing Smooth Scrolling

1. Code: Old School:

```
const btnScrollTo = document.querySelector('.btn--scroll-to');
const section1 = document.querySelector('#section--1');

btnScrollTo.addEventListener('click', function (e) {
  const s1coords = section1.getBoundingClientRect(); // **(M)**. x (from left), y (from top), width, height

  console.log(s1coords);
  console.log(e.target.getBoundingClientRect());

  console.log('Current scroll (X/Y)', window.pageXOffset, window.pageYOffset); // distance between current position of viewport and

  console.log(
    'height/width viewport',
    document.documentElement.clientHeight,
    document.documentElement.clientWidth
  ); // **(M)**

  // Scrolling
  // window.scrollTo(
  //   s1coords.left + window.pageXOffset,
  //   s1coords.top + window.pageYOffset
  // ); // **(M)** (doesn't work if we just use `s1coords.top` because `top` is relative to viewport and not the document)

  window.scrollTo({
    left: s1coords.left + window.pageXOffset,
    top: s1coords.top + window.pageYOffset,
    behavior: 'smooth',
  });
});
```

2. Code: Modern

```
section1.scrollIntoView({
  behavior: 'smooth',
}); // **(M)**
```

Types of Events and Event Handlers

1. Event: A signal generated by a certain DOM node
    1. It indicates something has happened
        1. A click
        2. Mouse move
        3. User triggering full-screen mode
        4. ...
    2. We can listen for events using event-listeners (to handle them)
        1. Irrespective of whether we handle an event or not, the event always happens when a user clicks
2. Other ways of handling events:
    1. Search: mdn events
        1. List of events
3. Code:

```
const alertH1 = function (e) {
  alert('addEventListener: Great! You are reading the heading!');

  h1.removeEventListener('mouseenter', alertH1); // Makes it listen to the event only once
};

const h1 = document.querySelector('h1');
h1.addEventListener('mouseenter', alertH1); // fired when a mouse enters the element

// h1.onmouseenter = alertH1; // **(M)**. Old-school. If we use it again, the existing function will get overridden

setTimeout(() => h1.removeEventListener('mouseenter', alertHi), 3000);

// Another way
// onclick="alert('Bla bla');"
```

Event Propagation: Bubbling and Capturing

Bubbling and Capturing

1. Example: DOM tree

```
DOCUMENT
    |
    v
ELEMENT (<html>)
    |
    v
ELEMENT (<body>)
    |
    v
ELEMENT (<section>)
    |
    v
ELEMENT (<p>)
    |
    v
ELEMENT (<a>)
```

1. If a click happens on `<a>`
    1. DOM generates a 'click' event immediately
        1. It is not generated at the target element (where the event actually happened)
            1. It is generated at the root of the document (at the top of DOM tree)
    2. **Capturing phase:**
        1. The event travels down from the root to the target element
            1. As the event passes through the tree, it travels to every parent element of the target element
                1. DOCUMENT -> html -> body -> section -> p -> a
    3. **Target Phase:**
        1. As soon as the event reaches target, this phase begins
            1. Events are handled at the target
                1. Using event listeners
                    1. Callback function is run
    4. **Bubbling Phase:**
        1. Event then travels all the way upto the target root
            1. Bubble up from the target to the root
                1. Passing through all the parents (not siblings)

2. Why?

    1. It is as if the event also happened in each of the parent elements
        1. If we attach an event listener to a parent element, then the event listener is triggered (handling multiple times)
            1. Powerful patterns can be implemented
    2. By default events are handled at the **target phase** and **bubbling phase**
        1. We can also set up event handling during **capturing phase**

3. Not all events have capturing and bubbling phases

    1. They are created at the target element
        1. They can only be handled there

4. Event propagation:

    1. Capturing and bubbling phases

Event Propagation in Practice

1. Example: Event Bubbling

2. Code:

```
// Bubbling and Propagation
// rgb(255, 255, 255)

const randomInt = (min, max) =>
  Math.floor(Math.random() * (max - min + 1) + min);

const randomColor = () =>
  `rgb(${randomInt(0, 255)}, ${randomInt(0, 255)}, ${randomInt(0, 255)})`;

console.log(randomColor());

document.querySelector('.nav__link').addEventListener(
  'click',
  function (e) {
    console.log('LINK', e.target, e.currentTarget); // where the event originated (not the element the handler is attached)
    // `currentTarget` - the element to which the handler is attached
    console.log(e.currentTarget === this);
    this.style.backgroundColor = randomColor();

    // Stop propagation
    // e.stopPropagation(); // parent elements will not receive the events (During bubbling phases) - required for complex handlers
  },
  true
);

document.querySelector('.nav__links').addEventListener('click', function (e) {
  console.log('LINK', e.target, e.currentTarget);
  this.style.backgroundColor = randomColor();
});

document.querySelector('.nav').addEventListener('click', function (e) {
  console.log('LINK', e.target, e.currentTarget);
  this.style.backgroundColor = randomColor();
});
```

    1. Event-handlers are not handling events during capture phase (by default)
        1. It is irrelevant for us
            1. Bubbling is useful for delegation
        2. To capture events during capture phase:
            1. Third param must be `true` - (capture phase)
            2. Third param must be `false` - (bubbling phase)

Event Delegation: Implementing Page Navigation

1. Asigning event listener on each element:

```
document.querySelectorAll('.nav__link').forEach(function (el) {
  el.addEventListener('click', function (e) {
    e.preventDefault(); // avoids browser moving the page to the specified anchor, which is not smooth
    console.log('LINK');
    const id = this.getAttribute('href');
    console.log(id);
```

```
    document.querySelector(id).scrollIntoView({
      behavior: 'smooth',
    });
  });
}); // not very efficient - we are adding the exact same function to each element (if we have 10000 elements, we would be having 100


// Event delegation - we put event listener in common parent

// 1. Add event listener to common parent element
// 2. Determine what element originiated the event

document.querySelector('.nav__links').addEventListener('click', function (e) {
  console.log(e.target); // the element where the event occurred
  e.preventDefault();

  // Matching strategy
  if (e.target.classList.contains('nav__link')) {
    console.log('LINK');
    const id = e.target.getAttribute('href');
    console.log(id);
    document.querySelector(id).scrollIntoView({
      behavior: 'smooth',
    });
  }
}); // better strategy
```

1. Another usecase of event delegation
    1. When working with elements that are not yet on the page on runtime (when page loads)
        1. Example: Buttons added dynamically while using the application
            1. It is not possible to add event listeners to elements that do not exist

DOM Traversing

1. DOM traversing
    1. Walking through the DOM
        1. Selecting an element based on another element
            1. Sometimes we need to select elements relative to other elements
                1. Direct child
                2. Direct parent
                3. ...
                4. We may not know the structure of DOM at runtime
2. Code:

```
const h1 = document.querySelector('h1');

// Going downwards: child
console.log(h1.querySelectorAll('.highlight')); // also works on elements
// Selects all elements that has .highlight class that are descendants of the `h1` element
console.log(h1.childNodes); // **(M)**, returns `NodeList`

console.log(h1.children); // **(M))**, returns `HTMLCollection`. Works only for direct children

h1.firstElementChild.style.color = 'white';
console.log(h1.lastElementChild);
h1.lastElementChild.style.color = 'orangered'; // **(M)**

// Going upwards
console.log(h1.parentNode); // **(M)**
console.log(h1.parentElement); // **(M)** (same as `parentNode` - `parentElement` is also a node in this case)

h1.closest('.header').style.background = 'var(--gradient-secondary)'; // **(M)**. Returns the closest ancestor that has the class

// If the element calling the method matches the closest element, then that element is returned
h1.closest('h1').style.background = 'var(--gradient-primary)'; // unlike `querySelector`, which selects children (no matter how deep

// Going sideways: siblings
console.log(h1.previousElementSibling); // **(M)**
console.log(h1.nextElementSibling); // **(M)**

console.log(h1.previousSibling); // **(M)** (Node)
console.log(h1.nextSibling); // **(M)** (Node)

console.log(h1.parentElement.children); // all siblings including itself
[...h1.parentElement.children].forEach(function (el) {
  if (el !== h1) el.style.transform = 'scale(0.5)';
});
```

Building a Tabbed Component

1. If we click a button, the content below will change
2. Code:

```
// Tabbed component
const tabs = document.querySelectorAll('.operations__tab');
const tabsContainer = document.querySelector('.operations__tab-container');
const tabsContent = document.querySelectorAll('.operations__content');

// tabs.forEach(t => t.addEventListener('click', () => console.log('TAB')));

tabsContainer.addEventListener('click', function (e) {
  const clicked = e.target.closest('.operations__tab');
  // console.log(e.target); // We get internal elements of the button sometimes

  // Guard clause
  if (!clicked) return;
```

```
  // if (clicked) {
  // ...
  // } // more traditional way

  // Active tab
  tabs.forEach(t => t.classList.remove('operations__tab--active'));
  clicked.classList.add('operations__tab--active');

  // Remove active classes
  tabsContent.forEach(c => c.classList.remove('operations__content--active'));

  // Activate content area
  // console.log(clicked.dataset.tab);
  document
    .querySelector(`.operations__content--${clicked.dataset.tab}`)
    .classList.add('operations__content--active');
});
```

Passing Arguments to Event Handlers

1. If we hover over a link, the rest of the links fade-out
2. Code:

```
// Menu fade animation
const handleHover = function (e) {
  if (e.target.classList.contains('nav__link')) {
    // since there are no child elements
    const link = e.target;
    const siblings = link.closest('.nav').querySelectorAll('.nav__link');
    const logo = link.closest('.nav').querySelector('img');

    siblings.forEach(el => {
      if (el !== link) {
        el.style.opacity = this;
      }
    });

    logo.style.opacity = this;
  }
};

const nav = document.querySelector('.nav');
// Passing "argument" into handler
nav.addEventListener('mouseover', handleHover.bind(0.5)); // mouseenter does not bubble
// If we want to pass multiple values, we can pass an array or an object

nav.addEventListener('mouseout', handleHover.bind(1));
```

Implementing a Sticky Navigation: The Scroll Event

1. Code:

```
// Sticky navigation
const initialCoords = section1.getBoundingClientRect();
console.log(initialCoords); // current top value of the section

window.addEventListener('scroll', function () {
  console.log(window.scrollY);

  if (this.window.scrollY > initialCoords.top) {
    nav.classList.add('sticky');
  } else {
    nav.classList.remove('sticky');
  }
}); // fired each time we scroll on our page
// It is bad in terms of performance - scroll event fires all the time - especially on mobile
```

A Better Way: The Intersection Observer API

1. What is intersection observer API and why is it helpful?
    1. API Allows the code to observe changes to the way certain target element intersects another element or it intersects the viewport
2. Example:

```
const obsCallback = function (entries, observer) {
  // object itself is passed
  // called when target element is intersecting the `root` element by the % specified in the `threshold`, when scrolling up or down
  // `entries` - an array of threshold entries
  // Why is it efficient? We get the event only in the case we are interested in

  entries.forEach(entry => {
    console.log(entry); // called when threshold is reached
  });
};

const obsOptions = {
  root: null, // element the target is intersecting, `null` - target intersecting the entire viewport
  // threshold: 0.1, // percentage of intersection at which the observer callback will be called
  threshold: [0, 0.2], // `0` - callback will trigger moves completely out of view or as soon as it enters the view
};

const observer = new IntersectionObserver(obsCallback, obsOptions);
observer.observe(section1); // <target-element>,
```

3. Code:

```
const obsCallback = function (entries, observer) {
  // object itself is passed
  // called when target element is intersecting the `root` element by the % specified in the `threshold`, when scrolling up or down
  // `entries` - an array of threshold entries
  // Why is it efficient? We get the event only in the case we are interested in

  entries.forEach(entry => {
    console.log(entry); // called when threshold is reached
  });
};

const obsOptions = {
  root: null, // element the target is intersecting, `null` - target intersecting the entire viewport
  // threshold: 0.1, // percentage of intersection at which the observer callback will be called
  threshold: [0, 0.2], // `0` - callback will trigger moves completely out of view or as soon as it enters the view
};

const observer = new IntersectionObserver(obsCallback, obsOptions);
observer.observe(section1); // <target-element>
const navHeight = nav.getBoundingClientRect().height;
console.log(navHeight); // bounding rect has `height` property

const stickyNav = function (entries) {
  const [entry] = entries;
  console.log(entry);

  if (!entry.isIntersecting) nav.classList.add('sticky');
  else nav.classList.remove('sticky');
};

const header1 = document.querySelector('.header');
const headerObserver = new IntersectionObserver(stickyNav, {
  root: null,
  threshold: 0,
  // rootMargin: '-90px', // a margin of 90px applied outside the target element - as though the header is smaller by 90px. 90px is
  rootMargin: `-${navHeight}px`,
});
headerObserver.observe(header1);
```

Revealing Elements on Scroll

1. We add class as we approach a section
2. Code:

```
// Reveal sections
const allSections1 = document.querySelectorAll('.section');

const revealSection = function (entries, observer) {
  const [entry] = entries;
  console.log(entry);

  if (!entry.isIntersecting) return;

  entry.target.classList.remove('section--hidden');

  observer.unobserve(entry.target); // **(M)**
};

const sectionObserver = new IntersectionObserver(revealSection, {
  root: null,
  threshold: 0.15,
});
allSections1.forEach(function (section) {
  sectionObserver.observe(section);
  section.classList.add('section--hidden');
});
```

Lazy Loading Images

1. Initially, image is blurred (which is very small in size)
2. To simulate slow network:
    1. Network tab > Slow 3G
3. Code:

```
// Lazy Loading Images
const imgTargets = document.querySelectorAll('img[data-src]');
console.log(imgTargets);

const loadImg = function (entries, observer) {
  const [entry] = entries;
  console.log(entry);

  if (!entry.isIntersecting) return;

  // Replace src with data-src
  entry.target.src = entry.target.dataset.src;

  entry.target.addEventListener('load', function () {
    entry.target.classList.remove('lazy-img');
  });

  observer.unobserve(entry.target);
};

const imgObserver = new IntersectionObserver(loadImg, {
  root: null,
  threshold: 0,
```

```
  rootMargin: '200px',
});

imgTargets.forEach(img => imgObserver.observe(img));
```

Building a Slider Component: Part 1

1. Layout
    1. Slides are side-by-side
        1. `overflow: hidden` - hides slides that overflow the parent
    2. `transform: translateX(-100%);`
        1. When we click, the `translateX` value changes
2. Code:

```
// Slider
let curSlide = 0;
const maxSlide = slides.length;

const slides = document.querySelectorAll('.slide');
const btnLeft = document.querySelector('.slider__btn-left');
const btnRight = document.querySelector('.slider__btn-right');

const goToSlide = function (slide) {
  slides.forEach((s, i) => {
    s.style.transform = `translateX(${(i - slide) * 100}%)`;
  });
};

goToSlide(0);
// 0%, 100%, 200%, 300%

const nextSlide = function () {
  if (curSlide === maxSlide - 1) {
    curSlide = 0;
  } else {
    curSlide++;
  }

  goToSlide(curSlide);
};

const prevSlide = function () {
  if (curSlide === 0) {
    curSlide = maxSlide - 1;
  } else {
    curSlide--;
  }

  goToSlide(curSlide);
};

// Next slide
btnRight.addEventListener('click', nextSlide);
btnLeft.addEventListener('click', prevSlide);
```

Building a Slider Component: Part 2

1. Code:

```
// Slider
const slider = function () {
  let curSlide = 0;
  const maxSlide = slides.length;

  const slides = document.querySelectorAll('.slide');
  const btnLeft = document.querySelector('.slider__btn--left');
  const btnRight = document.querySelector('.slider__btn--right');
  const dotContainer = document.querySelector('.dots');

  const goToSlide = function (slide) {
    slides.forEach((s, i) => {
      s.style.transform = `translateX(${(i - slide) * 100}%)`;
    });
  };

  // goToSlide(0);
  // 0%, 100%, 200%, 300%

  const nextSlide = function () {
    if (curSlide === maxSlide - 1) {
      curSlide = 0;
    } else {
      curSlide++;
    }

    goToSlide(curSlide);
    activateDot(curSlide);
  };

  const prevSlide = function () {
    if (curSlide === 0) {
      curSlide = maxSlide - 1;
    } else {
      curSlide--;
    }

    goToSlide(curSlide);
    activateDot(curSlide);
  };
```

```
// Next slide
btnRight.addEventListener('click', nextSlide);
btnLeft.addEventListener('click', prevSlide);

document.addEventListener('keydown', function (e) {
  console.log(e);
  if (e.key === 'ArrowLeft') prevSlide();
  else if (e.key === 'ArrowRight') nextSlide();
});

// Dots
const instantiateDots = function () {
  slides.forEach((_, i) => {
    dotContainer.insertAdjacentHTML(
      'beforeEnd',
      `<button class="dots__dot" data-slide="${i}"></button>`
    );
  });
};

// instantiateDots();/

const activateDot = function (slide) {
  document
    .querySelectorAll('.dots__dot')
    .forEach(dot => dot.classList.remove('dots__dot--active'));

  document
    .querySelector(`.dots__dot[data-slide="${slide}"]`)
    .classList.add('dots__dot--active');
};
// activateDot(0);

dotContainer.addEventListener('click', function (e) {
  if (e.target.classList.contains('dots__dot')) {
    console.log('DOT');
    const { slide } = e.target.dataset;
    goToSlide(slide);
    activateDot(slide);
  }
});

const init = function () {
  goToSlide(0);
  instantiateDots();
  activateDot(0);
};
init();
};

slider();
```

Lifecycle DOM Events

1. Events:
    1. DOMContentLoaded - event is fired as soon as the HTML is completely parsed (converted to DOM tree, all scripts are downloaded and executed)

Efficient Script Loading: defer and async

Section 14: Object-Oriented Programming (OOP) with JavaScript

Section Intro

1. Two paradigms:
    1. OOP
        1. This section
            1. Required to become a professional JS developer
                1. What is it?
                2. Different ways to implement it?
                    1. Constructor functions
                    2. ES6 classes
                    3. Object.create
                    4. ...
                    5. Inheritance between classes
    2. Functional Programming

Section Roadmap

What is Object-Oriented Programming?

1. Topics:
    1. What is OOP?
    2. How does it work in general?
    3. 4 fundamental principles

What is Object-Oriented Programming

1. Object-Oriented programming (OOP) is a programming paradigm based on the concept of objects.
    1. Paradigm: Style of code, "how" we write and organize code

2. We use objects to **model** (describe) real-world or abstract features.
    1. Examples:
        1. user
        2. todo list item
        3. HTML component
        4. Data structure
3. Objects may contain data (properties) and code (methods). By using objects, we pack **data and the corresponding behavior** into one block
    1. Example:

```
const user = {
    user: 'jonas',
    password: 'dk343s',

    login(password) {
        // login logic
    },

    sendMessage(str) {
        // Sending logic
    }
}
```

4. In OOP, objects are **self-contained** pieces/blocks of code
5. Objects are **building blocks** of applications, and **interact** with one another
6. Interactions happen through a **public interface** (API): methods that the code **outside** of the object can access and use to communicate with the object.
7. OOP was developed with the goal of **organizing** code, to make it **more flexible and easier to maintain** (avoid "spaghetti code")
    1. Otherwise it is scattered in multiple functions or in global scope without any structure
        1. Makes it hard to maintain large code-bases and add new functionality
    2. Functional programming is another paradigm - avoids spagheti code

Classes and Instances (Traditional OOP)

1. We have so far used objects as lose collections of data and we didn't make they interact with one another
2. We also didn't have a way to generate objects programmatically
    1. We used simple object literals
        1. For creation of objects:
            1. A way: Class
                1. Class: A blueprint used for creation of objects based on rules described in class (an abstract plan)
                    1. Example: User class
                        1. JS does not have classes. It works a bit differently in JS
                    2. Instance: All objects created using the class
                        1. Contains real data
                    3. Class can be used for creation of multiple instances (can have different data but can share the same functionality)
3. How do we actually design classes? How do we model real-world data into classes?
    1. There is no single correct way to design classes
        1. 4 fundamental principles can guide us to design good class implementation
            1. Abstraction
            2. Encapsulation
            3. Inheritance
            4. Polymorphism

The 4 Fundamental OOP Principles

1. Abstraction
    1. Ignoring or hiding details that **don't matter**, allowing us to get an **overview** perspective of the thing we're implementing, instead of messing with details that don't really matter to our implementation
    2. Example: Phone
        1. Without abstraction (everything including internal stuff)

```
Phone {
    charge
    volume
    voltage
    temperature

    homeBtn() {}
    volumeBtn() {}
    screen() {}
    verifyVolt() {}
    verifyTemp() {}
    vibrate() {}
    soundSpeaker() {}
    soundEar() {}
    frontCamOn() {}
    frontCamOff() {}
    rearCamOn() {}
    rearCamOff() {}
} // all low level details
```

            1. A user doesn't need those details when he interacts with a phone
                1. Solution:

```
Phone {
    charge
    volume

    homeBtn() {}
    volumeBtn() {}
    screen() {}
} // details have been abstracted away
```

1. The phone becomes a blackbox for us

2. Example: User can include phone number, hair color, shoe size, etc. - we may not need in the application

3. It is very important in programming
    1. Example: `addEventListener` - we don't know how it works behind the scenes
        1. We don't care how it works
            1. We can use it without understanding it, but understand how to use it

2. Encapsulation

    1. Keep some properties a methods private inside a class

        1. Not accessible outside the class
            1. Some methods can be exposed as public interface (API)

    2. Example:

```
User {
    username
    private password
    private email

    login(word) {
        this.password === word
    }

    comment(text) {
        this.checkSPAM(text)
    }

    private checkSPAM(text) {
        // Verify logic
    }
}
```

        1. Outside code cannot access private methods but inside code can access
            1. We prevent external code from accidentally manipulating internal properties/state
                1. Can cause issues in large code-bases and dev teams
        2. Private methods make it easier to change internal code without breaking the depending code.

    3. We must keep most of our state and methods private only leaving the essential methods public

3. Inheritance

    1. Example: User and Admin

```
User {
    user
    password
    email

    login(password) {
        // login logic
    }

    sendMessage(str) {
        // Sending logic
    }
}

Admin {
    user
    password
    email
    permissions

    login(password) {
        // Login logic
    }

    sendMessage(str) {
        // Sending logic
    }

    deleteUser(user) {
        // Deleting logic
    }
}
```

        1. Both have a lot in common
            1. Admin has all properties and methods that user has
                1. Admin is also a user
                    1. Needs password, email, needs to login
            2. The repitition results in duplicate code
                1. Solution: Inheritance
                    1. Child class extends parent class
                        1. Similar to a child inheriting some features of parents

2. Inheritance: Making all properties and methods of a certain class **available to a child class**, forming a hierarchical relationship between classes. This allows us to **reuse common logic** and to model real-world relationships

    1. If both user and admin needs to login, we don't have to duplicate the logic in both places
    2. The child class additionally can have more methods and properties

4. Polymorphism

    1. Polymorphism: A child class can **overwrite** a method it inherited from a parent class [it's more complex than that, but enough for our purposes]

    2. Example:

```
Admin {
    username
    password
    email
    permissions

    login(password, key) {
        // DIFFERENT LOGIN
    }

    deleteUser(username) {
        // Deleting logic
    }
}

User {
    username,
    password,
    email,

    login(password) {
        // Login logic
    }

    sendMessage(str) {
        // Sending logic
    }
}

Author {
    username
    password
    email
    posts

    login(password) {
        // MORE DIFFERENT
    }

    writePost() {
        // Writing Logic
    }
}
```

        1. Admin and Author inherit from user class
            1. They are special kinds of users
                1. Admin can have a more secure login method (2FA)
                2. Author can have a different login method

OOP in JavaScript

1. How is it different in JS?
2. Different ways of implementing the paradigm in JS

OOP in JavaScript: Prototypes

1. Classical OOP: classes
    1. A theoretical plan (similar to the one we use to build houses)
    2. Objects (instances) are **instantiated** from a class, which functions like a blueprint
        1. The concepts are used in JS
    3. Behavior (methods) is **copied** from class to all instances
2. OOP in JS: Prototypes
    1. Prototype
        1. All objects in JS are linked to a prototype
            1. Each object has a prototype
        2. Prototype object contains methods and properties that all the objects linked to the prototype can access and use
            1. **Prototypal Inheritance**:
                1. The prototype contains methods (behavior) that are **accessible to all objects** linked to that prototype
                2. Objects inherit methods and properties from the prototype
                    1. **Instance inherits from a class**
                3. Behavior (methods) is **delegated** to the linked prototype object
                    1. **Delegation**
                        1. Objects delegate their behavior to prototype
3. Examples:

    1. Array:

```
const num = [1, 2, 3];
num.map(v => v * 2); // `num` is linked to the prototype `Array.prototype` and hence inherits `map` method
```

        1. `Array.prototype.map()`
            1. `Array.prototype` is the prototype of all array objects we construct in JavaScript

1. All arrays have access to the `map` method

3 Ways of Implementing Prototypal Inheritance in JavaScript

1. Questions:
    1. How do we actually construct prototypes?
    2. How do we link objects to prototypes?
    3. How can we construct new objects, without having classes?
2. Three ways of doing OOP in JS:
    1. Constructor functions
        1. Technique to construct objects from a function
            1. How built-in objects like Arrays, Maps or Sets are actually implemented
                1. How OOP was done in JS since beginning
    2. ES6 classes
        1. Modern alternative to construct function syntax
        2. "Syntactic sugar": behind the scens, ES6 classes work **exactly** like constructor functions
            1. A layer of abstraction over constructor functions (nicer syntax)
        3. ES6 classes do **NOT** behave like classes in "classical OOP" (last lecture)
    3. `Object.create()` **(M)**
        1. Easiest and most straightforward way of linking an object to a prototype object.
            1. Not as used

Constructor Functions and the new Operator

1. 14-OOP
2. Constructor function is a normal function
    1. Difference: We call it with the `new` operator
    2. An arrow function does not work as a constructor function
        1. Why? It does not have its own `this` keyword
    3. `new` does 4 steps:
        1. **A new empty object is created**
        2. **Function is called, `this` is set to the newly created object** (`this = {}`)
        3. **Newly created object is linked to a prototype**
        4. **Function automatically returns the object** ({})
    4. It is used to simulate classes
3. Code:

```
const Person = function (firstName, birthYear) {
  console.log(this); // empty object
  this.firstName = firstName; // same names is convention
  this.birthYear = birthYear;

  // never instantiate a method inside a constructor function
  // If multiple instances are created, multiple copies of the function are created
  // this.calcAge = function () {
  //   console.log(2037 - this.birthYear);
  // }; // bad practice
}; // Convention: Use Capital letter for constructor function

const jonas = new Person('Jonas', 1991); // `new` - calls the defiend function and does much more

const matilda = new Person('Matilda', 2017);
const jack = new Person('Jack', 1975);

console.log(matilda);
console.log(jack);

console.log(jonas instanceof Person); // **(M)**
```

Prototypes

1. Each and every object in JS automatically has a property called prototype (which includes constructor functions)
    1. Every object created through constructor function will get access to all methods and properties that we define in constructor's prototype property
        1. `Person.prototype` (`prototype` property of `Person` constructor function)
            1. All objects created using the `Person` constructor function will get access to all the methods and properties defined in the `Person.prototype` property
            2. It works because any object always has access to the methods and properties of its prototype
                1. Each object has a special property called `__proto__` - this is the prototype of the object (not prototype property of the object)
                    1. This matches with the `prototype` property of the constructor function
                        1. `Person.prototype` is not the prototype of function but will be used as prototype (`__proto__`) of the objects instantiated through the constructor function
                            1. `new` will instantiate `__proto__` property and sets its value to `prototype` of the constructor function
2. Code:

```
// Prototypes
console.log(Person.prototype); // It has `calcAge` function

Person.prototype.calcAge = function () {
  // only one copy exists for all instances
  console.log(2037 - this.birthYear); // `this` is set to object calling the method
};

jonas.calcAge(); // the method is not on the object itself
matilda.calcAge();
jack.calcAge();
```

```
console.log(jonas.__proto__);
console.log(jonas.__proto__ === Person.prototype);

console.log(Person.prototype.isPrototypeOf(jonas)); // **(M)**
console.log(Person.prototype.isPrototypeOf(matilda));

// .prototypeOfLinkedObjects

Person.prototype.species = 'Homo Sapiens';
console.log(jonas.species);
console.log(matilda.species);

console.log(jonas.hasOwnProperty('firstName')); // **(M)**, true
console.log(jonas.hasOwnProperty('species')); // false
```

Prototypal Inheritance and the Protoype Chain

How Prototypal Inheritance / Delegation Works

1. Constructor Function
    1. [Person()]
    2. It has `.prototype` property which is an object
        1. Inside the object we define `calcAge` method
    3. `Person.prototype` has a reference back to `Person`
        1. `.constructor`
            1. `Person.prototype.constructor` points back to `Person`
    4. `Person.prototype` is not the prototype of the function, but the prototype of all the objects created through the function
2. Example:

```
const jonas = new Person('Jonas', 1998);
jonas.calcAge();
```

    1. The new operator:
        1. An empty object is created
        2. `this` keyword in constructor function call is set to the new object
            1. Next we set `this.name` and `this.birthYear`
        3. The new object is linked (`__proto__` property) to the constructor function's `prototype` property
            1. `__proto__`: `Person.prototype` (always points to an object's prototype)
        4. The new object is automatically returned from the constructor function call
            1. We can explicitly return something else (we usually never do that)
3. This is how it works with function constructors and ES6 classes
    1. But not with `Object.create`
4. Why does it work this way? Why is it useful?
    1. `jonas.calcAge()`
        1. JS will not find the `calcAge()` method directly in the object
        2. JS will look into its prototype `__proto__`
        3. All objects created through the constructor will get the methods and properties in `__proto__`
5. Prototype chain:
    1. The fact that an object is connected to a prototype and looks up properties and methods in its prototype is called prototype chain
        1. The object and its prototype forms a prototype chain

Prototype Chain

1. `Object`
    1. `prototype` - This is `__proto__` of `Person` function
        1. `Person` is an object built using the `Object()` constructor function
            1. `Object()` - built-in constructor function for objects. This is used when we write an object literal:
                1. `{...} === new Object(...)`
                    1. The function called behind the scenes whenever we construct an object literal (`{...}`)
                        1. A shortcut to writing `new Object(...)`
                2. `Person`'s prototype is the same as `Object.prototype`
2. `Person` (construction function) - This is an object itself! Remember, every object in JavaScript has a prototype
    1. `prototype`

Prototype Chain

1. Series of links between objects, linked through prototypes (Similar to the scope chain)
    1. `Object.prototype` is usually the top of the prototype-chain
        1. `Object`'s prototpye is `null` (`__proto__` is `null`)
        2. Similar to scope-chain but with prototypes
            1. Whenever a JS cannot find a property or method, it is going to look up into the next prototype in the prototype chain
2. `jonas.hasOwnProperty('name')`
    1. JS tries to find the method `hasOwnProperty` method in the object itself
    2. JS cannot find it, so it looks up in its prototype `__proto__`
    3. JS cannot find it, so it will move up to `__proto__` of `Person` constructor function object (`Object.prototype`)
    4. `Object.prototype` has many methods and `hasOwnProperty` is one of them
    5. JS then runs the function as if it were defined directly defined on the object itself

Prototypal Inheritance on Built-in Objects

1. Code:

```
console.log(jonas.__proto__);
console.log(jonas.__proto__.__proto__); // `Object.prototype`
console.log(jonas.__proto__.__proto__.__proto__); // `null`

console.dir(Person.prototype.constructor); // `Person` function

const arr = [4, 5, 2, 7, 1, 5, 5, 4, 3, 1, 1, 1]; // same as `new Array()`
console.log(arr.__proto__ === Array.prototype); // contains all array methods. `Array` is construction function
console.log(arr.__proto__.__proto__); // `Object.prototype`

// Array.prototype.filter - lives in __proto__ property of the array
// This mechanism exists for re-using code

Array.prototype.unique = function () {
  return [...new Set(this)];
};

console.log(arr.unique()); // We can call the method on any array we want
// Not a good idea - okay for small projects
// Reason 1: Next version of JS might add a method with the same name which might behave differetly. The existing code might break
// Reason 2: If multiple developers define methods with same name it might cause bugs

const h1 = document.querySelector('h1');
console.dir(h1); // shows methods and properties
// __proto__ - `HTMLHeadingElement`
// `HTMLHeadingElement.__proto__` - `HTMLElement`
// `HTMLElement.__proto__` - `Element` (constructor function)
// `Element.__proto__` - `Node`
// `Node.__proto__` - `EventTarget`
// `EventTarget.__proto__` - `Object`

console.log(x => x + 1); // It is an object
// __proto__ contains: `apply`, `bind`, `call`, ...
```

Coding Challenge #1

ES6 Classes

1. We can do the same thing as we do with constructor functions but using a nicer syntax.
2. Classes in JS are syntactic sugar over prototypal inheritance
    1. Goal: To make it convenient for people coming from other languages
3. Important points:
    1. Classes are not hoisted (even if there are class declarations)
        1. We cannot use them before they are declared
    2. Classes are also first-class citizens
        1. We can pass them into functions and return them from functions (they are just special functions)
    3. Classes are executed in strict-mode (even if not activated in the entire script)
4. Never use classes if constructor function and prototypal are not understood
5. Class syntax looks much more organized as compared to writing raw constructor functions
6. Code:

```
// Class expression - Classes are special type of functions
// const PersonCl = class {};

// Class declaration
class PersonCl {
  constructor(firstName, birthYear) {
    // works similar to constructor function, but it is a method
    // The constructor is automatically called when `new` is used for creation of objects, and a new object is created
    this.firstName = firstName;
    this.birthYear = birthYear;
  }

  calcAge() {
    // This will be on the prototype of the object (and not on the object)
    console.log(2037 - this.birthYear);
  }

  // greet() {
  //   console.log(`Hey ${this.firstName}`);
  // }
}

const jessica = new PersonCl('Jessica', 1996);
console.log(jessica); // `calcAge()` is in `__proto__`
jessica.calcAge();

console.log(jessica.__proto__ === PersonCl.prototype);

PersonCl.prototype.greet = function () {
  console.log(`Hey ${this.firstName}`);
};
jessica.greet();
```

Setters and Getters

1. Feature common to all objects in JS
2. Setters and getters - accessor properties
    1. Properties that set and get values

    1. But from the outside they look like regular properties

3. Code:

```
class PersonCl {
  constructor(fullName, birthYear) {
    // works similar to constructor function, but it is a method
    // The constructor is automatically called when `new` is used for creation of objects, and a new object is created
    this.fullName = fullName; // this will execute the setter. The name matches the setter
    this.birthYear = birthYear;
  }

  calcAge() {
    // This will be on the prototype of the object (and not on the object)
    console.log(2037 - this.birthYear);
  }

  // greet() {
  //   console.log(`Hey ${this.firstName}`);
  // }

  get age() {
    return 2037 - this.birthYear;
  }

  set fullName(name) {
    // Set a property that already exists
    if (name.includes(' ')) {
      this._fullName = name; // convention - avoids naming conflict
    } else {
      alert(`${name} is not a full name!`);
    }
  }

  get fullName() {
    return this._fullName; // `jessica.fullName`
  }
}

const jessica = new PersonCl('Jessica', 1996);
console.log(jessica); // `calcAge()` is in `__proto__`
jessica.calcAge();

console.log(jessica.__proto__ === PersonCl.prototype);

PersonCl.prototype.greet = function () {
  console.log(`Hey ${this.firstName}`);
};
jessica.greet();

const account = {
  owner: 'jonas',
  movements: [200, 530, 120, 300],

  get latest() {
    // It is on the prototype
    return this.movements.slice(-1).pop();
  },

  set latest(mov) {
    this.movements.push(mov);
  },
};

console.log(account.latest); // written as a property, instead of a method
// Use-case: To retrieve a value as a property but we want to do some calculations prior

account.latest = 50; // set like any other property
console.log(account.movements);

console.log(jessica.age);

const walter = new Person('Walter', 1965); // `_fullName` is not set
const walter1 = new Person('Walter White', 1965); // `_fullName` is set

// Many people don't use getters and setters
```

Static Methods

1. Example: `Array.from()` - converts any array-like structure to a real array

```
Array.from(document.querySelectorAll('h1'));
```

    1. `from` - is attached to `Array` constructor
        1. We cannot use it from an array
        2. It is not attached to the `prototyp` property of the constructor
            1. The arrays do not inherit the method
        3. A simple function that is attached to `Array` constructor
        4. It is in the `Array` namespace
    2. Static methods are used as helpers related to a constructor

2. Code:

```
// Static functions
Person.hey = function () {
  console.log('Hey there!');
  console.log(this); // the entire constructor function `Person` (the object calling the method)
};

Person.hey();
// jonas.hey(); // It is not in the `prototype`
```

```
// ...

class PersonCl {
    // ...
    static hey() {
        // **(M)**
        console.log('Hey there!');
        console.log(this);
      }
}

PersonCl.hey();
```

Object.create

1. Third way: Works differently than constructor functions and classes
2. Prototypal inheritance is application
    1. But
        1. No prototype properties involved
        2. No constructor functions involved
        3. No `new` operator
3. `Object.create()`
    1. Used to manually set prototype of an object (`__proto__`) to any other object we want

How Object.create Works

1. We can set the prototype of an object (`__proto__`) manually to any prototype object we want
2. The prototype chain is exactly the same as before
3. Differences:
    1. No constructor function required
    2. No `prototype` proeperty used
4. In practice, it is the least used way of implementing prototypal inheritance
5. Code:

```
// Object.create()
const PersonProto = {
  calcAge() {
    console.log(2037 - this.birthYear);
  },
  init(firstName, birthYear) {
    this.firstName = firstName;
    this.birthYear = birthYear;
  },
};

const steven = Object.create(PersonProto); // **(M)** returns a new object that is linked to the object passed in
console.log(steven);
steven.name = 'Steven';
steven.birthYear = 2002;

steven.calcAge();

console.log(steven.__proto__ === PersonProto); // same as `PersonProto`

const sarah = Object.create(PersonProto);
sarah.init('Sarah', 1979); // `this` is set to `sarah`
sarah.calcAge();
```

Coding Challenge #2

1. Code:

```
class CarCl {
  constructor(make, speed) {
    this.make = make;
    this.speed = speed;
  }

  accelerate() {
    this.speed += 10;
    console.log(this.make, ':', this.speed);
  }

  brake() {
    this.speed -= 5;
    console.log(this.make, ':', this.speed);
  }

  get speedUS() {
    return this.speed / 1.6;
  }

  set speedUS(speed) {
    this.speed = this.speed * 1.6;
  }
}

const ford = new CarCl('Ford', 120);
console.log(ford.speed);
console.log(ford.speedUS);

ford.accelerate();
```

```
ford.accelerate();
ford.brake();
ford.brake();
ford.brake();

console.log(ford.speed);
console.log(ford.speedUS);

ford.speedUS = 200;
console.log(ford.speed);
console.log(ford.speedUS);
```

Inheritance Between "Classes": Constructor Functions

1. The techniques discussed so far allow objects to inherit methods from its prototype

Inheritance Between Classes

1. A new Student class that inherits from Person class (Student is a Person)
2. Inheritance using:
    1. Constructor functions
        1. Prototype chain
            1. `Student()` - constructor function
                1. `Student.prototype`
                2. = `mike.__proto__` - `new` automatically links prototype
            2. Following prototype chain is desired:

```
Person() -> Person.prototype
                    ^
                    | __proto__
Student() ->Student.prototype
                    ^
                    | __proto__
                Object [mike]
```

                1. All instances of `Student` can get access to the properties and methods of `Person` through prototype chain
                    1. The idea of inheritance is that child classes can share the behavior of parent classes
    2. ES6 classes
        1. Easier
    3. `Object.create()`
        1. Used to manually define prototype for an object
3. Code:

```
// Inheritance
const Student = function (firstName, birthYear, course) {
  // this.firstName = firstName; // copy
  // this.birthYear = birthYear; // copy
  // Person(firstName, birthYear); // `new` is not used. It is a regular function call. `this` is undefined
  Person.call(this, firstName, birthYear); // `this` will be the empty object createed through the `new` operator
  this.course = course;
};

Student.prototype = Object.create(Person.prototype); // `Student.prototype` is empty at this point. Properties and methods are added

Student.prototype = Person.prototype; // This doesn't work
// Chain:  Student.__proto__ === Person.prototype
// mile.__proto__ === Person.prototype (no inheritance)
// The above is wrong

Student.prototype.introduce = function () {
  console.log(`My name is ${this.firstName} and I study ${this.course}`);
};

const mike = new Student('Mike', 2020, 'Computer Science');
console.log(mike);
mike.introduce();
mike.calcAge(); // Looks up in the prototype chain
// We can call the method that is in `Object` constructor function

console.log(mike.__proto__);
console.log(mike.__proto__.__proto__);
console.dir(Student.prototype.constructor); // points to `Person` - Because `Object.create` was used. Needs to be fixed (sometimes w

Student.prototype.constructor = Student; // Fix
console.dir(Student.prototype.constructor);

console.log(mike instanceof Student); // true
console.log(mike instanceof Person); // true
console.log(mike instanceof Object); // true
```

Coding Challenge #3

1. Code:

```
const EV = function (make, speed, charge) {
  Car.call(this, make, speed);
  this.charge = charge;
};

EV.prototype = Object.create(Car.prototype);
EV.prototype.chargeBattery = function (chargeTo) {
```

```
  this.charge = chargeTo;
};

EV.prototype.accelerate = function () {
  this.speed += 20;
  this.charge -= 0.01 * this.charge;
  console.log(
    `${this.make} going at ${this.speed} km/h, with a charge of ${this.charge}%`
  );
};

console.dir(EV);

const ev = new EV('Tesla', 120, 10);
console.log(ev);

ev.accelerate();
ev.accelerate();

ev.chargeBattery(90);

ev.accelerate();
ev.accelerate();
ev.accelerate();

ev.brake();
ev.brake();
```

Inheritance Between "Classes": ES6 Classes

1. Code:

```
// Classes
class StudentCl extends PersonCl {
  // Links prototyps behind the scenes
  constructor(fullName, birthYear, course) {
    // If no additional arguments are passed, the constructor function is not required. The `super` function is automatically called
    super(fullName, birthYear); // Constructor function of the `Parent` class
    // Always needs to happen first - `super` is responsible for creation of `this` keyword. Or else it is not accessible
    this.course = course; // This is not necessary
  }

  introduce() {
    console.log(`My name is ${this.fullName} and I study ${this.course}`);
  }

  calcAge() {
    // Shadows the one in the parent class
    console.log(
      `I'm ${
        2037 - this.birthYear
      } years old, but as a student I feel more like ${
        2037 - this.birthYear + 10
      }`
    );
  }
}

const martha = new StudentCl('Martha Jones', 2012, 'Computer Science');
const martha2 = new StudentCl('Martha Jones', 2012);
console.log(martha2);
martha2.introduce();
martha2.calcAge();
```

Inheritance Between "Classes": Object.create

1. Inheritance:

```
Prototype
[PersonProto]
    calcAge: function

    ^
    | .__proto__

Prototype
[StudentProto]
    __proto__: PersonProto

    ^
    | .__proto__

Object
[jay]
    __proto__: StudentProto
```

    1. This approach is better than faking classes using constructor functions and ES6 classes
    2. ES6 classes are used more

2. Code:

```
// Object.create
const PersonProto2 = {
  calcAge() {
    console.log(2037 - this.birthYear);
  },

  init(firstName, birthYear) {
    this.firstName = firstName;
    this.birthYear = birthYear;
  },
```

```
};

const steven = Object.create(PersonProto2);

const StudentProto = Object.create(PersonProto2); // `PersonProto2` is the prototype of `StudentProto`

StudentProto.init = function (firstName, birthYear, course) {
  PersonProto.init.call(this, firstName, birthYear);
  this.course = course;
};

StudentProto.introduce = function () {
  console.log(`My name is ${this.fullName} and I study ${this.course}`);
};

const jay = Object.create(StudentProto); // `StudentProto` is the prototype of `jay`
jay.init('Jay', 2010, 'Computer Science');
jay.introduce();
jay.calcAge();
```

Another Class Example

1. Code:

```
class Account {
  constructor(owner, currency, pin) {
    this.owner = owner;
    this.currency = currency;
    this.pin = pin;
    this.movements = {};
    this.locale = navigator.language; // **(M)**
  }

  deposit(val) {
    this.movements.push(val);
  }

  withdraw(val) {
    this.deposit(-val); // `this` is required
  }

  approveLoan(val) {
    return true;
  }

  requestLoan(val) {
    // We only want this method in public interface
    if (this.approveLoan(val)) {
      this.deposit(val);
      console.log('Loan approved');
    }
  }
}

const acc1 = new Account('Jonas', 'EUR', 1111);
console.log(acc1);

// acc1.movements.push(250); // not a good idea
// acc1.movements.push(-140);

acc1.deposit(250); // API
acc1.withdraw(140); // API

// We can still interact with properties directly
console.log(acc1.pin);

acc1.requestLoan(1000);
acc1.approveLoan(1000); // This must not be allowed
```

Encapsulation: Protected Properties and Methods

1. Encapsulation: To keep some properties and methods private inside the class so that they are not accessible outside of the class
    1. Rest of the methods are exposed as public interface called API
    2. Why do we need it?
        1. To prevent code outside of the class to accidentally manipulate code inside the class
        2. When we expose only a small API, we can change all internal methods with more confidence
            1. External code does not rely on private methods
                1. Avoids breaking other code
    3. JS classes do not truly support privacy
2. Code:

```
class Account {
  constructor(owner, currency, pin) {
    this.owner = owner;
    this.currency = currency;
    this._pin = pin;
    // Protected property
    this._movements = {}; // faking privacy
    this.locale = navigator.language; // **(M)**
  }

  deposit(val) {
    this._movements.push(val);
  }

  withdraw(val) {
    this.deposit(-val); // `this` is required
  }
```

```
  _approveLoan(val) {
    return true;
  }

  requestLoan(val) {
    // We only want this method in public interface
    if (this.approveLoan(val)) {
      this.deposit(val);
      console.log('Loan approved');
    }
  }

  get movements() {
    return this._movements;
  }
}
```

Encapsulation: Private Class Fields and Methods

1. It is part of a bigger proposal for improving and changing JS classes (called class-fields)
2. Some parts work in Google chrome
3. JS is moving away from classes being just syntactic sugars
4. Four kinds of fields/methods
    1. Public fields
        1. Field: It is a property that will be on all instances (public instance field)
    2. Private fields
    3. Public methods
    4. Private methods
5. There are static versions of the 4 kinds of fields/methods

```
static helper() {
    console.log('Helper');
}

// ...

Acount.helper();
```

6. Code:

```
class Account {
  // 1. Public field - they are part of the instances
  locale = navigator.language; // `;` is required. No `const` or `let` is required

  // 2. Private fields
  #movements = []; // **(M)** private field. Only Google Chrome supports at the time of recording
  #pin; // empty variable. Set to `undefined`

  constructor(owner, currency, pin) {
    this.owner = owner;
    this.currency = currency;
    this.#pin = pin;
    // Protected property
    this.#movements = []; // faking privacy
    this.locale = navigator.language; // **(M)**
  }

  // 3. Public methods
  deposit(val) {
    this.#movements.push(val);
  }

  withdraw(val) {
    this.deposit(-val); // `this` is required
  }

  requestLoan(val) {
    // We only want this method in public interface
    if (this.#approveLoan(val)) {
      this.deposit(val);
      console.log('Loan approved');
    }
  }

  get movements() {
    return this.#movements;
  }

  // 4. Private Methods
  #approveLoan(val) {
    return true;
  }
}

const acc1 = new Account('Jonas', 'EUR', 1111);
console.log(acc1);

// acc1.movements.push(250); // not a good idea
// acc1.movements.push(-140);

acc1.deposit(250); // API
acc1.withdraw(140); // API

// We can still interact with properties directly
// console.log(acc1.#pin);
```

```
acc1.requestLoan(1000);
// acc1.#approveLoan(1000); // This must not be allowed

// acc1.#movements.push(250); // this is possible. It is however not allowed by the team
// console.log(acc1.#movements); // calls the getter
```

Chaining Methods

1. Easy to do. We can just return the object that we want to be chainable
2. Code:

```
deposit(val) {
  this.#movements.push(val);
  return this;
}
```

ES6 Classes Summary

1. Child class: Student
    1. Parent class: Person (`extends` sets up inheritance - sets up prototype chain for us)
2. Public field
3. Private field (not accessible outside the class)
4. Static public field
5. `constructor` - method automatically called by `new` operator
    1. Can be omitted in child class (otherwise mandatory)
6. `super` - only when we have child class
    1. Required before we access `this` keyword
7. Instance property
    1. Based on input data of constructor (unique to each object)
8. Private field
9. Private methods may not yet work
    1. We can use _ convension
10. `get` - getter method
11. `set` - setter method
    1. We might need to use _ if same samed variable already exists
12. `static` method
    1. It can access only `static` members
    2. Usually used as helper methods
13. `new` - used for creation of objects
14. Classes are syntactic sugar
    1. They are not hoisted
    2. They are first-class citizens
    3. Their body is executed in strict mode

Coding Challenge #4

Section 15: Mapty App: OOP, Geolocation, External Libraries, and More!

Section Intro

1. We will apply everything learn't so far
2. How to use geolocation
3. How to use third-party libraries
4. How to plan a project
    1. Thinking
    2. Taking decisions
    3. Solving problems

Section Roadmap

Project Overview

1. Map - loaded from third-party service
2. Geolocation - fetches position and loads the map
3. Starter: 15-Mapty

How to Plan a Web Project

Project Planning

1. Process:
    1. Planning Step:
        1. User Stories
            1. A description of the application's functionality from the user's perspective. All user stories put together describe the entire application.
            2. All user stories put together will clearly describe the functionality of the entire application
            3. A high-level overview of the application, which allows developers to determine the features to implement to make user stories work as intended
        2. Features
        3. Flowchart (What we will build)
            1. To visualize actions that a user can take and how the program reacts to the actions, we put them in a flowchart
        4. Architecture (How we will build it)
            1. How we will organize our code
            2. What JS features we will use

    3. It gives us structure which we can use to build functionality
    4. Without architecture, we might end up with unmanageable spaggetti code
2. Development Step
    1. Implementation of the plan using code

User Stories

1. User story: Description of the application's functionality from the user's perspective
2. There are multiple formats
    1. Common format: As a [type of user], I want [an action] so that [a benefit]
        1. Answers: who, what, why
3. User Stories for mapty app:
    1. As a user, I want to **log my running workouts with location, distance, time, pace, and steps/minute**, so I can keep a log of all my running.
    2. As a user, I want to **log my cycling workouts with location, distance, time, speed and elevation gain**, so I can keep a log of all my cycling.
    3. As a user, I want to **see all my workouts at a glance**, so I can easily track my progress over time.
    4. As a user, I want to **also see my workouts on a map**, so I can easily check where I work out the most.
    5. As a user, I want to **see all my workouts when I leave the app and come back later**, so that I can keep using the app over time.

Features

1. User Stories
    1. Log my running workouts with location, distance, time, pace, and steps/minute
        1. Features:
            1. Map where user clicks to add new workout (best way to get location coordinates)
            2. Geolocation to display map at current location (more user friendly)
                1. Users don't have to scroll to their current location
                2. All mobile and desktop browsers support
            3. From to input distance, time, pace, steps/minute
    2. Log my cycling workouts with location, distnace, time, speed, and elevation gain
        1. Feature: Form to input distance, time, speed, elevation gain
    3. See all my workouts at a glance
        1. Feature: Display all workouts in a list
    4. See my workouts on a map
        1. Feature: Display all workouts on the map
    5. See all my workouts when I leave the app and come back later
        1. Feature:
            1. Store workout data in the browser using local storage API
            2. On page load, read the saved data from local storage and display

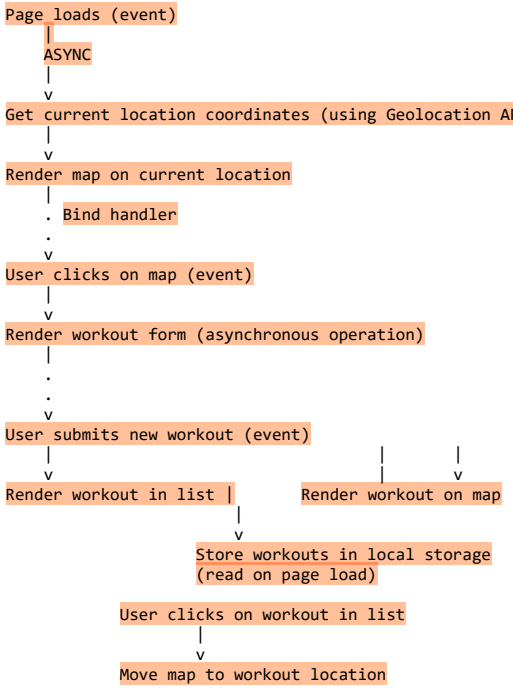
Flowchart

1. Flowchart should contain all features we are going to implement, and how different parts of the app are going to interact with each other, which event makes sense to implement, and how data flows across the application
2. Features:

    1. Geolocation to display map at current location

    2. Map where user clicks to add new workout

    3. Flowchart: (We start with events)



    1. Page load: First event that occurs
        1. All the code at the top level will get executed when the page loads
    2. Async operations:
        1. They take time, and the rest of the operations get executed only after its completion

Architecture

1. We can do experiments, play with code, and then update the architecture
2. We can come back to architecture

Using the Geolocation API

1. Geolocation API
    1. Browser API similar to
        1. I18n
        2. Timers
        3. ...
    2. A modern API
    3. Other modern APIs
        1. User's camera
        2. User's phone vibration
        3. ...
2. It is very easy to use
3. Code:

```
if (navigator.geolocation) {
  // **(M)**
  navigator.geolocation.getCurrentPosition(
    function (position) {
      // called when the call is successful
      console.log(position);
      const { latitude } = position.coords;
      const { longitude } = position.coords;
      console.log(latitude, longitude);
      console.log(`https://www.google.com/maps/@${latitude},${longitude}`);
    },
    function () {
      // called when the call has failed
      alert('Could not get your position');
    }
  );
}
```

Displaying a Map Using Leaflet Library

1. Leaflet: Google it
    1. An open-source JS library for mobile-friendly interactive maps
    2. Download
        1. Hosted version (CDN)
        2. npm
            1. `npm install leaflet`
2. Any variable global in any script will be available in the other scripts (if the scripts appear after the script defining the global variable)
3. Code:

```
if (navigator.geolocation) {
  // **(M)**
  navigator.geolocation.getCurrentPosition(
    function (position) {
      // called when the call is successful
      console.log(position);
      const { latitude } = position.coords;
      const { longitude } = position.coords;
      console.log(latitude, longitude);
      console.log(`https://www.google.com/maps/@${latitude},${longitude}`);

      const coords = [latitude, longitude];

      const map = L.map('map').setView(coords, 13); // L - main function that leaflet gives (namespace). It is a global variable.
      // 13 - zoom level

      L.tileLayer('https://tile.openstreetmap.fr/hot/{z}/{x}/{y}.png', {
        maxZoom: 19,
        attribution:
          '&copy; <a href="http://www.openstreetmap.org/copyright">OpenStreetMap</a>',
      }).addTo(map);
      // map is made up of tiles
      // openstreetmap - open source map we can use for free
      // leaflet also works with other maps (google maps say)

      var marker = L.marker(coords).addTo(map);
    },
    function () {
      // called when the call has failed
      alert('Could not get your position');
    }
  );
}
```

Displaying a Map Marker

1. Code:

```
if (navigator.geolocation) {
  // **(M)**
  navigator.geolocation.getCurrentPosition(
```

```
    function (position) {
      // called when the call is successful
      console.log(position);
      const { latitude } = position.coords;
      const { longitude } = position.coords;
      console.log(latitude, longitude);
      console.log(`https://www.google.com/maps/@${latitude},${longitude}`);

      const coords = [latitude, longitude];

      const map = L.map('map').setView(coords, 13); // L - main function that leaflet gives (namespace). It is a global variable.
      // 13 - zoom level

      L.tileLayer('https://tile.openstreetmap.fr/hot/{z}/{x}/{y}.png', {
        maxZoom: 19,
        attribution:
          '&copy; <a href="http://www.openstreetmap.org/copyright">OpenStreetMap</a>',
      }).addTo(map);
      // map is made up of tiles
      // openstreetmap - open source map we can use for free
      // leaflet also works with other maps (google maps say)

      var marker = L.marker(coords).addTo(map);

      console.log(map);
      map.on('click', function (mapEvent) {
        console.log(mapEvent);

        const { lat, lng } = mapEvent.latlng;
        L.marker([lat, lng])
          .addTo(map)
          .bindPopup(
            L.popup({
              maxWidth: 250,
              minWidth: 100,
              autoClose: false,
              closeOnClick: false,
              className: 'running-popup',
            })
          )
          .setPopupContent('Workout')
          .openPopup();
      }); // leaflet library method
    },
    function () {
      // called when the call has failed
      alert('Could not get your position');
    }
  );
}
```

Rendering Workout Input Form

1. Code:

```
let map;
let mapEvent;

if (navigator.geolocation) {
  // **(M)**
  navigator.geolocation.getCurrentPosition(
    function (position) {
      // called when the call is successful
      console.log(position);
      const { latitude } = position.coords;
      const { longitude } = position.coords;
      console.log(latitude, longitude);
      console.log(`https://www.google.com/maps/@${latitude},${longitude}`);

      const coords = [latitude, longitude];

      map = L.map('map').setView(coords, 13); // L - main function that leaflet gives (namespace). It is a global variable.
      // 13 - zoom level

      L.tileLayer('https://tile.openstreetmap.fr/hot/{z}/{x}/{y}.png', {
        maxZoom: 19,
        attribution:
          '&copy; <a href="http://www.openstreetmap.org/copyright">OpenStreetMap</a>',
      }).addTo(map);
      // map is made up of tiles
      // openstreetmap - open source map we can use for free
      // leaflet also works with other maps (google maps say)

      var marker = L.marker(coords).addTo(map);

      console.log(map);
      // Handling clicks on map
      map.on('click', function (mapE) {
        mapEvent = mapE;
        form.classList.remove('hidden');
        inputDistance.focus(); // **(M)**

        // console.log(mapEvent);
      }); // leaflet library method
    },
    function () {
      // called when the call has failed
      alert('Could not get your position');
    }
  );
}

form.addEventListener('submit', function (e) {
```

```
    e.preventDefault();

    // Clear input fields
    inputDistance.value =
      inputDuration.value =
      inputCadence.value =
      inputElevation.value =
        '';

    // Display marker
    const { lat, lng } = mapEvent.latlng;
    L.marker([lat, lng])
      .addTo(map)
      .bindPopup(
        L.popup({
          maxWidth: 250,
          minWidth: 100,
          autoClose: false,
          closeOnClick: false,
          className: 'running-popup',
        })
      )
      .setPopupContent('Workout')
      .openPopup();
  });

  inputType.addEventListener('change', function () {
    inputElevation.closest('.form__row').classList.toggle('form__row--hidden');
    inputCadence.closest('.form__row').classList.toggle('form__row--hidden');
  });
```

Project Architecture

Architecture: Initial Approach

1. User Stories:
    1. Log my running workouts with **location**, **distance**, **time**, **pace**, and **steps/minute** (cadence)
    2. Log my cycling workouts with **location**, **distance**, **time**, **speed**, and **elevation gain**
2. Data is fundamental to an application
    1. Class Workout
        1. id
        2. distance
        3. duration
        4. coords
        5. date
        6. constructor()
        7. ...
    2. Child Class Runing
        1. name
        2. **cadence**
        3. **pace**
        4. constructor()
        5. ...
    3. Child Class Cycling
        1. name
        2. **elevationGain**
        3. **speed**
        4. constructor()
        5. ...
3. Events: We need a structure to handle the following events
    1. Load page
    2. Receive position
    3. Click on map
    4. Change input
    5. Submit form
4. Structure:
    1. Class App
        1. workouts - Array holding all Running or Cycling objects
        2. map
        3. constructor() <- Load page
        4. _getPosition()
        5. _loadMap(position) <- Receive position
        6. _showForm() <- Click on map
        7. _toggleElevationField() <- Change input
        8. _newWorkout() <- Submit form (Stores all workouts in `workouts`)
            1. Child Class Running
                1. new Running()
            2. Child Class Cycling
                1. new Cycling()
5. Application and data will be separated in a logical way
6. Data and methods protected

Refactoring for Project Architecture

1. Event handler will have `this` pointing to the DOM element to which the handler is attached

2. Code:

```
class App {
  #map;
  #mapEvent;
```

```
  constructor() {
    this._getPosition();
    form.addEventListener('submit', this._newWorkout.bind(this)); // `this` points to the DOM element to which event handler is atta

    inputType.addEventListener('change', this._toggleElevationField);
  }

  _getPosition() {
    if (navigator.geolocation) {
      // **(M)**
      navigator.geolocation.getCurrentPosition(
        this._loadMap.bind(this),
        function () {
          // called when the call has failed
          alert('Could not get your position');
        }
      );
    }
  }

  _loadMap(position) {
    // called when the call is successful
    console.log(position);
    const { latitude } = position.coords;
    const { longitude } = position.coords;
    console.log(latitude, longitude);
    console.log(`https://www.google.com/maps/@${latitude},${longitude}`);

    const coords = [latitude, longitude];

    this.#map = L.map('map').setView(coords, 13); // L - main function that leaflet gives (namespace). It is a global variable.
    // 13 - zoom level

    L.tileLayer('https://tile.openstreetmap.fr/hot/{z}/{x}/{y}.png', {
      maxZoom: 19,
      attribution:
        '&copy; <a href="http://www.openstreetmap.org/copyright">OpenStreetMap</a>',
    }).addTo(this.#map);
    // map is made up of tiles
    // openstreetmap - open source map we can use for free
    // leaflet also works with other maps (google maps say)

    var marker = L.marker(coords).addTo(this.#map);

    console.log(map);
    // Handling clicks on map
    this.#map.on('click', this._showForm.bind(this)); // leaflet library method. It is event handler function (`this` points to obje
  }

  _showForm(mapE) {
    this.#mapEvent = mapE;
    form.classList.remove('hidden');
    inputDistance.focus(); // **(M)**
  }

  _toggleElevationField() {
    inputElevation.closest('.form__row').classList.toggle('form__row--hidden');
    inputCadence.closest('.form__row').classList.toggle('form__row--hidden');
  }

  _newWorkout() {
    e.preventDefault();

    // Clear input fields
    inputDistance.value =
      inputDuration.value =
      inputCadence.value =
      inputElevation.value =
        '';

    // Display marker
    const { lat, lng } = this.#mapEvent.latlng;
    L.marker([lat, lng])
      .addTo(this.#map)
      .bindPopup(
        L.popup({
          maxWidth: 250,
          minWidth: 100,
          autoClose: false,
          closeOnClick: false,
          className: 'running-popup',
        })
      )
      .setPopupContent('Workout')
      .openPopup();
  }
}

const app = new App();
// app._getPosition();
```

Managing Workout Data: Constructing Classes

1. Code:

```
class Workout {
  date = new Date();
  id = (Date.now() + '').slice(-10); // not yet part of the official JS language

  constructor(coords, distance, duration) {
    this.coords = coords; // [lat, lng]
    this.distance = distance; // in Km
    this.duration = duration; // in min
```

```
  }
}

class Running extends Workout {
  constructor(coords, distance, duration, cadence) {
    super(coords, distance, duration);
    this.cadence = cadence;
    this.calcPace();
  }

  calcPace() {
    // min/km
    this.pace = this.duration / this.distance;
    return this.pace;
  }
}

class Cycling extends Workout {
  constructor(coords, distance, duration, elevationGain) {
    super(coords, distance, duration);
    this.elevationGain = elevationGain;
    this.calcSpeed();
  }

  calcSpeed() {
    // km/h
    this.speed = this.distance / (this.duration / 60);
    return this.speed;
  }
}

// const run1 = new Running([39, -12], 5.2, 24, 178);
// const cycle1 = new Cycling([39, -12], 27, 95, 523);

// console.log(run1, cycle1);
```

Constructing a New Workout

1. Code:

```
////////////////////////////
// Application Architecture
const form = document.querySelector('.form');
const containerWorkouts = document.querySelector('.workouts');
const inputType = document.querySelector('.form__input--type');
const inputDistance = document.querySelector('.form__input--distance');
const inputDuration = document.querySelector('.form__input--duration');
const inputCadence = document.querySelector('.form__input--cadence');
const inputElevation = document.querySelector('.form__input--elevation');

class App {
  #map;
  #mapEvent;
  #workouts = [];

  constructor() {
    this._getPosition();
    form.addEventListener('submit', this._newWorkout.bind(this)); // `this` points to the DOM element to which event handler is atta

    inputType.addEventListener('change', this._toggleElevationField);
  }

  _getPosition() {
    if (navigator.geolocation) {
      // **(M)**
      navigator.geolocation.getCurrentPosition(
        this._loadMap.bind(this),
        function () {
          // called when the call has failed
          alert('Could not get your position');
        }
      );
    }
  }

  _loadMap(position) {
    // called when the call is successful
    console.log(position);
    const { latitude } = position.coords;
    const { longitude } = position.coords;
    console.log(latitude, longitude);
    console.log(`https://www.google.com/maps/@${latitude},${longitude}`);

    const coords = [latitude, longitude];

    this.#map = L.map('map').setView(coords, 13); // L - main function that leaflet gives (namespace). It is a global variable.
    // 13 - zoom level

    L.tileLayer('https://tile.openstreetmap.fr/hot/{z}/{x}/{y}.png', {
      maxZoom: 19,
      attribution:
        '&copy; <a href="http://www.openstreetmap.org/copyright">OpenStreetMap</a>',
    }).addTo(this.#map);
    // map is made up of tiles
    // openstreetmap - open source map we can use for free
    // leaflet also works with other maps (google maps say)

    var marker = L.marker(coords).addTo(this.#map);

    console.log(map);
    // Handling clicks on map
    this.#map.on('click', this._showForm.bind(this)); // leaflet library method. It is event handler function (`this` points to obje
  }
```

```
  _showForm(mapE) {
    this.#mapEvent = mapE;
    form.classList.remove('hidden');
    inputDistance.focus(); // **(M)**
  }

  _toggleElevationField() {
    inputElevation.closest('.form__row').classList.toggle('form__row--hidden');
    inputCadence.closest('.form__row').classList.toggle('form__row--hidden');
  }

  _newWorkout() {
    const validInputs = (...inputs) =>
      inputs.every(inp => Number.isFinite(inp));

    const allPositive = (...inputs) => inputs.every(inp => inp > 0);

    e.preventDefault();

    // Get data from form
    const type = inputType.value;
    const distance = +inputDistance.value;
    const duration = +inputDuration.value;
    const { lat, lng } = this.#mapEvent.latlng;
    let workout;

    // If workout running, construct running object
    if (type === 'running') {
      const cadence = +inputCadence.value;
      // Check if data is valid
      if (
        // !Number.isFinite(distance) ||
        // !Number.isFinite(duration) ||
        // !Number.isFinite(cadence)
        !validInputs(distance, duration, cadence) ||
        !allPositive(distance, duration, cadence)
      )
        return alert('Inputs have to be positive numbers!');

      workout = new Running([lat, lng], distance, duration, cadence);
    }

    // If workout cycling, construct cycling object
    if (type === 'cycling') {
      const elevation = +inputElevation.value;
      // Check if data is valid
      if (
        // !Number.isFinite(distance) ||
        // !Number.isFinite(duration) ||
        // !Number.isFinite(cadence)
        !validInputs(distance, duration, elevation) ||
        !allPositive(distance, duration)
      )
        return alert('Inputs have to be positive numbers!');

      workout = new Cycling([lat, lng], distance, duration, elevation);
    }

    // Add new object to workout array
    this.#workouts.push(workout);
    console.log(workout);

    // Render workout on map as marker
    this.renderWorkoutMarker(workout);

    // Render workout on list

    // Hide form + clear input fields

    // Clear input fields
    inputDistance.value =
      inputDuration.value =
      inputCadence.value =
      inputElevation.value =
        '';
  }

  renderWorkoutMarker(workout) {
    // Display marker
    L.marker(workout.coords)
      .addTo(this.#map)
      .bindPopup(
        L.popup({
          maxWidth: 250,
          minWidth: 100,
          autoClose: false,
          closeOnClick: false,
          className: `${workout.type}-popup`,
        })
      )
      .setPopupContent('' + workout.distance)
      .openPopup();
  }
}

const app = new App();
// app._getPosition();
```

Rendering Workouts

1. Code:

```
'use strict';

// let map;
// let mapEvent;

class Workout {
  date = new Date();
  id = (Date.now() + '').slice(-10); // not yet part of the official JS language

  constructor(coords, distance, duration) {
    this.coords = coords; // [lat, lng]
    this.distance = distance; // in Km
    this.duration = duration; // in min
  }

  _setDescription() {
    // prettier-ignore
    const months = ['January', 'February', 'March', 'April', 'May', 'June', 'July', 'August', 'September', 'October', 'November', 'D

    this.description = `${this.type[0].toUpperCase()}${this.type.slice(1)} on ${
      months[this.date.getMonth()]
    } ${this.date.getDate()}`;
  }
}

class Running extends Workout {
  type = 'running';

  constructor(coords, distance, duration, cadence) {
    super(coords, distance, duration);
    this.cadence = cadence;
    this.calcPace();
    this._setDescription();
  }

  calcPace() {
    // min/km
    this.pace = this.duration / this.distance;
    return this.pace;
  }
}

class Cycling extends Workout {
  type = 'cycling';

  constructor(coords, distance, duration, elevationGain) {
    super(coords, distance, duration);
    this.elevationGain = elevationGain;
    this.calcSpeed();
    this._setDescription();
  }

  calcSpeed() {
    // km/h
    this.speed = this.distance / (this.duration / 60);
    return this.speed;
  }
}

// const run1 = new Running([39, -12], 5.2, 24, 178);
// const cycle1 = new Cycling([39, -12], 27, 95, 523);

// console.log(run1, cycle1);

///////////////////////////
// Application Architecture
const form = document.querySelector('.form');
const containerWorkouts = document.querySelector('.workouts');
const inputType = document.querySelector('.form__input--type');
const inputDistance = document.querySelector('.form__input--distance');
const inputDuration = document.querySelector('.form__input--duration');
const inputCadence = document.querySelector('.form__input--cadence');
const inputElevation = document.querySelector('.form__input--elevation');

class App {
  #map;
  #mapEvent;
  #workouts = [];

  constructor() {
    this._getPosition();
    form.addEventListener('submit', this._newWorkout.bind(this)); // `this` points to the DOM element to which event handler is atta

    inputType.addEventListener('change', this._toggleElevationField);
  }

  _getPosition() {
    if (navigator.geolocation) {
      // **(M)**
      navigator.geolocation.getCurrentPosition(
        this._loadMap.bind(this),
        function () {
          // called when the call has failed
          alert('Could not get your position');
        }
      );
    }
  }

  _loadMap(position) {
    // called when the call is successful
    console.log(position);
    const { latitude } = position.coords;
    const { longitude } = position.coords;
```

```
    console.log(latitude, longitude);
    console.log(`https://www.google.com/maps/@${latitude},${longitude}`);

    const coords = [latitude, longitude];

    this.#map = L.map('map').setView(coords, 13); // L - main function that leaflet gives (namespace). It is a global variable.
    // 13 - zoom level

    L.tileLayer('https://tile.openstreetmap.fr/hot/{z}/{x}/{y}.png', {
      maxZoom: 19,
      attribution:
        '&copy; <a href="http://www.openstreetmap.org/copyright">OpenStreetMap</a>',
    }).addTo(this.#map);
    // map is made up of tiles
    // openstreetmap - open source map we can use for free
    // leaflet also works with other maps (google maps say)

    var marker = L.marker(coords).addTo(this.#map);

    console.log(map);
    // Handling clicks on map
    this.#map.on('click', this._showForm.bind(this)); // leaflet library method. It is event handler function (`this` points to obje
  }

  _showForm(mapE) {
    this.#mapEvent = mapE;
    form.classList.remove('hidden');
    inputDistance.focus(); // **(M)**
  }

  _hideForm() {
    // Empty inputs
    inputDistance.value =
      inputDuration.value =
      inputCadence.value =
      inputElevation.value =
        '';

    form.style.display = 'none';
    form.classList.add('hidden');
    setTimeout(() => (form.style.display = 'grid'), 1000);
  }

  _toggleElevationField() {
    inputElevation.closest('.form__row').classList.toggle('form__row--hidden');
    inputCadence.closest('.form__row').classList.toggle('form__row--hidden');
  }

  _newWorkout() {
    const validInputs = (...inputs) =>
      inputs.every(inp => Number.isFinite(inp));

    const allPositive = (...inputs) => inputs.every(inp => inp > 0);

    e.preventDefault();

    // Get data from form
    const type = inputType.value;
    const distance = +inputDistance.value;
    const duration = +inputDuration.value;
    const { lat, lng } = this.#mapEvent.latlng;
    let workout;

    // If workout running, construct running object
    if (type === 'running') {
      const cadence = +inputCadence.value;
      // Check if data is valid
      if (
        // !Number.isFinite(distance) ||
        // !Number.isFinite(duration) ||
        // !Number.isFinite(cadence)
        !validInputs(distance, duration, cadence) ||
        !allPositive(distance, duration, cadence)
      )
        return alert('Inputs have to be positive numbers!');

      workout = new Running([lat, lng], distance, duration, cadence);
    }

    // If workout cycling, construct cycling object
    if (type === 'cycling') {
      const elevation = +inputElevation.value;
      // Check if data is valid
      if (
        // !Number.isFinite(distance) ||
        // !Number.isFinite(duration) ||
        // !Number.isFinite(cadence)
        !validInputs(distance, duration, elevation) ||
        !allPositive(distance, duration)
      )
        return alert('Inputs have to be positive numbers!');

      workout = new Cycling([lat, lng], distance, duration, elevation);
    }

    // Add new object to workout array
    this.#workouts.push(workout);
    console.log(workout);

    // Render workout on map as marker
    this._renderWorkoutMarker(workout);

    // Render workout on list
    this._renderWorkout(workout);
```

```
    // Hide form + clear input fields
    this._hideForm();

    // Clear input fields
    inputDistance.value =
      inputDuration.value =
      inputCadence.value =
      inputElevation.value =
        '';
  }

  _renderWorkoutMarker(workout) {
    // Display marker
    L.marker(workout.coords)
      .addTo(this.#map)
      .bindPopup(
        L.popup({
          maxWidth: 250,
          minWidth: 100,
          autoClose: false,
          closeOnClick: false,
          className: `${workout.type}-popup`,
        })
      )
      .setPopupContent(
        `${workout.type === 'running' ? '🏃' : '🚴'} ${workout.description}`
      )
      .openPopup();
  }

  _renderWorkout(workout) {
    const html = `
    <li class="workout workout--${workout.type}" data-id="${workout.id}">
      <h2 class="workout__title">${workout.description}</h2>
      <div class="workout__details">
        <span class="workout__icon">${
          workout.type === 'running' ? '🏃' : '🚴'
        }</span>
        <span class="workout__value">${workout.distance}</span>
        <span class="workout__unit">km</span>
      </div>
      <div class="workout__details">
        <span class="workout__icon">⏱</span>
        <span class="workout__value">${workout.duration}</span>
        <span class="workout__unit">min</span>
      </div>
    `;

    if (workout.type === 'running') {
      html += `
        <div class="workout__details">
            <span class="workout__icon">⚡</span>
            <span class="workout__value">${workout.pace.toFixed(1)}</span>
            <span class="workout__unit">min/km</span>
          </div>
          <div class="workout__details">
            <span class="workout__icon">🦶</span>
            <span class="workout__value">${workout.cadence}</span>
            <span class="workout__unit">spm</span>
          </div>
        </li>
      `;

      if (workout.type === 'cycling') {
        html += `
          <div class="workout__details">
            <span class="workout__icon">⚡</span>
            <span class="workout__value">${workout.speed.toFixed(1)}</span>
            <span class="workout__unit">km/h</span>
          </div>
          <div class="workout__details">
            <span class="workout__icon">⛰</span>
            <span class="workout__value">${workout.elevationGain}</span>
            <span class="workout__unit">m</span>
          </div>
        </li>
        `;

        form.insertAdjacentHTML('afterend', html);
      }
    }
  }
}

const app = new App();
// app._getPosition();
```

Move to Marker On Click

1. Code:

```
  _moveToPopup() {
    const workoutEl = e.target.closest('.workout');
    console.log(workoutEl); // `data-id` is used to uniquely identify the element

    if (!workoutEl) return;

    const workout = this.#workouts.find(
      work => work.id === workoutEl.dataset.id
    );
```

```
    this.#map.setView(workout.coords, this.#mapZoomLevel, {
      animate: true,
      pan: {
        duration: 1,
      },
    });

    // using the public interface
    workout.click();
  }
```

Working with localStorage

1. Local storage:
    1. A place in browser, where we can store data which will stay even after we close the page
    2. Data is linked to the URL we are using on the page

Final Considerations

Section 16: Asynchronous JavaScript: Promises, Async/Await, and AJAX

Section Info

1. Goal: To deal with long-running tasks that run in the backgroud
    1. Use-case: Fetch data from remote servers (using AJAX calls)
2. Topics:
    1. Promises
    2. Fetch function
    3. Async-Await
    4. Error handling

Section Roadmap

Asynchronous JavaScript, AJAX and APIs

1. Use-cases
    1. AJAX calls to APIs
    2. Example:

```
const p = document.querySelector('.p');
p.textContent = 'My name is Jonas!';
alert('Text set!');
p.style.color = 'red';
```

2. Synchronous:
    1. Most code is **synchronous**
    2. Synchronous code is **executed line by line** (in the exact order of execution we defined in the code)
        1. Each line is executed in the execution thread
            1. It is part of execution context that actually executes the code in computer's CPU
            2. Each line of code waits for previous line to finish execution
                1. Example: `alert('...')`
                    1. A long-running operation block code execution
        2. If execution needs to wait, it wastes CPU cycles
    3. Asynchronous code:
        1. Example:

```
const p = document.querySelector('.p'); // sync
setTimeout(function () {

}, 5000); // async - timer will run in the backgroud without preventing the main code from executing
p.style.color = 'red'; // sync
```

            1. Callback function is asynchronous JS (non-blocking - rest of the code can keep running normally)
                1. Asynchronous code is executed **after a task that runs in the "background" finishes;**
                2. Asynchronous code is **non-blocking**
                3. Execution doesn't wait for an asynchronous task to finish its work
                4. Callback runs after all other synchronous code
                    1. Action was deferred to the future

Asynchronous Code

1. Asynchronous programming is about coordinating the behavior of the program over a certain period of time
2. We need a callback function for asynchronous programming
    1. Callback functions alone do NOT make code asynchronous
        1. Not asynchronous callbacks

```
[1, 2, 3].map(v => v * 2);
```

            1. Only timeouts work in asynchronous way
3. Example:

```
const img = document.querySelector('.dog');
img.src = 'dog.jpg'; // the operation is asynchronous (loads image in the background while rest of the code is running)
img.addEventListener('load', function () { // 'load' event is automatically emitted by JS
    img.classList.add('fadeIn');
}); // deferred into future - asynchronous
p.style.width = '300px';
```

1. `eventListeners` alone do not make a code asynchronous
    1. If waiting on 'click' event on a button, it is not asynchronous (just waiting for click but not doing anything in the background)
    2. Image load: the load is happening in the background - hence asynchronous (some work is happening in the background)
    3. Other examples of asynchronous behavior:
        1. Geolocation API
        2. AJAX calls - most important usecase

What are AJAX Calls?

1. AJAX:
    1. Asynchronous JavaScript and XML: Allows us to communicate with remote web servers in an **asynchronous way**. With AJAX calls, we can **request data** from web servers dynamically (without reloading the page)
    2. Consider a JS application running in the browser (can happen in the background)
        1. Client (Browser) can do a HTTP request to web server
        2. Web server sends back response to client
2. API:
    1. Application Programming Interface: Piece of software that canbe used by another piece of software, in order to allow **application to talk to each other**
        1. To exchange information
    2. There are many types of APIs in web development (self-containing pieces of software that allow other software to interact with them)
        1. DOM API
        2. Geolocation API
        3. Own Class API
            1. Objects made from a class can be seen as self-contained encapsulated pieces of software that other pieces of software can interact with
        4. Online API (Just API)
            1. Application running on a server, that receives requests for data, and sends data back as response
        5. We can build **our own** APIs (requires back-end development, e.g. with node.js) or use **3rd party** APIs
            1. Servers
            2. Databases
    3. 3rd Party APIs
        1. Weather data
        2. Data about countries
        3. Flights data
        4. Currency conversion data
        5. APIs for sending email or SMS
        6. Google Maps
    4. API data formats
        1. XML - data format used to be widely used
            1. No API uses XML anymore
        2. JSON - Most APIs use this format
            1. JavaScript object converted to a string
                1. Easy to send across the web and convert to objects in JavaScript

IMPORTANT: API URL Change

1. Use https://countries-api-836d.onrender.com/countries/

Our First AJAX Call: XMLHttpRequest

1. https://github.com/public-apis/public-apis
    1. CORS - must be set to Yes or Unknown
        1. Or else we will not be able to use a third party API
    2. API Endpoints
        1. https://restcountries.com/v3.1/name/{name}

2. Code:

```
const getCountryData = function (country) {
  const request = new XMLHttpRequest(); // Old-school way
  request.open('GET', `https://restcountries.com/v3.1/name/${country}`);
  request.send();

  request.addEventListener('load', function () {
    console.log(this.responseText); // property is set only when the data has arrived

    const [data] = JSON.parse(this.responseText);
    console.log(data);

    const html = `
  <article class="country">
    <img class="country__img" src="${data.flags['svg']}" />
    <div class="country__data">
      <h3 class="country__name">${data.name.common}</h3>
      <h4 class="country__region">${data.region}</h4>
      <p class="country__row"><span>👫</span>${(
        +data.population / 1000000
      ).toFixed(1)} Million people</p>
      <p class="country__row"><span>🗣️</span>${
        Object.values(data.languages)[0]
      }</p>
      <p class="country__row"><span>💰</span>${
        Object.values(data.currencies)[0].name
```

```
      }</p>
    </div>
  </article>
  `;

    countriesContainer.insertAdjacentHTML('beforeend', html);

    countriesContainer.style.opacity = 1;
  });
};

getCountryData('india');
// getCountryData('portugal');
getCountryData('usa');
getCountryData('arabia');
getCountryData('palestine'); // AJAX call happens in parallel. The results might be returned in different order
```

[OPTIONAL] How the Web Works: Requests and Responses

Welcome to Callback Hell

1. Problems with callback hell:
    1. It makes our code looks very messy, hard to maintain, difficult to understand and reason about
        1. It might have more bugs and worse code
        2. It is hard to add new features or fix bugs
    2. Solutions: Promises
2. Code:

```
const renderCountry = function (data, className = '') {
  const html = `
  <article class="country ${className}">
    <img class="country__img" src="${data.flags['svg']}" />
    <div class="country__data">
      <h3 class="country__name">${data.name.common}</h3>
      <h4 class="country__region">${data.region}</h4>
      <p class="country__row"><span>👫</span>${(
        +data.population / 1000000
      ).toFixed(1)} Million people</p>
      <p class="country__row"><span>🗣️</span>${
        Object.values(data.languages)[0]
      }</p>
      <p class="country__row"><span>💰</span>${
        Object.values(data.currencies)[0].name
      }</p>
    </div>
  </article>
  `;

  countriesContainer.insertAdjacentHTML('beforeend', html);

  countriesContainer.style.opacity = 1;
};

const getCountryAndNeighbor = function (country) {
  const request = new XMLHttpRequest(); // Old-school way

  // AJAX call country 1
  request.open('GET', `https://restcountries.com/v3.1/name/${country}`);
  request.send();

  request.addEventListener('load', function () {
    console.log(this.responseText); // property is set only when the data has arrived

    const [data] = JSON.parse(this.responseText);
    console.log(data);

    // Render country 1
    renderCountry(data);

    // Get neighbor country
    const [neighbor] = data.borders;

    if (!neighbor) return;

    const request2 = new XMLHttpRequest(); // Old-school way

    // AJAX call country 1
    request2.open('GET', `https://restcountries.com/v3.1/alpha/${neighbor}`);
    request2.send();

    request2.addEventListener('load', function () {
      console.log(this.responseText);
      const [data2] = JSON.parse(this.responseText);

      console.log(data2);

      renderCountry(data2, 'neighbour');
    });
  });
};

// getCountryAndNeighbor('portugal');
getCountryAndNeighbor('usa');

// Neighbor of neighbor of neighbor ... - 10 times
// Callback hell - a lot of nested callbacks to execute async tasks in sequence

setTimeout(() => {
  console.log('1 second passed');
```

```
  setTimeout(() => {
    console.log('2 seconds passed');
    setTimeout(() => {
      console.log('3 seconds passed');
      setTimeout(() => {
        console.log('4 seconds passed');
      }, 1000);
    }, 1000);
  }, 1000);
}, 1000);
```

Promises and the Fetch API

1. Fetch API: Modern way of making ajax calls

What are Promises?

1. Promise: An object that is used as a placeholder for the future result of an asynchronous operation.
    1. A container for an asynchronously delivered value
    2. A container for a future value
        1. Future value: Example - a response coming from an AJAX call
            1. Promise can be used to handle future value
    3. Example: Like a lottery ticket
        1. A promise that I will receive money if I guess correct outcome
            1. I buy lottery ticket (promise) right now
            2. Lottery draw happens asynchronously
            3. If correct outcome, I receive money, because it was promised
2. Advantages of using Promises (ES6 feature)
    1. We no longer need to rely on events and callbacks passed into asynchronous functions to handle asynchronous results
        1. Events and callback functions can cause unpredictable results
    2. Instead of nesting callbacks, we can **chain promises** for a sequence of asynchronous operations: **escaping callback hell**

The Promise Lifecycle

1. Since promises work with async operations, they are time-sensitive (they change over time. They can be in different states)
2. Lifecycle phases:
    1. Pending: Before the future value is available
        1. During this phase, the async task is still doing its work in the background
    2. Settled: Asynchronous task has finished
        1. There are two different types of settled promises
            1. Fulfilled
                1. Promise that has successfully resulted in a value as we expected (the value is now available)
            2. Rejected
                1. There has been an error during the async task
                    1. Example: When the user is offline and cannot connect to API server
3. We are able to handle these different states in our code! (if successful or failed)
4. A promise is settled only once
    1. The state will remain unchanged for ever
5. Consuming a promise
    1. When we already have a promise
        1. Example: Promise returned from Fetch API
    2. Needs building a promise
        1. Example: Fetch API returns a promise
6. Code:

```
const request = fetch('https://restcountries.com/v3.1/name/portugal');
console.log(request); // immediately returns a promise
```

Consuming Promises

1. Code:

```
const request = fetch('https://restcountries.com/v3.1/name/portugal');
console.log(request); // immediately returns a promise

const getCountryDataPromise = function (country) {
  fetch(`https://restcountries.com/v3.1/name/${country}`)
    .then(function (response) {
      console.log(response);
      return response.json(); // method is available on all response objects coming from all Promises
      // It is an async function which returns a Promise
    })
    .then(function (data) {
      console.log(data);
      renderCountry(data[0]);
    });
};

const getCountryDataPromiseArrow = country => {
  fetch(`https://restcountries.com/v3.1/name/${country}`)
    .then(response => response.json())
    .then(data => renderCountry(data[0]));
};
```

```
// getCountryDataPromise('portugal');
getCountryDataPromiseArrow('portugal');
getCountryDataPromiseArrow('india');
```

Chaining Promises

1. Getting neighbouring country
2. Code:

```
const getCountryDataPromiseArrow = country => {
  fetch(`https://restcountries.com/v3.1/name/${country}`)
    .then(response => response.json())
    .then(data => {
      renderCountry(data[0]);
      const neighbour = data[0].borders[0];

      if (!neighbour) return;

      return fetch(`https://restcountries.com/v3.1/alpha/${neighbour}`); // If we chain `then` here, it is callback hell again
    })
    .then(response => response.json())
    .then(data => renderCountry(data[0], 'neighbour')); // `then` method always returns a promise (if we actually return anything or
  // If we return anything from `then`, it becomes the fulfillment value of the returned promise. `data` will be the actual value
  // Callback hell is replaced with flat chaining of promises
};

// getCountryDataPromise('portugal');
getCountryDataPromiseArrow('portugal');
// getCountryDataPromiseArrow('india');
```

Handling Rejected Promises

1. Errors happen in web-applications
2. Two ways to handle errors
    1. Pass a second callback function to `then` function which will be called when the `Promise` is rejected
3. 404 errors: Fetch promise gets fulfilled and there is no rejection.
4. Code:

```
const renderError = function (msg) {
  countriesContainer.insertAdjacentText('beforeend', msg);
  // countriesContainer.style.opacity = 1;
};

const getCountryDataPromiseArrow = country => {
  fetch(`https://restcountries.com/v3.1/name/${country}`)
    .then(
      response => response.json()
      // ,
      // err => alert(err) // catches the error. The chain stops here due to error
    )
    .then(data => {
      renderCountry(data[0]);
      const neighbour = data[0].borders[0];

      if (!neighbour) return;

      return fetch(`https://restcountries.com/v3.1/alpha/${neighbour}`); // If we chain `then` here, it is callback hell again
    })
    .then(
      response => response.json()
      // ,
      // err => alert(err) // repeating callbacks
    )
    .then(data => renderCountry(data[0], 'neighbour'))
    .catch(err => {
      console.error(`${err} 💥💥💥`);
      renderError(`Something went wrong 💥💥💥 ${err.message}`); // Error can be constructed using a constructor. It contains `mess
      // Catch also returns a Promise
    })
    .finally(() => {
      // always called whether promise is fulfilled or rejected
      // Use it when something always needs to happen. Example: Stopping a loading spinner
      countriesContainer.style.opacity = 1;
    }); // `then` method always returns a promise (if we actually return anything or not)
  // If we return anything from `then`, it becomes the fulfillment value of the returned promise. `data` will be the actual value
  // Callback hell is replaced with flat chaining of promises
  // Global catch. Catches any errors that occur in any place in the promise chain. Errors propagate down to the `catch`
};

btn.addEventListener('click', function () {
  // getCountryDataPromise('portugal');
  getCountryDataPromiseArrow('portugal');
  // getCountryDataPromiseArrow('india');
});

getCountryDataPromiseArrow('ssafdsfsd');
```

Throwing Errors Manually

1. Handling errors is the only way to display error messages on the screen.
    1. It is also a bad practice to leave rejected promises hanging around

2. Application must handle all errors and display error messages that make sense
3. Code:

```
const getJSON = function (url, errorMsg = 'Something went wrong') {
  return fetch(url).then(response => {
    if (!response.ok) throw new Error(`${errorMsg} (${response.status})`);

    return response.json();
  });
};

const getCountryDataPromiseArrow = country => {
  getJSON(`https://restcountries.com/v3.1/name/${country}`, 'Country not found')
    .then(data => {
      renderCountry(data[0]);
      const neighbour = data[0].borders[0];

      if (!neighbour) throw new Error('No neighbour found!');

      return getJSON(
        `https://restcountries.com/v3.1/alpha/${neighbour}`,
        'Country not found'
      ); // If we chain `then` here, it is callback hell again
    })
    .then(data => renderCountry(data[0], 'neighbour'))
    .catch(err => {
      console.error(`${err} 💥💥💥`);
      renderError(`Something went wrong 💥💥💥 ${err.message}`); // Error can be constructed using a constructor. It contains `mess
      // Catch also returns a Promise
    })
    .finally(() => {
      // always called whether promise is fulfilled or rejected
      // Use it when something always needs to happen. Example: Stopping a loading spinner
      countriesContainer.style.opacity = 1;
    }); // `then` method always returns a promise (if we actually return anything or not)
  // If we return anything from `then`, it becomes the fulfillment value of the returned promise. `data` will be the actual value
  // Callback hell is replaced with flat chaining of promises
  // Global catch. Catches any errors that occur in any place in the promise chain. Errors propagate down to the `catch`
};

btn.addEventListener('click', function () {
  // getCountryDataPromise('portugal');
  getCountryDataPromiseArrow('portugal');
  // getCountryDataPromiseArrow('india');
});

getCountryDataPromiseArrow('ssafdsfsd');
```

Coding Challenge #1

1. https://myprojects.geoapify.com/api/xTXvtefd53YCiblQ2Vvo/keys
2. Code:

```
const getCountryDataFromCoords = function (coords) {
  fetch(
    `https://api.geoapify.com/v1/geocode/reverse?lat=${coords.lat}&lon=${coords.lon}&apiKey=856413c61f024071aef052b0ae18d9da`
  )
    .then(response => response.json())
    .then(data => {
      console.log(data);
      console.log(data.features[0].properties.country_code);

      const countryCode = data.features[0].properties.country_code;

      return getJSON(
        `https://restcountries.com/v3.1/alpha/${countryCode}`,
        'Country not found'
      );
    })
    .then(data => {
      renderCountry(data[0]);
    })
    .catch(err => console.error(err))
    .finally(() => (countriesContainer.style.opacity = 1));
};

const lat = 51.21709661403662;
const lon = 6.7782883744862374;

const coords = {
  lat,
  lon,
};

getCountryDataFromCoords(coords);

const coords1 = {
  lat: 19.037,
  lon: 72.873,
};
getCountryDataFromCoords(coords1);

getCountryDataFromCoords({ lat: -33.933, lon: 18.474 });
```

Asynchronous Behind the Scenes: The Event Loop

1. JS Runtime
    1. A container which includes all the pieces necessary to execute JavaScript code

        1. The heart of runtime is the Engine
            1. Call stack: Where code is actually executed
            2. Heap: Where objects are stored in memory
        2. JS has only one thread of execution
            1. It can do only one thing at a time
                1. No multi-tasking
    2. Web APIs
        1. APIs provided to the engine (not part of JS itself)
            1. DOM
            2. Timers
            3. Fetch API
            4. Geolocation API
            5. ...
    3. Callback Queue
        1. Ready-to-be-executed callback functions (coming from events)
    4. **Event-Loop**
        1. Whenever the callstack is empty, the event loop takes callbacks from the callback queue and puts them in the callstack (for execution)
        2. It is the essential piece that makes async behaviour in JS possible
            1. Reason for non-blocking concurrency model in JS
                1. Concurrency model: How JS handles multiple tasks happening at the same time

How Asynchronous JavaScript Works Behind the Scenes

1. How can **asynchronous** code be executed in a **non-blocking way**, if there is **only one thread** of execution in the engine?
2. Example:

```
el = document.querySelector('img');
el.src = 'dog.jpg'; // starts to load the image in the bg
el.addEventListener('load', () => {
    el.classList.add('fadeIn');
});
fetch('https://someurl.com/api')
    .then(res => console.log(res));
```

    1. Background?
        1. It is in the Web APIs environment (of the browser) where the asynchronous tasks related to the DOM will run
            1. Same for: timers, ajax calls, ...
        2. If it were synchronous, it would be doing it in the call-stack blocking the rest of the code
            1. Solution: asyn tasks run in the Web API environment (not in the main thread of execution)
        3. The callback for the 'load' event is registered in the Web APIs environment exactly where the image is loading.
            1. Callback stays there until 'load' event is emitted.
        4. A new execution context is added for `addEventListener()`, executed, and removed from the call-stack.
        5. We make an AJAX call
            1. Async call happens in the Web API environment
            2. `fetch` is followed by `then` method
            3. `then` registers a callback in the Web API environment associated with Promise that is fetching the data in the API
        6. If `load` event is emitted:
            1. Callback is put in the callback queue (a todo list that the callstack has to complete)
                1. Callback queue: Ordered list containing all the callback functions to be executed
                2. If there are other callbacks already in the queue, the new callback will go to the end of the queue
                    1. Problem: If there is a timeout function. If other callbacks are waiting, the callback will run after the other callbacks are run
                        1. Guarantee: callback will not get executed before the timeout
        7. DOM events are not async but they still use the callback queue
3. Event-Loop
    1. It looks into the call-stack and determines whether it is empty or not (except for Global context)
    2. If it is empty
        1. It will take the first callback from the callback queue and put it in the call-stack to be executed
            1. It is an event-loop tick (each time a callback is taken and placed in the call-stack)
    3. It does the orchestration of the entire JS runtime
        1. Runtime manages async behaviour.
4. Callbacks registered with Promises do not go into the callback queue
    1. They have a special queue for themselves
        1. **Microtasks queue**
            1. It has priority over callback queue
            2. At the end of an event-loop tick (after running a callback), it will check if there are any **microtasks** in the microtasks queue. (there are other microtasks)
                1. If there are, it will run all of them, before fetching another callback from the callback queue
                    1. event-loop puts the microtasks in the callstack (even if there are pending callbacks in the callback queue)
                    2. If a microtask adds another microtask, it will be executed before any callbacks in the callback queue
                        1. It has the potential to starve the callback queue
                            1. Usually not a problem in reality

The Event Loop in Practice

1. Code:

```
// Event Loop
console.log('Test start');
setTimeout(() => console.log('0 sec timer'), 0); // Finishes immediately. Callback will be put immediately
Promise.resolve('Resolved promise 1').then(res => console.log(res)); // To build a promise that is immediately resolved. The resolve
Promise.resolve('Resolved promise 2').then(res => {
  for (let i = 0; i < 100_000_000; i++) {}
  console.log(res);
});
```

```
console.log('Test end');

// 1. Test start - global context
// 2. Test end
// 3. Resolved promise 1
// 4. Resolved promise 2
// 5. 0 sec timer
```

Building a Simple Promise

1. Promises are usually built to wrap old callback-based functions into promises.
    1. Promisifying
2. Code:

```
// Promisifying setTimeout
const wait = seconds =>
  new Promise(resolve => setTimeout(resolve, seconds * 1000));
// no resolved values are needed
// It is impossible for timer to fail

wait(2).then(() => console.log('I waited for 2 seconds')); // Any code that needs to be executed after 2 seconds

wait(2)
  .then(() => {
    console.log('I waited for 2 seconds');
    return wait(1);
  })
  .then(() => console.log('I waited for 1 second'));

// setTimeout(() => {
//   console.log('1 second passed');
//   setTimeout(() => {
//     console.log('2 seconds passed');
//     setTimeout(() => {
//       console.log('3 seconds passed');
//       setTimeout(() => {
//         console.log('4 seconds passed');
//       }, 1000);
//     }, 1000);
//   }, 1000);
// }, 1000);
wait(1)
  .then(() => {
    console.log('1 second passed');
    return wait(1);
  })
  .then(() => {
    console.log('2 second passed');
    return wait(1);
  })
  .then(() => {
    console.log('3 second passed');
    return wait(1);
  })
  .then(() => {
    console.log('4 second passed');
  });

Promise.resolve('abc').then(x => console.log(x));
Promise.reject('abc').catch(x => console.error(x)); // **(M)**
```

Promisifying the Geolocation API

1. Code:

```
// Promisifying Geolocation API
console.log('Getting position');

const getPosition = function () {
  return new Promise(function (resolve, reject) {
    // navigator.geolocation.getCurrentPosition(
    //   position => resolve(position),
    //   err => reject(err)
    // );
    navigator.geolocation.getCurrentPosition(resolve, reject); // similar to the previous code. `resolve` gets called with `position
  });
};

getPosition().then(pos => console.log(pos));

const whereAmI = function () {
  getPosition()
    .then(pos => {
      console.log(pos.coords);
      const { latitude: lat, longitude: lon } = pos.coords;

      return fetch(
        `https://api.geoapify.com/v1/geocode/reverse?lat=${lat}&lon=${lon}&apiKey=856413c61f024071aef052b0ae18d9da`
      );
    })
    .then(response => response.json())
    .then(data => {
      console.log(data);
      console.log(data.features[0].properties.country_code);

      const countryCode = data.features[0].properties.country_code;

      return getJSON(
        `https://restcountries.com/v3.1/alpha/${countryCode}`,
```

```
          'Country not found'
        );
      })
      .then(data => {
        renderCountry(data[0]);
      })
      .catch(err => console.error(err))
      .finally(() => (countriesContainer.style.opacity = 1));
};

btn.addEventListener('click', whereAmI);
```

1. Flat chain of promises

2. We can also promisify `XMLHttpRequest`

Coding Challenge #2

1. MUST DO!!!

Consuming Promises with Async/Await

1. ES2017 - Async/Await

2. Code:

```
// Async/Await
const whereAmIAsync = async function () {
  // async function is a function that runs in the background while performing the code inside of it
  // When the function is done, it will return a `Promise` instance

  // Reverse geocoding
  const pos = await getPosition();
  const { latitude: lat, longitude: lon } = pos.coords;
  const resGeo = await fetch(
    `https://api.geoapify.com/v1/geocode/reverse?lat=${lat}&lon=${lon}&apiKey=856413c61f024071aef052b0ae18d9da`
  );
  const dataGeo = await resGeo.json();

  const res = await fetch(
    `https://restcountries.com/v3.1/name/${dataGeo.country}`
  ); // takes a `Promise`. It will stop code at this point of the function until the `Promise` is fulfilled (until data is fetched i
  console.log(res);

  // This is similar to
  // fetch(`https://restcountries.com/v3.1/name/${country}`).then(res => console.log(res));

  const data = await res.json();
  console.log(data);
  renderCountry(data[0]);
};
whereAmIAsync(); // This call will load the function in the background without blocking the main thread. The line immediately moves
console.log('FIRST'); // This is displayed first

// Async/Await is a syntactic sugar over `Promise`s
```

Error Handling With try...catch

1. Code:

```
// Async/Await
const whereAmIAsync = async function () {
  // async function is a function that runs in the background while performing the code inside of it
  // When the function is done, it will return a `Promise` instance

  try {
    // Reverse geocoding
    const pos = await getPosition();
    const { latitude: lat, longitude: lon } = pos.coords;
    const resGeo = await fetch(
      `https://api.geoapify.com/v1/geocode/reverse?lat=${lat}&lon=${lon}&apiKey=856413c61f024071aef052b0ae18d9da`
    );
    if (!resGeo.ok) throw new Error('Problem getting location data');
    const dataGeo = await resGeo.json();

    // Country data
    const res = await fetch(
      `https://restcountries.com/v3.1/name/${dataGeo.country}`
    ); // takes a `Promise`. It will stop code at this point of the function until the `Promise` is fulfilled (until data is fetched
    if (!res.ok) throw new Error('Problem getting country');
    console.log(res);

    // This is similar to
    // fetch(`https://restcountries.com/v3.1/name/${country}`).then(res => console.log(res));

    const data = await res.json();
    console.log(data);
    renderCountry(data[0]);
  } catch (err) {
    console.error(`${err} 💥`);
    renderError(`Something went wrong 💥 ${err.message}`);
  }
};
whereAmIAsync(); // This call will load the function in the background without blocking the main thread. The line immediately moves
console.log('FIRST'); // This is displayed first

// Async/Await is a syntactic sugar over `Promise`s

// try/catch
```

```
// try {
//   let y = 1;
//   const x = 2;
//   x = 3;
// } catch (err) {
//   alert(err.message);
// }
```

Returning Values from Async Functions

1. What is async function and how does it work?
2. Code:

```
// Async/Await
const whereAmIAsync = async function () {
  // async function is a function that runs in the background while performing the code inside of it
  // When the function is done, it will return a `Promise` instance

  try {
    // Reverse geocoding
    const pos = await getPosition();
    const { latitude: lat, longitude: lon } = pos.coords;
    const resGeo = await fetch(
      `https://api.geoapify.com/v1/geocode/reverse?lat=${lat}&lon=${lon}&apiKey=856413c61f024071aef052b0ae18d9da`
    );
    console.log(resGeo);
    if (!resGeo.ok) throw new Error('Problem getting location data');
    const dataGeo = await resGeo.json();
    console.log('DATA-GEO:', dataGeo);

    // Country data
    const res = await fetch(
      `https://restcountries.com/v3.1/name/${dataGeo.features[0].properties.country}`
    ); // takes a `Promise`. It will stop code at this point of the function until the `Promise` is fulfilled (until data is fetched
    if (!resGeo.ok) throw new Error('Problem getting country');
    console.log(res);

    // This is similar to
    // fetch(`https://restcountries.com/v3.1/name/${country}`).then(res => console.log(res));

    const data = await res.json();
    console.log(data);
    renderCountry(data[0]);

    return `You are in ${dataGeo.city}, ${dataGeo.country}`;
  } catch (err) {
    // `Promise` is still fulfilled and not rejected.
    console.error(`${err} 💥`);
    renderError(`Something went wrong 💥 ${err.message}`);

    // Reject promise returend from async function
    throw err; // propagates it down
  }
};
console.log('1: Will get location'); // Appears first
// const city = whereAmIAsync(); // This call will load the function in the background without blocking the main thread. The line im
// console.log('FIRST'); // This is displayed first
// console.log(city); // prints a `Promise`. The value returned from an `async` function is fulfilled value
whereAmIAsync()
  .then(city => console.log(`2: ${city}`))
  .catch(err => console.error(`2: ${err.message}`)) // resolved value is returned
  .finally(() => console.log('3: Finished getting location'));
// Appears second

// Async/Await is a syntactic sugar over `Promise`s

// try/catch

// try {
//   let y = 1;
//   const x = 2;
//   x = 3;
// } catch (err) {
//   alert(err.message);
// }

// Mixes Promises and async-await
// We can use async-await function

(async function () {
  try {
    const city = await whereAmIAsync();
    console.log(`2: ${city}`);
  } catch (err) {
    console.error(`2: ${err.message}`);
  } finally {
    console.log('3: Finished getting location');
  }
})(); // IIFE supports `async`

// Async functions can call other async functions
```

Running Promises in Prallel

1. Code: Combinator function

```
// Promises in Parallel
const get3Countries = async function (c1, c2, c3) {
  try {
    // The three calls run in sequence until the Promises are resolved.
```

```
    // Solution: We can run the promises in parallel
    // const [data1] = await getJSON(`https://restcountries.com/v3.1/name/${c1}`);
    // const [data2] = await getJSON(`https://restcountries.com/v3.1/name/${c2}`);
    // const [data3] = await getJSON(`https://restcountries.com/v3.1/name/${c3}`);

    const data = await Promise.all([
      getJSON(`https://restcountries.com/v3.1/name/${c1}`),
      getJSON(`https://restcountries.com/v3.1/name/${c2}`),
      getJSON(`https://restcountries.com/v3.1/name/${c3}`),
    ]); // Returns a new `Promise`. **(M)**
    // Short-circuits when one promise rejects (all other promises are rejected)
    // Use-case: When we have async tasks that do not depend on each other, then we must run them in parallel

    console.log(data);

    console.log(data.map(d => d[0].capital));
  } catch (err) {
    console.error(err);
  }
};

get3Countries('portugal', 'canada', 'tanzania');
```

Other Promise Combinators: race, allSettled and any

1. `Promise.race`
    1. Use-case: To prevent against never ending promises or long running promises.
        1. Example: If a user has very bad internet connection, then `fetch` might take a very long time. We can use a timeout `Promise` which automatically rejects when a certain time has passed.
2. Code:

```
// Promise.race - it receives an array of primises and returns a promise. It is settled as soon as one of the input promise settles

(async function () {
  try {
    const res = await Promise.race([
      getJSON('https://restcountries.com/v3.1/name/italy'),
      getJSON('https://restcountries.com/v3.1/name/egypt'),
      getJSON('https://restcountries.com/v3.1/name/mexico'),
    ]);
    console.log('RACE:', res[0]);
  } catch (err) {
    console.error();
  }
})();

const timeout = function (sec) {
  return new Promise(function (_, reject) {
    setTimeout(function () {
      reject(new Error('Request took too long!'));
    }, sec * 1000);
  });
};

Promise.race([
  getJSON('https://restcountries.com/v3.1/name/tanzania'),
  timeout(0.01),
])
  .then(res => console.log('RACE:', res[0]))
  .catch(err => console.error('RACE:', err));

// Promise.allSettled - ES2020 **(M)**
// It will take an array of `Promise`s and return an array of all settled `Promise`s. If they are rejected or not
// Promise.all short-circuits if one promise rejects. Promise.allSettled never short circuits

Promise.allSettled([
  Promise.resolve('Success'),
  Promise.reject('Error'),
  Promise.resolve('Another success'),
]).then(res => console.log(res));

Promise.all([
  Promise.resolve('Success'),
  Promise.reject('Error'),
  Promise.resolve('Another success'),
])
  .then(res => console.log(res))
  .catch(err => console.error(err));

// Promise.any - ES2021 **(M)**. It takes an array of promises and returns the first fulfilled promise. Rejected promises are ignore
Promise.any([
  Promise.resolve('Success'),
  Promise.reject('Error'),
  Promise.resolve('Another success'),
])
  .then(res => console.log(res))
  .catch(err => console.error(err));
```

Coding Challenge #3

1. MUST DO:

Section 17: Modern JavaScript Development: Modules, Tooling, and Functional

Section Intro

1. It is about the development process

1. Modern build tools that every developer uses
2. Modern ecosystem created around JS

Section Roadmap

An Overview of Modern JavaScript Development

1. General Overview of how we write JS today

Modern JavaScript Development

1. We divide our project into multiple modules
    1. The modules can share data between them and make code more organized and maintainable
    2. Advantages:
        1. We can include 3rd-party modules (packages) into our own code
            1. There are thousands of open-source modules
                1. The developers share them in `npm` repository
                    1. Examples:
                        1. React
                        2. jQuery
                        3. Leaflet
                        4. etc.
                2. We can use the modules (packages) for free in our own code
2. `npm` - Node Package Manager
    1. It was initially developed with Node.js and for Node.js
        1. It is however used for all kinds of packages in modern JS development
    2. We need to install `npm` software
    3. `npm` - It is repository in which our packages live, and program we use on our computers to install and manage the packages.
3. Build Process
    1. One big JS bundle is build - final file we deploy to production
        1. Production: Application used by real-users in the real-world
    2. It can be really complex
    3. Steps:
        1. Bundling - Join all modules into one file
            1. Eliminates un-used code
            2. Compresses code
            3. Why?
                1. Older browsers do not support modules (cannot execute)
                2. Better for performance:
                    1. Fewer files are sent to the browser
                    2. Compressed files are smaller in size
        2. Transpiling/Polyfilling - Convert modern JavaScript back to ES5
            1. Why?
                1. Older browsers can understand our code without breaking
            2. How?
                1. Babel
    4. We use special tool to implement build process
        1. JavaScript bunders: They take raw code and transform it into JS bundle
            1. Webpack
                1. More popular
                2. Hard to set it up
                    1. We need to configure many things manually to make it work properly
            2. Parcel
                1. Zero config bundler (works out of the box)
                    1. We don't have to write any setup code
    5. The tools are in `npm`
        1. `npm` contains **development tools** that help build our application (e.g. live-server, Parcel, Babel, etc.)

An Overview of Modules in JavaScript

An Overview of Modules

1. Module: Reusable piece of code that **encapsulates** implementation details (of a certain part of our project)

    1. It is usually a standalone file, but it doesn't have to be

    2. It contains code and `imports` and `exports`

    ```
    import { rand } from './math.js';
    const diceP1 = rand(1, 6, 2);
    const diceP2 = rand(1, 6, 2);
    const scores = { diceP1, diceP2 };
    export { scores };
    ```

        1. `export` - We can export out values form a module
            1. Simple values
            2. Functions
            3. Exported functions are public API
            4. Public API
                1. Consumed by importing values
                    1. The imported module is called dependency of the importing module

    3. When a code-base grows bigger and bigger, there are many advantages of using modules

        1. Modules make it very easy to compose software
            1. **Compose Software:** They are small building blocks we can put together to build complex applications
                1. Example: Digital camera - many modules are combined

1. Each module can be developed in complete isolation
2. Each engineer can work independently on their own module without understanding about other modules or the camera as a whole

2. **Isolate components**: Modules can be developed in isolation without thinking about the entire codebase
    1. Developer doesn't need to understand all of it
        1. Makes it easy to collaborate with larger teams
3. **Abstract code:** Implement low-level code in modules and import these abstractions into other modules
    1. Example: Screen module doesn't care about low-level details of the controller module
        1. It can import the controller without knowing how it works and use it to control other parts of the camera
4. **Organized code:** Modules naturally lead to a more organized codebase
    1. Makes it easier to understand and maintain
5. **Reuse code:** Modules allow us to easily reuse the same code, even across multiple projects
    1. Copy a module to a new project if we want to reuse
        1. Example: Screen can be re-used in multiple cameras because it is nicely abstracted out

Native Javascript (ES6) Modules

1. JavaScript has built-in native module system (ES6)
    1. We had modules before ES6 but we had to implement them ourselves or use external libraries
2. ES6 Modules
    1. Modules stored in files
        1. **Exactly one module per file**
3. ES6 Module vs Script
    1. ES6 Module
        1. Top-level variables: Scoped to module (by default)
            1. The only way an outside module can access the value is by exporting the value
                1. If we do not export, nobody from outside has access
        2. Default mode: Strict mode (by default)
            1. No need to explicity declare `"use strict";`
        3. Top-level `this`: `undefined`
        4. Imports and exports: Yes
            1. Need to happen at top-level (outside of any function or if block)
            2. Imports are hoisted
                1. No matter where we import, it is as though we are importing on the top
                    1. Import is first thing that happens in a module
        5. HTML linking: `<script type="module">` **(M)**
        6. File downloading: Asynchronous (by default)
            1. For modules loaded by HTML
            2. For modules loaded by `importing` one module into another using `import` syntax
    2. Script
        1. Top-level variables: Global
            1. Problems: Global namespace polution
                1. Multiple scripts might try to declare variables with the same name (variables collide)
                    1. Solution: private variables (ES6 modules)
        2. Default mode: "Sloppy" mode
        3. Top-level `this`: `window`
        4. Imports and exports: No (not possible)
        5. File downloading: Synchronous
            1. Blocking, synchronous way, unless we use `async` **(M)** or `defer` **(M)** attributes on the `script` tag

How ES6 Modules Are Imported

1. Example: index.js

```
import { rand } from './math.js';
import { showDice } from './dom.js';

const dice = rand(1, 6, 2);
showDice(dice);
```

    1. Step 1: Parsing index.js
        1. Reads without execution
            1. Imports are hoisted at this point in time
                1. Importing modules happens before execution of code in the main module
                    1. Modules are **imported synchronously**
                        1. Only after all imported modules are downloaded and executed, the main `index.js` will be executed.
                            1. Possible because of top-level ("static") imports, which make **imports kown before execution**
                                1. Engine can know all the imports and exports during the parsing phase (while code is read before being executed)
                                    1. If import is inside a function, then the function needs to be executed for the import to happen
            2. Why load modules in a synchronous way?
                1. The easiest way to bundling and dead-code elimination (deleting code that is not necessary)
                    1. If using 100s of modules, and if we need a small piece
                        1. If we know all the dependencies between modules before execution, bundlers (webpack, parcel) can join multiple modules together and eliminate dead-code
    2. Step 2: After parsing process has figured out which modules need to be imported, Modules are downloaded from the server
        1. Downloading happens in an asynchronous way
            1. Only import operation is synchronous
        2. Once the module arrives, it is parsed
    3. Step 3: The module exports are linked to imports (in index.js)
        1. export in math.js is connected to imports to math.js in index.js
            1. It is a live connection

        1. Exported values are not copied to imports
            1. Import is a reference to the exported value
                1. **If value changes in the exporting value, the value also changes in the importing module** (Unique to ES6 modules)
        4. Step 4: Code in the imported modules is executed

Exporting and Importing in ES6 Modules

1. 17-Modern-JS-Modules-Tooling
    1. Importing a module without importing any value
        1. Module names: In camel-case
    2. There are two types of `exports`
        1. Named exports
            1. We just have to put `export` in front of anything we want to export
        2. Default exports
            1. Used if we want to use to export only one thing from a module
            2. It is the preferred style to export one default thing and import it
2. Code:

```
// Importing module
// import { addToCart, totalPrice as price, tq } from './shoppingCart.js';
// extension is optional
// Needs specifying `type="module"` in HTML
// curly braces are required for named imports

// import * as ShoppingCart from './shoppingCart.js'; // an object containing everything
// A namespace is created

import add, { cart } from './shoppingCart.js'; // any name can be used for default export
// No need for {} for default exports

// import add, { addToCart, totalPrice as price, tq } from './shoppingCart.js'; // Mixing is not usually done

// Not recommended to import same module twice

console.log('Importing module');

// addToCart('bread', 5);
// console.log(price, tq);

// The following is similar to an object created from a class exporting a public API
// ShoppingCart.addToCart('bread', 5);
// console.log(ShoppingCart.totalPrice, ShoppingCart.tq);

add('pizza', 2);
add('pizza', 5);
add('pizza', 4);

console.log(cart);
// Import is not just a copy but a live connection because updates are reflected after the `import` statement
```

Top-Level await (ES2022)

1. We can use `await` keyword outside of `async` functions (at-least in modules)
    1. It only works in modules
2. Fake data:
    1. https://jsonplaceholder.typicode.com/
3. Top-level await - blocks the execution of the entire module
    1. It can be harmful
    2. If a module imports another module that has a top-level `await`, then the importing module will wait for the imported module to finish the blocking code
        1. Example:

```
// Blocking code
console.log('Start fetching users');
await fetch('https://jsonplaceholder.typicode.com/users');
console.log('Finish fetching users');
// Throttle request using slow 3G

// Then the importing module is executed
```

4. Code:

```
// Top-level Await
console.log('Start fetching');
const res = await fetch('https://jsonplaceholder.typicode.com/posts');
const data = await res.json();
console.log(data);
console.log('Something'); // printed after the data is fetched. Printed at the end

const getLastPost = async function () {
  const res = await fetch('https://jsonplaceholder.typicode.com/posts');
  const data = await res.json();
  console.log(data);
  return { title: data.at(-1).title, text: data.at(-1).body };
};

const lastPost = getLastPost();
console.log(lastPost);
```

```
// Not very clean
lastPost.then(last => console.log(last));

const lastPost2 = await getLastPost();
console.log(lastPost2);
```

The Module Pattern

1. Used in the past (we still see it around)
2. Main goal of module pattern
    1. Have private data
    2. Expose public API
3. A solution: Using functions

```
// Module pattern
const ShoppingCart2 = (function () {
  // It's purpose is to define a new scope and return data just once
  const cart = [];
  const shippingCost = 10;
  const totalPrice = 237;
  const totalQuantity = 23;

  const addToCart = function (product, quantity) {
    cart.push({ product, quantity });
    console.log(
      `${quantity} ${product} added to cart (Shipping cost is ${shippingCost})`
    );
    // `shippingCost` is private
  };

  const orderStock = function (product, quantity) {
    cart.push({ product, quantity });
    console.log(`${quantity} ${product} ordered from supplier`);
  };

  return {
    addToCart,
    cart,
    totalPrice,
    totalQuantity,
  };
})();

ShoppingCart2.addToCart('apple', 4);
ShoppingCart2.addToCart('pizza', 2);

console.log(ShoppingCart2);
console.log(ShoppingCart2.shippingCost); // Not accessible

// Console is in global scope and this is a module
// Works because of closures - Allows a function to have access to all variables and functions that were present at its birthplace
// `addToCart` - never loses connection to it's birthplace (contains all variables at that point)
```

4. Problems:
    1. If we want one module per file, we need to construct different scripts and link them in the HTML file
        1. Problems:
            1. We need to be careful with the order we declare in the HTML
            2. We will have variables in the global scope
            3. We cannot bundle them together using module-bundler
    2. Solution: Native modules

CommonJS Modules

1. These module systems are not native to JS so they depended on an external implementations.
    1. Examples:
        1. AMD modules
        2. Common.js modules
2. common.js - used in Node.js (only recently ES6 modules have been implemented in Node.js)
    1. Almost all modules in the NPM repo still use `common.js` module system
        1. NPM was originally intended for Node
            1. We are stuck with `common.js`
3. Code:

```
// Export
export.addToCart = function (product, quantity) {
  cart.push({ product, quantity });
  console.log(
    `${quantity} ${product} added to cart (Shipping cost is ${shippingCost})`
  );
  // `shippingCost` is private
}; // Doesn't work in browser

// Import
const { addToCart } = require('./shoppingCart.js'); // works in Node.js
```

    1. There are many more module systems

A Brief Introduction to the Command Line

1. All build-tools available in NPM work only in command-line

Introduction to NPM

1. NPM - Node Package Manager
    1. It it a software
    2. It is a repository
2. Why do we need NPM?
    1. We were including external libraries directly inside HTML code
        1. Problems:
            1. Exposes global variables
                1. Problems in large projects
                    1. HTML loading JS is messy
                    2. If we are including local files, we need to manually download and update HTML
                    3. There is no single package with all the dependencies we need
3. Terminal:

```
npm -v # **(M)**. It should be > 6. Use LTS

# For each project

npm init

# Hit enter for defaults
```

    1. `package.json`
        1. Stores the entire configuration of our project
    2. Install `leaflet` library using `npm`

```
npm install leaflet
npm i leaflet
```

        1. We cannot direclty import the `common.js` module into our code (we can do it using a module bundler)
4. Lodash: A collection of useful functions for arrays, object, dates, etc.
    1. We want native module version (not `lodash`)

```
npm i lodash-es
```

5. Code:

```
import cloneDeep from './node_modules/lodash-es/cloneDeep.js'; // Full path is required because we don't have module-bundler

const state = {
  cart: [
    {
      product: 'bread',
      qantity: 5,
    },
    {
      product: 'pizza',
      quantity: 5,
    },
  ],
  user: {
    loggedIn: true,
  },
};

const stateClone = Object.assign({}, state);
console.log(stateClone);

const stateDeepClone = cloneDeep(state);
console.log(stateDeepClone);

state.user.loggedIn = false;
console.log(stateClone); // modified original object
```

    1. Never include `node_modules` folder if we want to share the project
        1. Reason:
            1. It is huge
                1. Slows transfer
    2. If we don't have `node_modules`
        1. We can install dependencies again

```
npm install # **(M)**
```

Bundling with Parcel and NPM Scripts

1. Parcel:
    1. A build tool which is on npm

```
npm install parcel --save-dev # **(M)**
npm install parcel@1.12.4 --save-dev
```

        1. Dev dependency: It is only a tool to build the application

1. It is not a dependency used in the application (used only for development)

2. How to run Parcel?

    1. `npx`

        ```
        npx parcel index.html # **(M)**

        # We need to pass entry point html
        # The file contains the js script (script.js) that we want to bundle up
        # parcel starts a new development server automatically
        ```

        1. Parcel converts everything to scripts (not modules)
            1. To support older browsers
        2. `dist`
            1. Folder that is sent to the final users (for production)
                1. `index.html` contains bundled script
        3. Hot module reload maintains state (if we reload the page)
            1. The variables are persisted on page reloads

    2. `npm` scripts

        1. Another way of running locally installed packages in the command-line

        2. They allow us to automate repetetive tasks

            1. package.json

                ```
                "scripts": {
                    "start": "parcel index.html"
                },
                ```

                1. `start` - name of the script
                2. `parcel index.html` - we cannot run this on the commandline

            3. Running the script:

                ```
                npm start
                ```

            4. Building final bundled (compressed, dead-code eliminated, ...)

                ```
                // "main": "clean.js",
                "source": "index.html",
                "scripts": {
                    ...
                    "build": "parcel build"
                }
                ```

            5. Global installation:

                ```
                npm i parcel -g
                ```

                1. We can use it directly in the commandline without npm scripts
                    1. Local installation is recommended
                        1. We can choose the version and it is part of the application

Configuring Babel and Polyfilling

1. Babel - transpiling modern JS to ES5

    1. We will continue to do this in the future

        1. Why?
            1. Many people are stuck with old browsers (Windows XP, Windows 7 with IE that cannot be upgraded)
                1. We want to make the application work for everyone

    2. Parcel automatically uses Babel to transpile code

        1. We can configure Babel
            1. Defining what browsers should be suppported
                1. Parcel makes default decisions

    3. babeljs.io

        1. Plugins
            1. Babel works with plugins and presets that can both be configured
            2. Plugin:
                1. Specific JS feature that we want to transpile
                    1. Example: Arrow functions
        2. Preset:
            1. A bunch of plugins bundled together
            2. Parcel uses `preset-env` preset (`@babel/preset-env`)
                1. Automatically selects which JS features should be compiled based on browser support
                    1. Happens automatically and out of the box
                    2. Only Browsers with 0.25% market share (say) will not be supported by this preset
                2. Preset includes only the final features that are in the language (features that passed all 4 stages of ECMA process)

    4. Polyfilling:

        ```
        npm i core-js
        ```

        1. `find` and `Promise` will not be removed, but polyfilling will give versions of the functions not available in ES6

2. Code:

```
import 'core-js/stable'; // **(M)**
```

3. We can cherry-pick features we need (reduces bundle-size)

```
import 'core-js/stable/array/find';
import 'core-js/stable/promise';
```

4. A feature not polyfilled: For polyfilling `async` functions

```
npm i regenerator-runtime
```

```
import 'regenerator-runtime/runtime';
```

2. Code:

```
// Only Parcel understands the following code
// It does not make it into the final bundle
// Hot module reloading - whenever we change a module, it will trigger a rebuild, and the modified bundle automatically gets injecte
// Maintains state. Example: We don't have to login again
if (module.hot) {
  module.hot.accept();
}

class Person {
  #greeting = 'Hey';
  constructor(name) {
    this.name = name;
    console.log(`${this.#greeting}, ${this.name}`);
  }
}
const jonas = new Person('Jonas');

console.log('Jonas' ?? null);

console.log(cart.find(el => el.quantity >= 2)); // ES6 `find` is not converted
Promise.resolve('TEST').then(x => console.log(x)); // ES6 `Promise` is also not converted.
// Babel transpiles only ES6 syntax (arrow functions, classes, const, spread operator, ...). Things having an equivalent way of writ
// Babel does not convert real features added to the language (`find`, `Promise`, ...). Additions cannot be transpiled.
// Solution: Polyfilling
// Babel stopped doing polyfilling. We need to import a library. `core-js/stable`

import 'core-js/stable';
import 'regenerator-runtime/runtime';
```

Review: Writing Clean and Modern JavaScript

Review: Modern and Clean Code

1. Principles:
    1. Readable code:
        1. Write code so that **others** can understand it
        2. Write code so that **you** can understand it in 1 year
        3. Avoid too "clever" and overcomplicated solutions
            1. Might make the code very confusing and unreadable
                1. Write most straightforward solutions
        4. Use descriptive variable names: **what they contain**
        5. Use descriptive function names: **what they do**
    2. General
        1. Use DRY principle (refactor your code)
        2. Don't pollute global namespace, encapsulate instead (functions, classes, modules)
        3. Don't use var
            1. `const` always, and if required `let`
        4. Use strong type checks (=== and !==)
            1. `==` does not perform type-checks
    3. Functions:
        1. Generally, functions should do **only one thing**
            1. It must do it really well.
        2. Don't use more than 3 function parameters
            1. If a function does only one thing, it may not usually need more than 3 parameters.
        3. Use default parameters whenever possible
        4. Generally, return the same data type as received
        5. Use arrow functions when they make code more readable
            1. They may make code unreadable
                1. Use them if they make it more readable
                    1. Example: Callback methods of array functions
    4. OOP
        1. Use ES6 classes
        2. Encapsulate data and **don't mutate** it from outside the class
            1. Implement public API that can mutate the data as we want
        3. Implement method chaining
            1. Makes methods easier to use (for me and other developers)
        4. Do **not** use arrow functions as methods (in regular objects)
            1. We will not get access to the `this` keyword
    5. Avoid Nested Code
        1. Use early `return` (guard clauses)
        2. Use ternary (conditional) or logical operators instead of `if`
            1. ternary operator does not instantiate a code-block
        3. Use multiple `if` instead of `if/else-if`
            1. Makes it more readable
        4. Avoid `for` (and `for-of`) loops (to avoid nested code), use array methods instead
        5. Avoid callback-based asynchronous APIs

6. Asynchronous code
    1. Consume promises with `async/await` for best readability
        1. `then` and `catch` require callback functions (more nested code)
    2. Whenever possible, run promises in **parallel** (`Promise.all` **(M)**)
    3. Handle errors and promise rejections

Let's Fix Some Bad Code: Part 1

1. MUST DO!

Declarative and Functional JavaScript Principles

Imperative vs Declarative Code

1. Two fundamentally different ways of writing coe (paradigms)
    1. Imperative
        1. Programmer explains "HOW to do things"
        2. We explain the computer every single step it has to follow to achieve a result
            1. Example: Baking a cake
                1. Steps-by-step recipe of a cake (every single step)
            2. Example: Double `arr` array

```
const arr = [2, 4, 6, 8];
const doubled = [];
for (let i = 0; i < arr.length; i++)
    doubled[i] = arr[i] * 2;
```

    2. Declarative
        1. Programmer tells "WHAT to do"
        2. We simply describe the way the computer should achieve the result
        3. The HOW (step-by-step instructions) gets abstracted away
            1. Example: Description of a cake (to a person)
                1. Person comes up the step by step process on their own
            2. Example: Doubling the array items

```
const arr = [2, 4, 6, 8];
const doubled = arr.map(n => n * 2);
```

                1. We are telling it what to do
            3. Advantages:
                1. The details are abstracted away

Functional Programming Principles

1. Functional Programming
    1. **Declarative** programming paradigm
    2. Based on the idea of writing software by combining many **pure functions**, avoiding **side effects** and **mutating** data
2. It is the modern way of writing code in JS world
3. **Side effect**:
    1. Modification (mutation) of any data outside of the function (mutating external variables, logging to console, writing to DOM, etc.)
4. **Pure function**:
    1. Function without side effects. Does not depend on external variables. **Given the same inputs, always returns the same outputs**
5. **Immutability**:
    1. State (data) is **never** modified! instead, state is **copied** and the copy is mutated and returned.
        1. Original state is never touched.
        2. Helps us keep track of how the data flows in our entire application
            1. Better code
            2. Less bugs
            3. More readable code
6. Functional programming is very difficult to implement
7. Libraries are built around the functional programming principles
    1. React
        1. State is completely immutable
    2. Redux
8. Functional Programming Techniques
    1. Try to avoid data mutations
    2. Use built-in methods that don't produce side effects
    3. Do data transformations with methods such as `.map()`, `.filter()`, `.reduce()`
    4. Try to avoid side effects in functions: this is of course not always possible!
        1. Not always necessary
9. Declarative Syntax: (the following ways makes the code more declarative)
    1. Use array and object destructuring
    2. Use the spread operator (...)
    3. Use the ternary (conditional) operator
    4. Use template literals

Let's Fix Some Bad Code: Part 2

1. MUST DO!

Section 18: Forkify App: Building a Modern Application

Section Into

1. Beautiful, modern, advanced application
2. We will use:
    1. Classes
    2. Modules
    3. Async/Await
    4. Parcel
3. Flow charts & Project architecture
    1. For writing application by ourselves

Section Roadmap

Project Overview and Planning (I)

1. Final project: http://forkify-v2.nelify.app
    1. Searching for recipes

Project Planning

1. User Stories
    1. User story: Description of the application's functionality from the user's perspective
        1. When we write user stories, we need to put ourselves in user's shoes
        2. When we put all the user stories together, we will get a clear picture of how the application is going to work
            1. Application's features will be implemented based on the features
    2. Common format: As a [type of user], I want [an action] so that [a benefit]
2. Features
3. Flowchart
4. Architecture
5. Development

User Stories

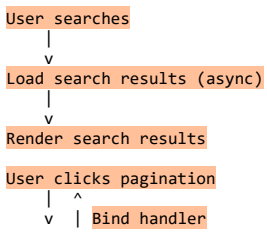
1. As a user, I want to **search for recipes**, so that I can find new ideas for meals
2. As a user, I want to be able to **update the number of servings**, so that I can cook a meal for different number of people
3. As a user, I want to **bookmark recipes**, so that I can review them later
4. As a user, I want to be able to **define my own recipes**, so that I have them all organized in the same app
5. As a user, I want to be able to **see my bookmarks and own recipes when I leave the app and come back later**, so that I can close the app safely after cooking

Features

1. Search for recipes ->
    1. Feature 1: Search functionality: input field to send request to API with searched keywords
    2. Feature 2: Display results with pagination
    3. Feature 3: Display recipe with cooking time, servings and ingredients
2. Update the number of servings ->
    1. Feature 4: Change servings functionality: update all ingredients according to current number of servings
3. Bookmark recipes ->
    1. Feature 5: Bookmarking functionality: display list of all bookmarked recipes
4. Define my own recipes ->
    1. Feature 6: User can upload own recipes
    2. Feature 7: User recipes will automatically be bookmarked
    3. Feature 8: User can only see their own recipes, not recipes from other users
        1. Associate recipes with an API key
        2. We can also use user accounts
5. See my bookmarks and own recipes when I leave the app and come back later ->
    1. Store bookmark data in the browser using local storage
    2. On page load, read saved bookmarks from local storage and display

Flowchart (Part 1)

1. Features
    1. Search functionality: API search request
    2. Results with pagination
    3. Display recipe
    4. ...
2. Flowchart:



```
Render pagination buttons (re-render each time)
    | Bind handler
    v
User selects recipe            Page loads with recipe ID
    |                                       |
    v                                       |
Load recipe    <----------------+
    |
    v
Render recipe
```

Latest Code Updates (Parcel v2 and more)

1. If there are issues with Parcel, refer to updates-and-fixes
    1. Previously: `controller.js` in `dist`
    2. Now: `index.js`
    3. Previously: `Fractional`
    4. Now: `Fracty`

Loading a Recipe from API

1. 18-forkify
    1. src
        1. img
        2. js
        3. sass
            1. A better way of writing css
                1. Makes it easier to write css in a large scale application
                    1. Parcel converts sass into css (because browsers cannot understand sass)
    2. index.html
    3. Flowchart & architecture diagram
2. Package:

    ```
    npm init

    package name: forkify
    description: Recipe application
    ...
    author: <author>
    ...
    ```

3. Open `package.json`
    1. Remove: `"main": "index.js"` (for Parcel v2)
    2. scripts:

        ```
        "start": "parcel index.html",
        "build": "parcel build"
        ```

4. Install parcel:

    ```
    npm i parcel -D # Parcel 2

    # npm i parcel@next -D # beta version
    # npm i parcel@2 -D
    ```

5. Run:

    ```
    npm start

    # npm run start

    # If error
    npm install # installs sass

    # If more errors
    npm i sass@1.26.10 -D

    # Run
    npm start
    ```

6. `dist`
    1. All files are at this level
        1. Actual css
        2. Images
        3. js
        4. html
    2. If we reference sass files, Parcel will automatically install sass

        ```
        <link rel="stylesheet" href="src/sass/main.scss" />
        ```

    3. If using Parcel v2, replace `defer` with `type="module"`

        ```
        <script type="module" src="src/js/controller.js"></script>
        ```

    4. All we see in browser is coming from `dist` folder
        1. Bundler packages raw source files into `dist` folder that can be shipped for browsers

        2. https://forkify-api.herokuapp.com/v2
            1. search queries
                1. 100 API requests/hour

    5. APIs:

        1. Get recipes
        2. Get recipe
            1. Search recipes: https://forkify-api.herokuapp.com/api/v2/recipes?search=pizza

7. Code:

```
// console.log('TEST');

const showRecipe = async function () {
  try {
    const res = await fetch(
      'https://forkify-api.herokuapp.com/api/v2/recipes/5ed6604591c37cdc054bc886'
    );
    const data = await res.json();

    if (!res.ok) throw new Error(`${data.message} (${res.status})`);

    console.log(res, data);

    let { recipe } = data.data;
    recipe = {
      id: recipe.id,
      title: recipe.title,
      publisher: recipe.publisher,
      sourceUrl: recipe.source_url,
      image: recipe.image_url,
      servings: recipe.servings,
      cookingTime: recipe.cooking_time,
      ingredients: recipe.ingredients,
    };

    console.log(recipe);
  } catch (err) {
    alert(err);
  }
};

showRecipe();
```

Rendering the Recipe

1. Installing polyfills

```
npm i core-js regenerator-runtime
```

2. Import them in js file:

```
import 'core-js/stable'; // for polyfilling everything else
import 'regenerator-runtime/runtime'; // for polyfilling async/await
```

3. Code:

```
// import icons from '../img/icons.svg'; // Parcel 1
import icons from 'url:../img/icons.svg'; // Parcel 2

// For polyfilling
import 'core-js/stable';
import 'regenerator-runtime/runtime';

console.log(icons); // path to the new icons file

const recipeContainer = document.querySelector('.recipe');

const timeout = function (s) {
  return new Promise(function (_, reject) {
    setTimeout(function () {
      reject(new Error(`Request took too long! Timeout after ${s} second`));
    }, s * 1000);
  });
};

// https://forkify-api.herokuapp.com/v2

///////////////////////////////////////
// console.log('TEST');

const renderSpinner = function (parentEl) {
  const markup = `
  <div class="spinner">
    <svg>
      <use href="${icons}#icon-loader"></use>
    </svg>
  </div>
  `;
  parentEl.innerHTML = '';
  parentEl.insertAdjacentHTML('afterbegin', markup);
};

const showRecipe = async function () {
  try {
    const id = window.location.hash.slice(1); // **(M)**
    console.log(id);

    if (!id) return;
```

```
// 1) Loading recipe
renderSpinner(recipeContainer);
const res = await fetch(
  `https://forkify-api.herokuapp.com/api/v2/recipes/${id}`
);
const data = await res.json();

if (!res.ok) throw new Error(`${data.message} (${res.status})`);

console.log(res, data);

let { recipe } = data.data;
recipe = {
  id: recipe.id,
  title: recipe.title,
  publisher: recipe.publisher,
  sourceUrl: recipe.source_url,
  image: recipe.image_url,
  servings: recipe.servings,
  cookingTime: recipe.cooking_time,
  ingredients: recipe.ingredients,
};

console.log(recipe);

// 2) Rendering recipe
const markup = `
<figure class="recipe__fig">
  <img src="${recipe.image}" alt="${recipe.title}" class="recipe__img" />
  <h1 class="recipe__title">
    <span>${recipe.title}</span>
  </h1>
</figure>

<div class="recipe__details">
  <div class="recipe__info">
    <svg class="recipe__info-icon">
      <use href="${icons}#icon-clock"></use>
    </svg>
    <span class="recipe__info-data recipe__info-data--minutes">${
      recipe.cookingTime
    }</span>
    <span class="recipe__info-text">minutes</span>
  </div>
  <div class="recipe__info">
    <svg class="recipe__info-icon">
      <use href="${icons}#icon-users"></use>
    </svg>
    <span class="recipe__info-data recipe__info-data--people">${
      recipe.servings
    }</span>
    <span class="recipe__info-text">servings</span>

    <div class="recipe__info-buttons">
      <button class="btn--tiny btn--increase-servings">
        <svg>
          <use href="${icons}#icon-minus-circle"></use>
        </svg>
      </button>
      <button class="btn--tiny btn--increase-servings">
        <svg>
          <use href="${icons}#icon-plus-circle"></use>
        </svg>
      </button>
    </div>
  </div>

  <div class="recipe__user-generated">
    <svg>
      <use href="${icons}#icon-user"></use>
    </svg>
  </div>
  <button class="btn--round">
    <svg class="">
      <use href="${icons}#icon-bookmark-fill"></use>
    </svg>
  </button>
</div>

<div class="recipe__ingredients">
  <h2 class="heading--2">Recipe ingredients</h2>
  <ul class="recipe__ingredient-list">
    ${recipe.ingredients
      .map(ing => {
        return `
      <li class="recipe__ingredient">
        <svg class="recipe__icon">
          <use href="${icons}#icon-check"></use>
        </svg>
        <div class="recipe__quantity">${ing.quantity}</div>
        <div class="recipe__description">
          <span class="recipe__unit">${ing.unit}</span>
          ${ing.description}
        </div>
      </li>
      `;
      })
      .join('')}
  </ul>
</div>

<div class="recipe__directions">
  <h2 class="heading--2">How to cook it</h2>
```

```
        <p class="recipe__directions-text">
          This recipe was carefully designed and tested by
          <span class="recipe__publisher">${
            recipe.publisher
          }</span>. Please check out
          directions at their website.
        </p>
        <a
          class="btn--small recipe__btn"
          href="${recipe.sourceUrl}"
          target="_blank"
        >
          <span>Directions</span>
          <svg class="search__icon">
            <use href="${icons}#icon-arrow-right"></use>
          </svg>
        </a>
      </div>
      `;
      recipeContainer.innerHTML = '';
      recipeContainer.insertAdjacentHTML('afterbegin', markup);
    } catch (err) {
      alert(err);
    }
  };

  showRecipe();

  ['hashchange', 'load'].forEach(ev => window.addEventListener(ev, showRecipe));
  // window.addEventListener('hashchange', showRecipe); // **(M)**
  // window.addEventListener('load', showRecipe); // **(M)**
```

Listening for load and hashchange Events

1. Code:

```
const showRecipe = async function () {
  try {
    const id = window.location.hash.slice(1); // **(M)**
    console.log(id);

    if (!id) return;
    ...
}

...

['hashchange', 'load'].forEach(ev => window.addEventListener(ev, showRecipe));
```

The MVC Architecture

Why worry About Architecture?

1. **Structure**: It gives the project a structure using which we can write the code
    1. Like a house, software needs a structure: the way we **organize our code**
    2. Structure:
        1. How we organize and divide the code into modules, classes, and functions
2. **Maintainability**: We need to think about the future. The project is never done. We need to change things in the future, and we need to maintain the project
    1. We need to be able to easily **change it in the future**
3. **Expandability**: The ability to easily add new feature
    1. Only possible with good structure and good overall architecture
4. **Perfect architecture**: Allows all the three aspects of **structure**, **maintainability**, and **expandability**
    1. Option 1: We can construct our own architecture (Mapty project)
        1. Works only for small projects
    2. Option 2: We can use a well-established architecture pattern like MVC, MVP (Model View Presenter), Flux, etc (**this project**)
    3. We can use a framework like React, Angular, Vue, Svelte, etc.
        1. The frameworks take care of architecture
        2. Good for large-scale applications

Components of Any Architecture

1. Business logic
    1. Code that **solves the actual business problem**
    2. Directly related to what business does and what it needs
    3. **Example**: sending messages (WhatsApp), storing transactions (Bank), calculating taxes (Budget Manager), etc.
2. State
    1. Essentially **stores all the data** about the application (running in the browser)
        1. Data about application's frontend
            1. Data from API
            2. Data input by user
            3. Page viewed by user
            4. ...
    2. Should be the "single source of truth"
    3. UI should be kept in sync with the state (it is one of the most difficult tasks)
        1. If some data changes in the state, UI should reflect that
        2. If something changes in the UI, the state needs to change
    4. State libraries exist
        1. Redux
        2. Mobx https://mobx.js.org/README.html
3. HTTP Library
    1. Responsible for making and receiving AJAX requests
        1. `fetch` function

    2. Optional but almost always necessary in real-world apps
4. Application logic (Router)
    1. Code that is only concerned about the **implementation of application itself**
        1. Technical aspects of the application (not directly related to the underlying business problem)
    2. Handles navigation and UI events
5. Presentation logic (UI layer)
    1. Code that is concerned about the **visible part** of the application
    2. Essentially displays application state
        1. Keeping in sync with **State**

Good Architecture

1. Separates the above components (instead of mixing them together in one big file)

The Model-View-Controller (MVC) Architecture

1. Model
    1. About Application's data
    2. Contains:
        1. Business Logic
        2. State
            1. Business logic manipulates the state
                1. Hence kept closely together
        3. HTTP Libary
            1. Gets some data from the web
            2. This is also about the data
2. View
    1. Presentation Logic
        1. It is the part that is interacting with the user
3. Controller
    1. Application Logic
        1. Bridge between model and views (which don't know about one another)
            1. M & V will live completely independent of one another, without knowing anything about the other

Goals of MVC Pattern

1. To separate business logic from view logic
    1. Makes it easier to develop the application
    2. Consequence: We need something to connect the parts
        1. Controller

Flow of Actions

1. User clicks
2. Controller handles the event
    1. Handling event is doing something in the application (application logic)
    2. It can trigger:
        1. Updating the UI
        2. Asking model for some data
    3. Implementation:
        1. Handles UI events and **dispatches tasks to model and view**
            1. It controls and orchestrates the entire action (the whole application)
3. Model makes ajax request to the web
    1. When the data arrives, the controller takes the data and sends it to the view
4. View takes the data and renders to the user
5. Observations:
    1. Only the controller imports and calls functions from model and the view (never the other way around)
        1. Model and view are completely isolated
            1. They don't know each other
            2. They don't know that controller exists

Model, View and Controller in Forkify (Recipe Display Only)

1. Controller
    1. Handling events is associated with the controller
    2. Once data arrives, controller receives it and sends it to the view
2. Model
    1. Loading recipe is associated with the model
        1. Controller calls a function that is in the model
        2. Model asynchronously gets the recipe data from the API
3. View
    1. Once view receives data, it renders it onto the screen

MVC Implementation (Recipe Display Only)

1. Module model.js

```
Module model.js
    export state
    recipe {}           <-+
    search {}             |
    bookmarks []          | recipe data
                          |
    export loadRecipe() --+
```

2. Module controller.js

```
Module controller.js
    controlRecipes()
    init()                              ^
        ^                               |
        |                               |
    (program starts)   |
                    (User clicks search result)
```

3. Class RecipeView

```
Class RecipeView
    _data
    _parentEl

    render()            --+
    _generateMarkup()   <-+
    renderSpinner()
    addHandlerRender()
```

1. `recipe` data (goes through controller)
2. `controlRecipes()` calls `loadRecipe()`
3. Steps:
    1. User clicks search result
    2. `controlRecepes()` handles the event
    3. Controller invokes `renderSpinner()` and `loadRecipe()` functions (to fetch recipes data)
    4. Model contains `state` variable
        1. `state` contains
            1. `recipe`s
            2. `search` results
            3. `bookmark`s
        2. Model stores that data when it arrives in `state` object
    5. Controller grabs recipe data from the `state` object
    6. Controller calls `render()` method on `RecipeView` with the data
    7. View renders the recipe in the UI

Refactoring for MVC

1. Why views are in multiple classes?
    1. View is much bigger and single file will be too big

Helpers and Configuration Files

1. Real-world applications have two modules that are independent of the rest of the architecture
    1. Module for project configuration
    2. Module for general helper functions (useful for the entire project)

Event Handlers in MVC: Publisher-Subscriber Pattern

Event Handling in MVC: Publisher-Subscriber Pattern

1. Module controller.js
    1. `controlRecipes()`
        1. View cannot call it directly (because it doesn't know anything about the Controller
            1. Solution: Publisher/Subscriber design pattern
                1. Publisher: Code that known when to react:
                    1. `addHandlerRender()` - it contains `addEventListener` function
                2. Subscriber: Code that wants to react
                    1. `controlRecipes()`
        2. How to subscribe?
            1. Subscribe to publisher by passing in the subscriber function
                1. Implementation:
                    1. When program loads `init()` function is called
                    2. `init()` function immediately calls `addHandlerRender()` function
                        1. Since controller imports both view and the model
                        2. It is called by passing `controlRecipes()` function as an argument
                            1. Connecting the functions
                    3. `addHandlerRender` listens for events (`addEventListener`), and uses `controlRecipes` as callback
                        1. As soon as a publisher publishes an event, the subscriber gets called immediately
            2. Example:

```
A() -> B()

  B
  |
  v
A(X) -> X()
  ^
  |
  C
```

                1. Function `A` has no control in the second scenario. It just executes what it receives
    2. init() <- Program starts
2. Class RecipeView
    1. addHandlerRender() <- User clicks search result
3. Handling events:
    1. Events should be **handled** in the **controller** (otherwise we would have application logic in the view)
    2. Events should be **listened for** in the **view** (otherwise we would need DOM elements in the controller)

Implementing Error and Success Messages

1. Displaying errors in the UI (View)

Implementing Search Results - Part 1

1. Search functionality
2. Views:
    1. We can have separate views for search panel & search results section
        1. We can keep each view focussed
        2. They are also in different locations
3. Controllers:
    1. We don't want to have any DOM elements nor want to know anything about the DOM

Implementing Search Results - Part 2

1. ResultsView

Implementing Pagination - Part 1

Implementing Pagination - Part 2

1. Data attributes - JS can read it and perform element specific tasks

Project Planning II

1. Core functionality is complete
2. Other features
    1. Changing servings functionality
    2. Bookmarking functionality
        1. Stored in local storage
        2. We load and render them
    3. Store bookmark data in the browser
    4. On page load, read saved bookmarks

Updating Recipe Servings

Developing a DOM Updating Algorithm

1. Code: view.js

```
update(data) {
  this._data = data;
  const newMarkup = this._generateMarkup();

  const newDOM = document.createRange().createContextualFragment(newMarkup); // converts string to real DOM node objects. Like a Virtual
  const newElements = Array.from(newDOM.querySelectorAll('*'));
  // console.log(newElements);
  const curElements = Array.from(this._parentElement.querySelectorAll('*'));
  // console.log(curElements);

  newElements.forEach((newEl, i) => {
    const curEl = curElements[i];
    console.log(curEl, newEl.isEqualNode(curEl)); // contents must be the same.

    // Updates changed TEXT
    if (
      !newEl.isEqualNode(curEl) &&
      newEl.firstChild?.nodeValue?.trim() !== ''
    ) {
      curEl.textContent = newEl.textContent;
    }

    // Updates changed ATTRIBUTES
    if (!newEl.isEqualNode(curEl)) {
      // console.log(newEl.attributes); // logs attributes property of all elements that have changed
      Array.from(newEl.attributes).forEach(attr =>
        curEl.setAttribute(attr.name, attr.value)
      );
    }
  });
}
```

Implementing Bookmarks - Part 1

Implementing Bookmarks - Part 2

Storing Bookmarks with localStorage

Project Planning III

1. Features
    1. Own recipe upload
    2. Own recipes automatically bookmarked
    3. User can only see own recipes, not from others
        1. Done using API developer key

Uploading a New Recipe - Part 1

1. API calls happen in the model

Uploading a New Recipe - Part 2

Uploading a New Recipe - Part 3

Wrapping Up: Final Considerations

1. Documentation for code
    1. Standard way to write docs for JS functions
        1. JS Docs (jsdoc.app)
            1. Examples:

```
/**
 * Render the received object to the DOM
 * @param {Object | Object[]} data The data to be rendered (e.g. recipe)
 * @param {boolean} [render=true] If false, construct markup string instead of rendering to the DOM
 * @returns {undefined | string} A markup string is returned if render=false
 * @this {Object} View instance
 * @author Jonas Schmedtmann
 * @todo Finish implementation
 */
render(data, render = true) {
  ...
}
```

1. If we hover over the function, VS Code gives an overview with the docs information

2. Challenges and improvements

Improvement and Feature Ideas: Challenges

1. Display **number of pages** between the pagination buttons
    1. User will know how many pages there are
2. Ability to **sort** search results by duration or number of ingredients - needs API changes or get all results and sort
3. Perform **ingredient validation** in view, before submitting the form
    1. Warn if format is wrong - User friendly
4. **Improve recipe ingredient input**: Separate in multiple fields and allow more than 6 ingredients
5. **Shopping list feature: button recipe to add ingredients to a list
    1. We can display the list in another hidden panel beside menu bar
    2. Contains different ingredients added from different recipes
6. **Weekly meal planning feature**: assign recipes to the next 7 days and show on a weekly calendar
    1. Dropdown menu and choose on which of the 7 days you want the recipe to appear
    2. Panel to show recipes for 7 days
7. **Get nutrition data** on each ingredient from spoonacular API (https://spoonacular.com/food-api) and calculate total calories of recipe.
    1. One recipe
    2. Per serving
8. Implementing the challenges is good to practice skills

Section 19: Setting Up Git and Deployment

Section Intro

1. Hosting on the internet
    1. Netlify
    2. Fundamentals of Git

Section Roadmap

Simple Deployment with Netlify

1. In development mode
    1. Contains dev server
    2. Code is not compressed
2. Building for production
    1. Pre-requisite:
        1. Delete `.parcel-cache` and `dist` folders
    2. Config:
        1. Version 2.x: `parcel build index.html --dist-dir ./dist`
            1. `package.json`: Change `main` to `default`

```
"default": "index.html"
```

        2. Version 1.x:

```
"out": "index.html"
```

```
npm run build
```

3. netlify.com - for static web pages (HTML, CSS, JS, images, ...) (not server side code)
    1. Free plan

        1. CI with Git
    2. Alternative: surge
        1. `npm install --global surge`
        2. `surge` - deploy without an account
    3. Deployment:
        1. Drag and drop `dist` folder into the interface
    4. Settings > Change site name
        1. `forkify-abdullah` - unique name
    5. Features
        1. Automatically secured using HTTPS (free)
        2. Site assets were deployed to a CDN (lives in many locations spread out across the world)
            1. Users get from servers closest to them
    6. Another way to deploy - Git

Setting up Git and GitHub

Git Fundamentals

Pushing to GitHub

1. New repo in Github
2. Name: forkity-course-video
    1. Private
    2. Don't add README etc...
    3. Click: Create repository
3. Copy and push
4. Add README.md file

Setting up Continuous Integration with Netlify

1. Netlify
    1. Settings
        1. Build & deploy
            1. Link site to Git (Continuous Deployment)
                1. If we push our changes, the site will be rebuilt and deployed
                2. Github (Gitlab, Bitbucket)
                3. Search: forkify-course
                4. Branch - Main
                5. Build command: parcel build index.html --dist-dir ./dist
                6. Publish directory: dist
            2. Reload the page

Section 20: The End!

Where to Go from Here

1. Practice JS skills
    1. Own small applications
    2. Find new challenges
    3. Rebuild apps and challenges
    4. Learn frameworks / Libraries
        1. React
        2. Angular
        3. View
        4. Node.js
2. To find real job in the industry
3. Never stop learning
    1. Things change very fast
4. Newsletter
    1. News
    2. Tutorials
    3. Articles

My Other Courses + Updates

1. Ultimate React Course
2. Node.js Bootcamp
3. Beginner HTML+CSS course
4. Advanced CSS course
5. @jonasschmedtmann
6. Sign up for my mailing list
    1. Gives discounts for all future courses
7. my YouTube channel

Section 21: [LEGACY] Access the Old Course

Access the Old Course